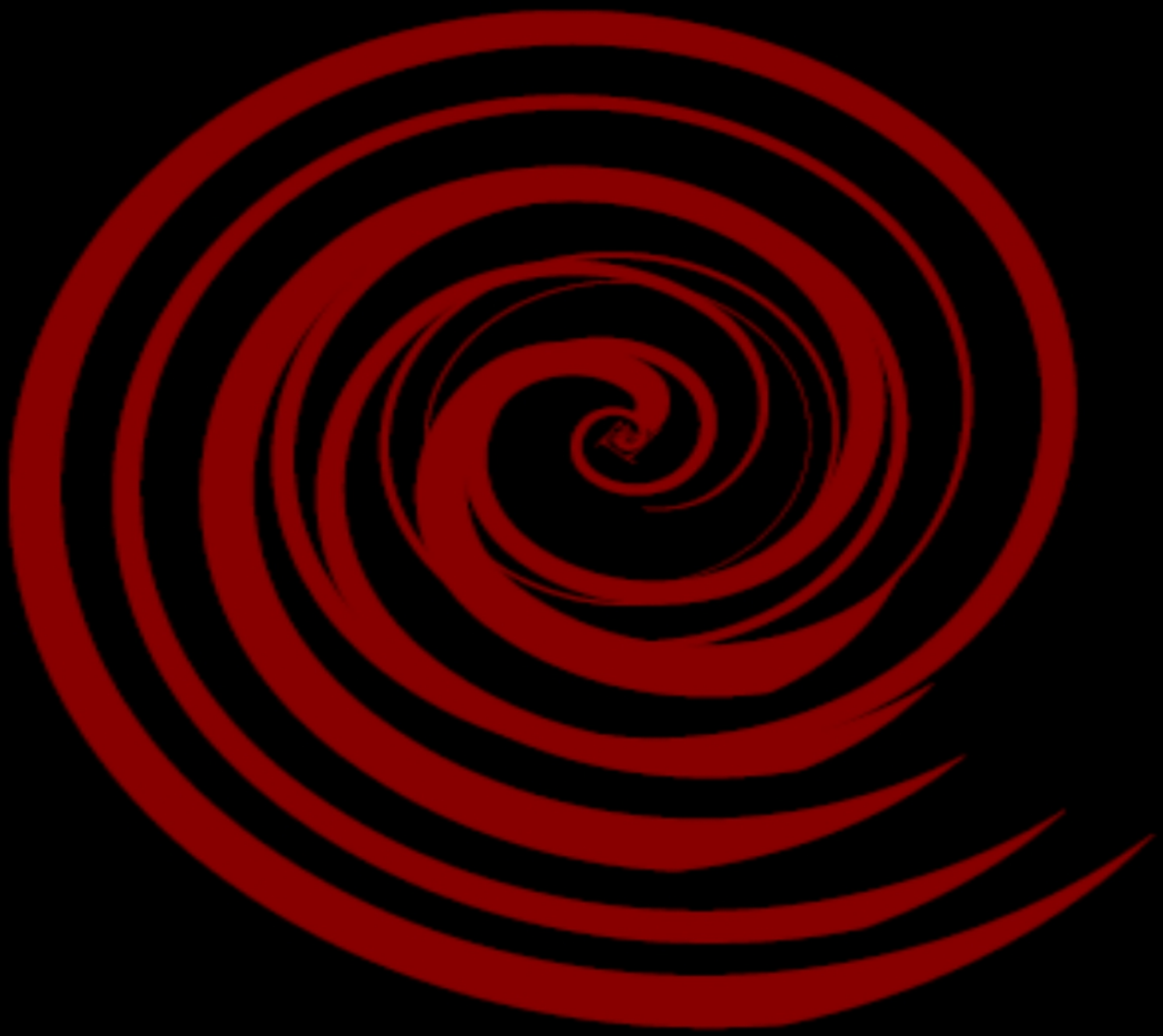
Algorithmic Trading Hands-On Approach Using Python



Ziad Francis

Algorithmic Trading Hands-On Approach Using Python

Ziad Francis

Copyright © 2024 Ziad Francis

All rights reserved.

ISBN:

To Family, Friends, and Colleagues… and all those who overlap in between these beautiful categories

CONTENTS

CHAPTER 1: PRELIMINARIES

1 Don't Skip This Introduction

2 How much can you make from algorithmic trading?

3 What about using leverage to increase returns?

4 Am I Rich, and Why Am I Still Teaching if Algorithmic Trading Works Well?

5 Algorithmic Trading or Automated Trading?

6 Why Python for Algorithmic Trading?

7 Who is this book for?

CHAPTER 2: UNDERSTANDING FINANCIAL MARKETS AND AVAILABLE DATA

1 Key Financial Instruments

1.1 Equities: A Quick Dive into Stocks, Shares, and Equity Markets

1.2 Bonds and Fixed Income: Understanding Bonds and Their Role in Portfolio Management

1.3 Forex and Currency Trading: Navigating the World of Foreign Exchange

1.4 Derivatives: Exploring Options, Futures, and Swaps

1.5 Commodities: Trading in the Global Markets of Gold, Oil, and Agriculture

2 Selecting the Right Asset for Algorithmic Trading

3 Market Orders, Types and Execution

3.1 Types of Market Orders

3.2 Order Execution

4 Liquidity and Spread

5 The Role of Market Data in Algorithmic Trading

5.1 Market Data Types

5.2 Data Providers

5.3 Data Feeds and APIs

5.4 Data Accuracy and Latency

CHAPTER 3: INTRODUCTION TO PYTHON FOR TRADING

1 Setting up your python environment

1.1 Direct installation from Python's Official Website

1.2 Integrated Development Environments (IDEs)

1.2.1 Why Choose an IDE?

1.2.2 Jupyter Notebook: Interactivity and Visualization

1.2.3 VSCode: Flexibility and Extensibility

1.2.4 Installing VSCode:

1.2.5 PyCharm: Comprehensive Features for Python Development

1.3 Installing Python Using the Anaconda Package Manager

2 Key Python Libraries For Testing Strategies

2.1 Pandas

2.2 NumPy

2.3 Matplotlib and Pyplot

2.4 Statistical Packages (Scipy, Statsmodels)

2.5 Pandas TA

2.6 TA-Lib

2.7 Backtesting.py

2.8 Scikit-learn

3 Live Testing and Connecting to Broker APIs

3.1 Choosing the Right Broker and API
3.2 Key OANDA Python Libraries
3.3 Getting Started with OANDA in Python
4 Conclusion

CHAPTER 4: GETTING AND READING FINANCIAL DATA, ADDING TECHNICAL INDICATORS, AND VISUALIZATION TIPS

1 Importing Data
1.1 Loading And Cleaning Data In Pandas
1.2 Verifying the Integrity of 'Gmt time' column
1.2.1 Check for missing values
1.2.2 Verify that the Time Values are Always Increasing
1.3 Loading And Cleaning Data with Yfinance
1.3.1 Downloading Data
1.3.2 Downloading Data for Multiple Tickers
1.3.3 Specifying a Custom Date Range
1.3.4 Downloading Intraday Data with Different Timeframes
1.3.5 Handling Missing Data and Adjustments
1.3.6 Understanding the Limitations
1.4 Streaming Data Using Oanda's API
2 Charting And Visualization
2.1 Simple Plot Line
2.2 Candlestick Chart
3 Computing Technical Indicators
3.1 Accessing Pandas TA functions
3.2 On Chart Indicators
3.2.1 Computing and Visualizing Moving Averages

3.2.2 Computing and Visualizing Bollinger Bands
3.3 Different Scale Indicators

CHAPTER 5: ADVANCED INDICATORS AND AUTOMATED PATTERNS DETECTION

1 Introduction
2 Combining and layering classic indicators
2.1 Windowed Moving Averages
2.2 Layering Windowed RSI
3 Regression And Slopes Measurements
3.1 Linear Regression
3.2 Using Regression For Trend And Momentum Detection
4 Fractals, Minimas And Maximas
5 Price Patterns Detection
5.1 Price Channels
5.2 Triangle Patterns
6 Candles Patterns Detection
6.1 Coding The Engulfing Pattern
6.2 Engulfing Variation
6.3 Coding The Rejection Pattern
7 Conclusion

CHAPTER 6: SUPPORT RESISTANCE AND FIBONACCI LEVELS

1 Introduction
1.1 Key Levels
1.2 Fibonacci Levels
2 Coding Support And Resistance

3 Coding Fibonacci Levels
4 Conclusion

CHAPTER 7: STATISTICAL EVALUATION OF TECHNICAL INDICATORS

1 Labelling Future Price
1.1 Average Future Price
1.2 Category Of Future Price
2 Testing The Rejection Candle
3 Testing The Engulfing Pattern
4 Inferential Statistics and Future Price Correlation
4.1 Descriptive Analysis
4.2 One Way ANOVA Testing
4.3 Chi-Squared Independence Testing
5 Leveraging Statistical Results

CHAPTER 8: BUILDING TRADING STRATEGIES

1 Introduction
2 Types of Trading Strategies
2.1 Scalping
2.2 Day Trading
2.3 Swing Trading
2.4 Position Trading
3 Algorithmic Trading
4 Structuring the Core Components of an Algorithmic Trading Strategy
4.1 Trend Identification and Position Filtering

4.2 Strategic Entry Points
4.3 Intelligent Exit Strategies
4.4 Managing Risk and Position Sizing
5 Trend And Entry Detection Examples
5.1 Position Entry Signal Examples
6 How To Place a Stop Loss
7 Risk And Trade Sizing
7.1 Method 1: Constant Trade Size
7.2 Method 2: Equity-Based Trade Size
7.3 Method 3: Risk-Reward Ratio-Based Trade Size
7.4 Balancing Risk and Reward
8 Conclusion

CHAPTER 9: BACKTESTING TRADING STRATEGIES

1 Introduction
2 Case Study 1: Daily Support And Resistance Strategy
3 The Backtesting.Py Package
4 Understanding Backtesting Reports
5 Backtest Quality Ratios: Sharpe, Sortino, and Calmar
5.1 Sharpe Ratio
5.2 Sortino Ratio
5.3 Calmar Ratio
5.4 Understanding the Metrics Together
6 Case Study: Backtesting With Fixed Stop Loss And Take Profit Values
7 Case Study: Backtesting With ATR Related Stop Loss Levels
8 Case Study: Backtesting Using A Trailing Stop

9 Case Study: Break Even Dynamic Management Approach
10 Visualizing The Backtest Data
11 Conclusion

CHAPTER 10: ROBUSTNESS ANALYSIS USING THE MONTE-CARLO METHOD

1 Introduction
2 Applying Normally Distributed Blur
3 Visualizing Monte-Carlo Results
3.1 Equity Curves
3.2 Visualizing Distribution of Final Outcomes
4 Conclusion

CHAPTER 11: AUTOMATED TRADING SYSTEMS

1 Introduction
2 Automating trading alerts
3 Automating Alerts for Multiple Assets
4 Fully Automated Trading Approach
5 Conclusion

CHAPTER 12: FINAL THOUGHTS

1 Personal Experience, Lessons Learned, and Future Directions
2 The Intersection of Coding, Trading, and Learning
3 The Role of AI in the Future of Algorithmic Trading
4 How to Stay Updated in the Fast-Paced World of Algorithmic Trading
5 Summary of Key Points

6 Final Thoughts
7 Additional Features

Chapter 1: Preliminaries

1 Don’t Skip This Introduction

If you are a technical person or someone aiming to develop technical skills for trading, you might feel inclined to skip this introduction. It’s understandable, your focus is likely on the strategies, the coding, and the automation aspects that make algorithmic trading so powerful and intriguing. After all, these are the core elements that promise to transform how you approach trading and potentially lead to better results, more profit and more money.

However, before we jump into the technicalities, I want to take a moment to address some foundational ideas. This introduction is more than just a formality; it's an essential part of setting the stage for everything that follows and mainly manage expectations. Over the years, I've taught many students and interacted with numerous individuals interested in algorithmic trading. A common thread among them is the presence of certain questions and concerns, questions that often go unaddressed in purely technical discussions.

What is algorithmic trading really about? How does it differ from traditional and automated trading? Is Python truly the best language for this field? And most importantly, what mindset should one have when stepping into the world of algorithmic trading? These are just a few of the questions that tend to arise.

By taking a few moments to explore these topics, I hope to clear up any confusion and provide a solid foundation as we delve deeper into the technical aspects of algorithmic trading. This understanding will not only enhance your learning experience but also ensure that you approach the subject with the right perspective and the right expectations, ready to tackle

the challenges and complexities that come with it.

So, even if you're eager to get to the coding and strategy development, I encourage you to stick with me through this introduction. It may just answer some of the questions you didn’t even know you had and set you up for a better understanding as you move forward.

2 How much can you make from algorithmic trading?

This is, without a doubt, one of the most common questions I receive from students, and it’s completely understandable. If you’re about to invest your time, money, and energy into learning the intricacies of trading and then translating that knowledge into code, you naturally want to know if it’s going to be worth it in the long run. For many, the idea of dedicating months or even years to mastering this field can seem daunting especially when they’re hoping for quick returns. The desire to see results quickly is human nature, after all. But here's the truth: the reality of algorithmic trading, like any worthwhile investment, is that it requires patience, persistence, and realistic expectations.

So, how much can you realistically expect to make from algorithmic trading or trading in general? The answer might be frustrating, but it’s one you’ll hear often: “It depends.” While this might sound like I’m dodging the question, it’s actually the most honest response. There are so many variables at play; market conditions, the quality of your strategy, your risk management, and even luck—that giving a definitive number would be misleading.

Let’s take a step back and look at one of the most successful examples in this field: Renaissance Technologies, the firm behind the Medallion Fund. These are the true pioneers, the "Wizards of

Wall Street" who were using artificial intelligence and sophisticated mathematical models long before these concepts became mainstream. But even with their vast resources, top-tier talent, and decades of research, it took Renaissance over ten years of hard work to start seeing significant profits. And by significant, I don't mean the kind of overnight success that some social media ads promise. They weren't turning $100 into $5 million in a matter of weeks like that guy I saw on social media selling courses. Today, Renaissance Technologies averages around 62% returns per year, with their best year reaching an impressive 98% in 2000. These numbers are phenomenal, but they're the result of years—decades even—of relentless research and development.

Now, let's bring it back to what you can realistically expect. I'm not suggesting that everyone can achieve the kind of success that Renaissance Technologies has. But with a couple of years of dedicated research, learning, and refinement of your trading strategies, it's possible to build a robust model that could yield returns anywhere in the range of 15-30% annually. And let me tell you, that's already an excellent outcome. Why? Because once you have a model that consistently delivers such returns, you're in a position to scale, to get hired or to be recruited as a freelancer among other opportunities.

For an individual trader, a 30% return on a modest investment might not seem like a life-changing sum. But imagine the power of that same 30% when applied to larger sums of money, the kind that institutions, hedge funds, and industry players can access. While a single trader might be limited in the amount they can invest, an institution with millions of dollars at its disposal can see substantial profits from that same 30% growth.

But here's where I want to manage expectations honestly. If it were possible to simply write a program, sit back, and watch it churn out millions each week, everyone would be doing it. This

field of algorithmic trading would be far more popular, and the barriers to entry would be much lower. The reality is that successful algorithmic trading requires a deep understanding of both trading and coding, coupled with rigorous testing and constant refinement. It's not a magic bullet, and it won't make you rich overnight. Anyone who tells you otherwise is selling you a fantasy.

Back to our original question: how much can you expect to make from algorithmic trading? A realistic expectation is to aim for returns in the range of 15-30% annually, depending on your initial investment and your strategy's robustness. For example, a 20% annual return might break down to about 2% per month. While this isn't the kind of return that will turn you into a millionaire in a short time, it's steady, sustainable, and more importantly, achievable.

3 What about using leverage to increase returns?

Now, let's address the topic of leverage, which often comes up in discussions about increasing returns. Leveraging can indeed amplify your gains, but it also magnifies your risks. For instance, if you're working with a leverage of 1,000 on a $100 investment, and your system is successful, you could theoretically see a return of $20,000 by the end of the year, considering a 20% yearly return. However, and this is crucial to understand, leverage works both ways. A small negative move in the market can wipe out your entire investment when using such high leverage. That's why I typically advise caution with leverage. In my systems, I generally try not to exceed a leverage ratio of 10, and I typically use a ratio of 5. This keeps things more manageable and helps to limit risk while still offering the potential for significant returns.

But there's more to leverage than just the numbers. The psychology behind using high leverage is often tied to greed, a powerful emotion that can cloud judgment and lead to reckless decisions. I speak from experience when I say that high leverage can be a dangerous temptation. When I first started trading over a decade ago, I was eager to make big gains quickly. The idea of using leverage to multiply my returns was incredibly appealing. I convinced myself that with the right strategy, I could handle the risks and turn small investments into substantial profits in no time.

Unfortunately, the reality was much harsher. My overconfidence, fueled by greed, led me to take on more leverage than I could manage. At first, things seemed to be going well, but it didn't take long for the market to turn against me. A few small negative movements wiped out my account, leaving me not only with financial losses but also with the painful realization that I had underestimated the risks. The experience was a hard lesson in the dangers of high leverage and the destructive power of greed.

The temptation to use high leverage comes from the desire for quick success, after all, who doesn't want to see their account balance skyrocket everyday? But the truth is, high leverage is a double-edged sword. While it can lead to significant gains, it can just as easily lead to devastating losses. And the higher the leverage, the thinner the margin for error. Greed can make it seem like the rewards justify the risks, but more often than not, it's the greed itself that leads to failure.

Looking back, I realize that those early losses were some of the most valuable lessons I learned in trading. They taught me the importance of discipline, risk management, and the dangers of letting emotions drive decisions. That's why, today, I approach leverage with caution and always recommend others do the same. A leverage ratio of 5 or 10 may not seem as exciting as 1,000, but it allows for more controlled, sustainable growth.

More importantly, it helps protect your capital, ensuring that you can stay in the game long enough to see your strategies pay off.

One of the great advantages of algorithmic trading is the ability to backtest your strategies in a risk-free environment before ever putting real money on the line. By backtesting the same strategy twice; once with low leverage and once with high leverage, you can clearly see the impact leverage has on your trading results. Typically, you'll notice that while high leverage might boost potential profits, it also leads to significantly higher drawdowns, meaning your strategy experiences deeper losses during unfavorable market conditions. This is something you'll be able to experiment with easily as you progress through this book, using Python to put your trading strategies to the test. This ability to simulate and analyze different scenarios is one of the powerful reasons why algorithmic backtesting is so valuable, it allows you to understand the risks and refine your approach without exposing yourself to real-world losses.

4 Am I Rich, and Why Am I Still Teaching if Algorithmic Trading Works Well?

This is another one of the most frequently asked questions I get, and I totally understand why. It's natural to wonder, if trading is so lucrative, why do I still need to work, sell courses, teach, or write books? If algorithmic trading is as effective as I say, shouldn't I be lounging on a beach somewhere, living off my trading profits?

Let me start by saying that I'm living a comfortable life. But if your idea of "rich" involves driving supercars, living in a mansion, or jetting off to exotic locations on private planes, then no, that's not my reality. I don't have a supercar parked in my

driveway, I don't live in a palace, and I'm not traveling around the world in luxury jets every week. And, to be honest, most successful traders I know aren't living that lifestyle either. It's a stereotype that's been glamorized in movies and media, but it's far from the reality for most people who work in this field. Of course, there are always a few exceptions who have scaled their businesses and significantly increased their returns, but I'm speaking about the common experience shared by the majority.

The truth is, trading, even when done well, comes with its own set of challenges and limitations. Sure, with a solid algorithmic trading system, you can achieve consistent returns, but those returns are usually capped at a certain percentage each year. For individual traders, this might mean a steady income, but it's not necessarily life-changing wealth. And that's okay because trading is just one piece of the puzzle.

But let's get a bit more personal here. Sometimes, no matter how well you trade or how much money you make, life can throw unexpected challenges your way that can set you back significantly. I've experienced this firsthand. For example, during the economic crisis in Lebanon, many people, including myself, saw their savings wiped out by the banks. This wasn't just a small setback, this included retirement funds, insurance, and all the savings we had worked so hard to build. It was a harsh reminder that no matter how skilled you are at trading, external factors can have a profound impact on your financial security. It's not just about making money, it's also about how you manage and protect that wealth, where you invest it, and how safe your overall portfolio is.

So, why do I sell books and courses if I'm successful at trading? The answer is simple and practical. Trading returns, while steady, are often limited to a certain percentage per year. For an individual trader, these returns might be modest, and it's not uncommon for traders to discover that they can earn more by engaging in business activities related to trading, rather than

just trading alone. Teaching, writing, and consulting are natural extensions of the skills and knowledge I've built over the years, and they offer additional income streams that complement my trading.

In my own case, after years of experience, I now make more from my day job than from trading alone. So, it's no surprise that I've chosen to limit my trading time and focus more on my career, freelancing, preparing courses, and writing books. These activities not only provide a reliable income but also offer me a sense of fulfilment and satisfaction that goes beyond the financial rewards. I've found that the cumulative income from these activities is not only less stressful but also more sustainable in the long run. It's less risky, there's no anxiety over market crashes or volatility, no unexpected surprises to constantly manage, and in some cases, it's even more lucrative.

There's another aspect to this that's worth mentioning, trading isn't for everyone. It's a field that requires a unique blend of skills, patience, and a certain tolerance for risk and uncertainty. Some people thrive on the challenge of tuning algorithms, monitoring markets, and constantly adjusting to new conditions. For them, trading is not just a job, it's a passion. But others, including myself at times, find that different activities can be more fulfilling. For instance, teaching, writing, and developing new projects allow me to share my knowledge, connect with others, and continue learning in a way that trading alone doesn't.

Over the years, I've come to realize that my interests and priorities have evolved. When I was younger, trading was a central focus of my life, and I spent countless hours fine-tuning strategies and watching the markets. But as I've advanced in my career and received more opportunities for freelancing and consulting, I've found that I enjoy the variety and balance that comes from doing a mix of things. Trading is still part of my life, but it's no longer the center of it. I trade much less than I used

to, fitting it in when my workload slows down and I have time to study the market and tweak my algorithms.

In the end, the decision to diversify my activities was driven by a desire for a more balanced and sustainable lifestyle. Trading full-time can be incredibly rewarding, but it can also be stressful and unpredictable. By expanding my focus to include teaching, writing, and other ventures, I've been able to create a career that not only supports me financially but also aligns with my values and interests.

So, to answer the question: Yes, algorithmic trading works, and it can be profitable. But for me, the choice to teach is about creating a fulfilling and balanced life, one where I can continue to grow and share my knowledge with others, while still enjoying the financial benefits that come from trading. Trading remains a part of my journey, but it's just one of many paths I've chosen to explore.

5 Algorithmic Trading or Automated Trading?

Algorithmic trading is the use of computer algorithms, essentially, step-by-step instructions or rules, to execute trading orders in financial markets. These algorithms are designed to follow predefined strategies, such as buying or selling at certain price levels, and they can process a vast amount of market data in real-time to make decisions much faster than a human could. The key advantage of algorithmic trading is its ability to minimize human error, manage large volumes of data, and execute trades at optimal times, all while adhering strictly to the strategy it was programmed to follow.

However, it's important to differentiate between fully automated trading and algorithmic trading that serves as an assistant to human traders. Fully automated trading systems are

designed to operate with little to no human intervention. Once the algorithm is set in motion, it runs autonomously, executing trades according to its programming, even if the market conditions change rapidly. This approach is powerful, as it can remove emotional biases from trading decisions and allows for consistent execution. However, it also comes with its own set of risks, since the system follows its rules without question, it may not always adapt well to unexpected market events.

On the other hand, algorithmic trading as an assistant to human traders is a bit different. In this setup, the algorithms are still responsible for analyzing data and generating signals or even executing trades, but there's an ongoing role for the human trader. The trader might review the algorithm's signals, override certain trades based on their insights, or adjust the parameters in response to changing market conditions. This approach blends the efficiency and speed of algorithms with the intuition and experience of a human trader, offering a more flexible and adaptive trading experience. While it doesn't eliminate the possibility of human error entirely, it does allow for a more hands-on approach where the trader remains actively engaged in the decision-making process.

While both fully automated and assisted algorithmic trading leverage the power of technology to improve trading outcomes, they cater to different needs and preferences. Fully automated systems are ideal for those who want to set and forget, letting the machine handle everything, whereas assisted algorithmic trading is suited for those who prefer to stay involved and use algorithms as a tool to enhance their own trading strategies, and this is usually the better way to maximize the returns.

6 Why Python for Algorithmic Trading?

When I first discovered that trading could be programmed into a set of rules and automated through code, I was introduced to MQL5, the scripting language for MetaTrader. At the time, MQL5 was very similar to C, a language I was already using in my daily work. It was a thrilling discovery, and I quickly began developing what were then called "expert advisors", essentially trading bots that ran on MetaTrader. The ability to automate trading strategies opened up a whole new world of possibilities, and I was eager to explore it.

However, as I continued to delve into the field of trading, I eventually discovered Python. At that point, Python had already become a powerful tool in the data science community, a field in which I also worked first as an instructor then as a consultant for a large firm. Python's versatility and the breadth of its libraries piqued my curiosity, especially when I started to wonder if machine learning could enhance technical indicators and provide smarter forecasting tools for financial markets. This curiosity led me to gradually transition from MQL5 to Python.

Since making the switch, my interest in MQL5 has waned, and I've lost touch with its current capabilities. While I'm not sure if neural networks can be implemented using MQL5 today, I know for certain that Python has everything needed to build some of the most complex and sophisticated financial trading systems imaginable. Python's ecosystem is incredibly rich, offering a vast array of libraries and tools that make it possible to do just about anything in algorithmic trading.

For instance, Python supports the development of basic and advanced machine learning models, including supervised and unsupervised learning, clustering, regression, and ensemble methods like XGBoost. If you're into statistical analysis, Python

has you covered, from basic statistics to inferential techniques, correlation studies, and beyond. And when it comes to deep learning, Python's capabilities are even more impressive, with libraries that support neural networks, long short-term memory (LSTM) networks, and even cutting-edge architectures like Transformers.

For those who are more interested in futuristic approaches, Python also offers tools for reinforcement learning, allowing you to experiment with systems that learn and adapt over time. The beauty of Python lies in its ability to import pre-coded modules and packages, giving you access to powerful functions without having to code everything from scratch. This makes Python not just a programming language, but a comprehensive toolkit that can handle virtually any challenge you throw at it in the world of algorithmic trading.

The trading industry has recognized Python's potential, with many brokers now offering Python APIs. This means you can bypass traditional trading platforms entirely and manage your trades, opening positions, closing positions, setting stop-losses, trailing stops, and take-profit levels, all directly from within Python. This is particularly advantageous for fully automated trading systems. Brokers like Oanda, Binance, and Interactive Brokers are just a few examples of those who provide Python API support for automated and algorithmic trading.

Another significant advantage of using Python in algorithmic trading is the wealth of packages available for backtesting strategies. Before deploying a strategy live in the market, you can backtest it on historical data to evaluate how it would have performed under past market conditions. This allows you to estimate potential returns, assess maximum and average drawdowns, and calculate key performance metrics like the Sharpe ratio and the Sortino ratio, critical indicators of a strategy's viability before you risk real money.

Python's powerful libraries, extensive community support,

and growing acceptance within the trading industry make it an ideal choice for algorithmic trading.

7 Who is this book for?

This book is designed for anyone who wants to delve into the world of algorithmic trading, but it's important to note upfront that some basic knowledge of Python is required to fully engage with the examples and exercises provided. We won't be covering the basics of Python programming here; instead, we'll be focusing on what really matters, building trading strategies, backtesting them, and deploying them live in the market using broker APIs.

If you're already comfortable with Python and are eager to apply your skills to the financial markets, this book is for you. It's tailored for beginners in algorithmic trading, so even if you're new to this specific domain, you'll find the content accessible and practical. This book will guide you step-by-step through the process of transforming your trading ideas into automated strategies that can run independently, whether you're sitting at your desk or not.

This book is also perfect for traders who are looking to automate part or all of their trading systems. If you've been manually trading and want to leverage the power of automation to execute your strategies more efficiently, reduce human error, or simply free up more of your time, you'll find the tools and techniques you need right here. We'll explore how to build algorithms that can manage trades according to predefined rules, analyze market data in real-time, and adapt to changing conditions, without constant manual input.

In addition, if you're a developer or data scientist with an interest in finance, this book will help you bridge the gap between coding and trading. You'll learn how to apply your technical skills to create trading algorithms, and how to integrate these with real-world financial markets using APIs provided by brokers.

For those who are curious about the intersection of finance, technology, and data science, this book offers a practical, hands-on approach to understanding how algorithmic trading works and how you can use Python to take your trading to the next level. If you aim to build fully automated trading systems or simply want to enhance your existing trading strategies with algorithmic support, this book will provide the foundation you need.

In brief, this book is for:

- Beginners in algorithmic trading who have a basic understanding of Python and want to learn how to build, test, and deploy trading algorithms.
- Manual traders looking to automate part or all of their trading processes to improve efficiency and reduce the emotional impact of trading.
- Developers and data scientists interested in applying their coding skills to the financial markets.
- Anyone curious about algorithmic trading who wants a practical guide to understanding and implementing trading strategies using Python.

I hope this introduction has addressed some of the questions you may have had about algorithmic trading and set the stage for what's to come. With a clearer understanding of the key concepts and the tools we'll be using, we're now ready to dive into the exciting world of financial markets. In the next chapter, we'll explore the fundamentals of financial markets, setting the foundation before we move on to the technical aspects in the chapters that follow. Let's get started!

Chapter 2: Understanding Financial Markets And

Available Data

In this chapter, we're setting the stage by diving into the fundamentals of financial markets. Before we get into the nitty-gritty of building and deploying trading strategies, it's important to have a strong grasp of the markets you'll be operating in. This understanding is crucial for guiding your decisions, helping you pick the right assets to trade, and selecting the markets that align with your strategies. Without this foundation, even the most advanced algorithms can end up directionless, leaving you reliant on luck rather than informed decisions.

Financial markets are incredibly vast, offering a wide range of instruments and opportunities, from equities and bonds to currencies and commodities. This diversity is what makes trading exciting, but it can also be overwhelming, especially if you're just starting out. With so many options, having a clear understanding of the different types of markets and the assets traded within them is essential. This knowledge will help you narrow down your choices to those that best fit your trading goals and risk tolerance.

The goal of this chapter is to highlight the importance of choosing the right market for your trading activities. Your decision should be influenced not just by your trading style, but also by your environment, location, and the news you're regularly exposed to. For example, a farmer might naturally have insights into the supply and demand of certain commodities. This knowledge makes trading these commodities easier because they're already part of the ecosystem that drives prices, they don't need to spend hours researching the market online because they live it every day.

This idea applies to any field. For instance, if you work in electronics or nanotechnology, you're constantly exposed to the latest developments and news in that industry. This makes it easier for you to trade stocks in electronics or software

companies because you're already in the loop, news reaches you naturally, giving you an edge in making informed trading decisions. Similarly, someone working in import and export might have access to supply and demand data, helping them anticipate seasonal trends and trade more effectively.

The key takeaway here is that your choice of market and assets should align with what you're already comfortable with and knowledgeable about. This approach can save you time on daily research. For example, I've lived in a country where the U.S. dollar, euro, and British pound are commonly used, and currency exchange between them is a part of daily life. As a result, I find it more comfortable to trade these forex pairs because I'm frequently exposed to their exchange rates and have a deep understanding of their historical movements.

If you're venturing into a new market that you're not familiar with, it's important to immerse yourself in relevant forums, online discussions, and news sources. Spend some time observing how the asset behaves over time. While it doesn't require a massive time investment, it's a necessary step to become comfortable and informed about your chosen market.

Think of this chapter as your roadmap to the financial landscape. Just as a traveler needs to understand the terrain before setting out on a journey, a trader needs to grasp the market environment before diving into algorithmic trading. Familiarizing yourself with the basics will help you decide which markets to focus on and what assets to trade, ultimately increasing your chances of success.

For example, if you're interested in equities, you'll need to understand how stock markets operate, what influences stock prices, and how major indices like the S&P 500 or NASDAQ provide insights into market performance. This knowledge will help you choose the right stocks for your trading strategies and identify the best times to enter or exit positions.

On the other hand, if you're drawn to the fast-paced world of

currency trading, you'll need a solid understanding of the forex market. This includes knowing how currency pairs are quoted, what drives exchange rates, and how economic indicators and geopolitical events can cause significant price movements. The forex market operates 24 hours a day, offering numerous trading opportunities, but it also requires a good grasp of global economic dynamics to trade effectively.

Commodities, bonds, and derivatives present yet another set of opportunities and challenges. Commodities like gold, oil, and agricultural products are influenced by supply and demand dynamics, geopolitical tensions, and environmental factors. Bonds, which represent debt obligations, are affected by interest rates, credit risk, and economic conditions. Derivatives, including options and futures, allow traders to hedge risks or speculate on price movements without owning the underlying asset. Each of these markets has its own unique characteristics and understanding them is crucial as you develop your trading strategies.

While the variety of market options might initially seem overwhelming, in the context of algorithmic trading, especially for the purposes of this book, these differences become less significant. Our analysis will be primarily technical, meaning that most charts will look the same regardless of the asset or market. The core principles we apply will remain consistent across different markets.

However, there are some differences that could be crucial, such as the way data is serialized, which affects how volume information is streamed to the trader. For instance, it's important to consider whether you're dealing with local volume or global volume, and whether valid order book data or level 2 data is available. These factors can influence the granularity and accuracy of your analysis, particularly in markets where depth of market information is critical.

Apart from these considerations, most other differences

between markets won't be of primary concern to us. The technical strategies we'll develop can be applied broadly, regardless of the specific market or asset. This approach simplifies the process and allows you to focus on building, testing, and refining your algorithmic trading strategies without getting bogged down by market-specific details that don't directly impact your technical analysis.

To ease your entry into algorithmic trading, you might start by exploring more popular and accessible markets like Forex or equities. These markets are well-established, highly liquid, and offer a wealth of data for backtesting and analysis, making them ideal starting points for developing and honing your trading algorithms. As you gain experience and confidence, you can consider branching out into other areas, such as commodities or derivatives, depending on your interests and the strategies you wish to develop.

1 Key Financial Instruments

1.1 Equities: A Quick Dive into Stocks, Shares, and Equity Markets

Equities, commonly referred to as stocks or shares, represent ownership in a company. When you buy a share, you're essentially purchasing a small piece of the company, giving you a claim on part of its assets and earnings. This concept is foundational to equity markets, where companies raise capital by issuing shares to the public, and investors trade these shares on stock exchanges like the New York Stock Exchange (NYSE).

In this section, we'll explore how stocks are traded, the role of stock indices, and the importance of dividends and earnings.

Stock trading occurs in two primary markets: the primary market, where companies issue new shares through initial public offerings (IPOs), and the secondary market, where existing shares are bought and sold among investors. Understanding how these markets function is crucial for anyone looking to trade equities, as market conditions, investor sentiment, and economic indicators can all influence stock prices.

Stock indices, such as the S&P 500, Dow Jones Industrial Average, and NASDAQ Composite, play a significant role in equity markets. These indices track the performance of a group of selected stocks, providing a snapshot of overall market health. Investors and traders often use indices as benchmarks to compare the performance of their portfolios or to gauge market trends. Additionally, indices are instrumental in the creation of index funds and exchange-traded funds (ETFs), which offer a way to invest in a diversified portfolio of stocks without selecting individual companies.

Dividends and earnings are critical components of equity valuation. Dividends are payments made by a company to its shareholders, usually from profits. They provide investors with a steady income stream and can be a sign of a company's financial health. Earnings, on the other hand, represent the company's profitability and are a key factor in determining a stock's price. Earnings reports, released quarterly, can significantly impact stock prices, as they provide insight into the company's financial performance and future prospects.

Understanding these aspects of equities is essential for developing effective trading strategies. For instance, we can analyze market trends, company fundamentals, and economic indicators, and so we can make informed decisions about when to buy or sell stocks. Whatever you're looking to invest in, blue-chip companies for long-term growth or short-term trading, understanding how equity markets operate will be of help and following specific quarterly reports will definitely give you an

advantage.

1.2 Bonds and Fixed Income: Understanding Bonds and Their Role in Portfolio Management

Bonds are debt securities issued by governments, municipalities, or corporations to raise capital. When you purchase a bond, you are essentially lending money to the issuer in exchange for regular interest payments (known as the coupon) and the return of the bond's face value when it matures. Bonds are a cornerstone of the fixed-income market, offering a more stable and predictable income stream compared to equities.

In this section, we will discuss the different types of bonds, including government bonds, corporate bonds, and municipal bonds. Government bonds, such as U.S. Treasury bonds, are considered one of the safest investments because they are backed by the full faith and credit of the issuing government. Corporate bonds, issued by companies, offer higher yields but come with greater risk, as they depend on the issuing company's ability to repay its debt. Municipal bonds, issued by states, cities, and other local governments, often offer tax-free income to investors but also carry varying levels of risk depending on the issuer's financial health.

Key concepts such as yield, and duration are vital to understanding bond investments. Yield represents the return an investor can expect from a bond, typically expressed as a percentage of the bond's price. Duration, on the other hand, measures the sensitivity of a bond's price to changes in interest rates. A bond with a longer duration is more sensitive to interest rate changes, which can affect its price and yield.

Credit ratings play a crucial role in the bond market, providing investors with an assessment of the issuer's

creditworthiness. Ratings agencies like Moody's, Standard & Poor's, and Fitch assign ratings based on the issuer's financial health and ability to repay its debts. Bonds with higher ratings (investment-grade bonds) are considered safer, while lower-rated bonds (junk bonds) offer higher yields but come with increased risk.

So, bonds are more of an investment approach, I wouldn't call it trading really but still they play a role in portfolio management, and they might be an essential part for traders looking to diversify their investments and manage risk. Bonds can provide a stable income stream, reduce portfolio volatility, and act as a hedge against equity market downturns. By analyzing yield curves, interest rates, and credit ratings, traders can get an idea about bond investments, and maybe forecast the potential returns on investment within a limited amount of time.

1.3 Forex and Currency Trading: Navigating the World of Foreign Exchange

The foreign exchange market, or forex, is the largest and most liquid financial market in the world, with trillions of dollars traded daily. Forex trading involves buying and selling currency pairs, where one currency is exchanged for another. The value of a currency pair fluctuates based on various factors, including interest rates, economic data, geopolitical events, and market sentiment.

Currency pairs are quoted in terms of one currency's value relative to another, with the first currency in the pair known as the base currency and the second as the quote currency. For example, in the EUR/USD pair, the euro is the base currency, and the U.S. dollar is the quote currency. The price of the pair indicates how much of the quote currency is needed to purchase one unit of the base currency.

Central banks play a significant role in the forex market by setting interest rates and engaging in monetary policy. For example, if a central bank raises interest rates, it can attract foreign capital, leading to an appreciation of the country's currency. Conversely, lowering interest rates can weaken the currency as investors seek higher returns elsewhere.

Economic indicators, such as GDP growth, employment data, inflation rates, and trade balances, also have a profound impact on currency values. Traders must stay informed about these indicators and their potential effects on the forex market. Additionally, geopolitical events, such as elections, trade disputes, and conflicts, can cause significant volatility in currency prices, presenting both opportunities and risks for traders.

Forex trading offers unique advantages, including 24-hour market access and high liquidity, allowing traders to enter and exit positions quickly. However, it also requires a deep understanding of global economics and the ability to manage risk effectively. A trader needs to understand the basics of forex trading, how to analyze market trends, and the impact of economic data, to be able to develop strategies to capitalize on currency price movements.

1.4 Derivatives: Exploring Options, Futures, and Swaps

Derivatives are financial instruments whose value is derived from an underlying asset, such as a stock, bond, commodity, or currency. Common derivatives include options, futures, and swaps. These instruments are widely used for hedging, speculation, and leveraging trading positions, offering traders a way to manage risk or profit from price movements without owning the underlying asset.

Options give the holder the right, but not the obligation, to buy or sell an asset at a predetermined price before a specified

expiration date. There are two types of options: calls and puts. Call options give the holder the right to buy an asset, while put options give the holder the right to sell. Traders use options to speculate on the direction of an asset's price or to hedge against potential losses in their portfolios.

Futures are standardized contracts that obligate the buyer to purchase, and the seller to sell, an asset at a predetermined price and date in the future. Futures are commonly used in commodity markets, where producers and consumers hedge against price fluctuations. For example, a farmer might use futures contracts to lock in the price of their crop, protecting against the risk of falling prices at harvest time. Traders can also use futures to speculate on price movements, taking advantage of leverage to amplify potential returns.

Swaps are agreements between two parties to exchange cash flows or financial instruments over a specified period. The most common type of swap is an interest rate swap, where one party agrees to pay a fixed interest rate while receiving a variable interest rate from the other party. Swaps are often used by companies to manage interest rate risk or to achieve a more favorable cost of borrowing.

Derivatives are powerful tools that can enhance a trader's ability to manage risk, speculate on price movements, and leverage their positions. However, they also come with significant risks, including the potential for large losses if the market moves against the trader's position.

The strategies for trading derivatives are quite different from those used in stocks and forex trading, and understanding the associated risks is crucial for any trader considering these instruments. Personally, I've never traded derivatives because once you become accustomed to a particular way of trading in a specific market, everything you do, from the coding libraries you use to how you analyze charts and design strategies, becomes tailored to that market. Shifting to a different market with its

own set of strategies and tools requires adapting your entire approach, which can be quite challenging. Even when using a flexible language like Python, transitioning to a new trading environment involves significant effort. Given that the potential returns might be similar across markets, it often doesn't seem worth the investment of time and energy to make such a move.

1.5 Commodities: Trading in the Global Markets of Gold, Oil, and Agriculture

Commodities are raw materials or primary agricultural products that can be bought and sold, such as gold, oil, and wheat. Trading in commodities offers unique opportunities and challenges, as prices are influenced by a wide range of factors, including supply and demand dynamics, geopolitical events, weather conditions, and economic indicators.

In this brief section, we'll explore the basics of trading in commodity markets, focusing on key products like precious metals, energy resources, and agricultural goods.

Gold, is often considered a safe-haven asset, influenced by factors like inflation, interest rates, and geopolitical instability. Traders typically use gold as a hedge against economic uncertainty and currency devaluation. It's important to note that some assets lend themselves well to technical trading, where traders rely mainly on chart analysis, using indicators and an events calendar to avoid periods of high volatility that could lead to unpredictable price movements. However, gold is different. Geopolitical events can have a significant and immediate impact on its price. For example, at the time of writing this book, the war in Ukraine and increased gold investments from countries in the Far East pushed gold prices to record highs in a very short period. Many traders I know, who invest as a side hustle, were buying gold, and my colleagues even struggled to find physical

gold on the market. Bullion and gold pieces were in short supply, and sellers were sometimes reluctant to part with their stock, preferring to wait a few weeks to sell at an even higher price.

What I'm trying to emphasize is that some assets, like gold, are better traded with a focus on news and an understanding of geopolitical and economic situations. As a technical trader, I find it much easier to analyze charts, write code that detects patterns, and automate signals or trades. However, Python is versatile enough that you could also set up a web scraper to gather news, analyze it with sentiment analysis algorithms, and assess the market's mood, whether positive or negative, towards a particular asset.

In my opinion, trading or investing in gold is more suited to a fundamental approach, where decisions are based on a broader understanding of global events. This approach tends to be slower and more long-term, with trades potentially staying open for weeks or even months, rather than the rapid in-and-out style of day trading. But this is just my personal view, and many traders might disagree. That’s the beauty of trading, it's highly personal. There are many different strategies that work, and that diversity is what makes trading so fascinating. Even successful traders are always curious about how others approach the market because there is always something new to learn in this field.

Oil is a crucial energy resource, and its price is affected by global supply and demand, geopolitical tensions, and decisions made by major producers like OPEC (Organization of the Petroleum Exporting Countries which is an intergovernmental organization). The oil market is highly volatile, offering traders opportunities for significant profits but also presenting substantial risks. Understanding the factors that influence oil prices, such as production levels, inventory reports, and economic growth, is essential for successful trading.

Agricultural commodities include products like wheat,

corn, soybeans, and coffee. These markets are heavily influenced by weather conditions, crop yields, and government policies. Traders in agricultural commodities must stay informed about factors such as planting and harvest seasons, climate patterns, and global trade agreements.

Supply and demand dynamics play a central role in determining commodity prices. When supply exceeds demand, prices tend to fall, while a shortage of supply relative to demand can drive prices higher. Geopolitical events, such as conflicts, trade disputes, and sanctions, can also disrupt supply chains and impact commodity prices. Additionally, economic indicators, such as GDP growth, industrial production, and consumer spending, influence the demand for commodities and, consequently, their prices.

Trading commodities requires a deep understanding of global markets, the ability to analyze supply and demand factors, and the skill to manage risk effectively. Commodities can add diversification to a trader's portfolio, providing exposure to different sectors of the global economy.

2 Selecting the Right Asset for Algorithmic Trading

So, when it comes to choosing between equities, bonds, derivatives, forex, and commodities for algorithmic trading, which are the most suitable? It's clear that forex and equities are at the top of the list. These markets are often considered the best fit for algorithmic trading due to their high liquidity, extensive data availability, and the presence of price trends that can be effectively analyzed through computational methods. Derivatives also offer significant opportunities, given their complexity and leverage, while bonds and commodities might require more sophisticated models that integrate a broader range

of data inputs.

Equities (Stocks) often exhibit high volatility and trading volume, which creates numerous opportunities for algorithmic trading strategies like statistical arbitrage and momentum trading. The frequent price fluctuations in the stock market provide a fertile ground for algorithms designed to exploit short-term movements. Also, one of the biggest advantages of trading equities is the availability of extensive historical price and volume data. This wealth of information allows traders to develop and backtest robust trading algorithms, ensuring that their strategies are based on sound historical performance. The abundance of data also makes it easier to refine and optimize strategies over time which is crucial for systematic trading approaches.

Bonds market, especially for corporate and municipal bonds, tends to be less liquid and transparent compared to the stock market. This lower liquidity can make it more challenging to execute trades efficiently, and the lack of transparency complicates the process of gathering the necessary data for developing and testing trading algorithms. Additionally, bonds are highly sensitive to changes in interest rates, which means that algorithms designed for bond trading often need to incorporate macroeconomic data and interest rate forecasts. This requirement makes it difficult to rely solely on technical trading strategies, as a broader integration of fundamental analysis is necessary. Given these complexities, other markets generally offer easier and more feasible options for algorithmic trading. As mentioned earlier, bonds are better suited for inclusion in a diversified investment portfolio rather than for systematic trading.

Derivatives (Options and Futures) are particularly well-suited for algorithmic trading because they allow traders to leverage positions and speculate on both upward and downward market movements. This flexibility makes derivatives attractive for strategies that seek to capitalize on small price changes.

However, Trading algorithms for derivatives often go beyond simple technical analysis, relying on complex models that predict price movements based on the dynamics of the underlying assets. These models may incorporate factors like volatility, time decay, and interest rates, making derivative trading a sophisticated yet potentially lucrative area for algorithmic strategies.

Forex (Foreign Exchange) market is one of the most liquid markets in the world and operates 24 hours a day, making it ideal for algorithms that trade on small price movements. The continuous nature of the forex market allows traders to execute strategies at any time, capitalizing on opportunities as they arise. Moreover, Forex trading is particularly well-suited for technical analysis because currency prices are heavily influenced by trends, repetitive patterns, and macroeconomic announcements. These characteristics make forex an excellent candidate for algorithmic trading, as algorithms can effectively capture and exploit these predictable patterns. Also, we, traders have access to historical data which makes it easier to backtest systematic strategies and refine our models, which as we mentioned is an essential point to consider. In other words, if there is no data we can’t build and test any trading system.

Commodities prices are heavily influenced by external factors such as weather, geopolitical events, and economic indicators. These factors can make it challenging to model price movements using purely technical methods, as they introduce a level of unpredictability that is difficult to quantify. Commodities are often traded in the form of futures contracts, which can be effectively managed by algorithms, particularly when they incorporate factors like seasonality and inventory levels into their models. By accounting for these external variables, algorithmic trading in commodities can become more reliable, though it still requires a sophisticated approach to handle the complexities of the market. If you are curious about how well algorithms might perform on commodities without accessing fundamental data,

news and market analysis, relying purely on technical systems, you can easily run any of the python strategies we will present in this book on commodities data and see how algorithms work well at times and fail with major events depending on the season.

3 Market Orders, Types and Execution

Understanding market orders, their types, and how they are executed is also important since we can benefit from these different types of executions also while coding trading systems.

3.1 Types of Market Orders

Market orders are the basic instructions you give to your broker or trading platform to execute a trade. The different types of market orders serve various purposes, allowing traders to manage risk, control entry and exit points, and optimize trade execution. Here, we will explore the most common types of market orders:

Market Orders: A market order is the most straightforward type of order, instructing your broker to buy or sell a security immediately at the best available current price. Market orders guarantee execution but not the price at which the order will be executed. This type of order is commonly used when the priority is to enter or exit a position quickly, rather than securing a specific price. However, market orders can be risky in volatile markets where prices can change rapidly, potentially leading to execution at a price significantly different from what was expected, or even orders being rejected by trading servers for significant price differences.

Limit Orders: A limit order specifies the maximum price at which you are willing to buy or the minimum price at which you

are willing to sell a security. In other words, it is an order to buy or sell a security at a specific price **or better**. For a buy limit order, the trade will only be executed if the market price reaches or falls below the specified limit price. For a sell limit order, the trade will only be executed if the market price reaches or exceeds the specified limit price. Unlike market orders, limit orders do not guarantee execution, as they will only be filled if the market reaches the specified price. Limit orders are useful for traders who are more concerned with achieving a specific entry or exit price than with the speed of execution. They are particularly effective in controlling transaction costs and avoiding the adverse effects of market volatility. However, the downside is that the order may not be executed if the market never reaches the specified price, potentially resulting in missed trading opportunities.

Example: Suppose you want to buy a stock currently trading at $50 but are only willing to purchase it if the price drops to or below $48. You can place a buy limit order at $48. If the stock's price reaches $48 or lower, your order will be executed. If the price doesn't reach $48 or below, the order remains unfilled.

Stop or Stop-Loss Orders: A stop order, also known as a stop-loss order, is an order to buy or sell a security once its price reaches a specified level, known as the stop price. When the stop price is reached, the stop order becomes a market order and is executed at the next available price. This type of orders is designed to limit an investor's loss on a position by triggering a market order once the price of a security reaches a specified level, known as the stop price. For example, if you own a stock currently trading at $50, you might place a stop-loss order at $45. If the stock price falls to $45, the stop-loss order becomes a market order, and the stock is sold immediately at the next available price. Stop-loss orders are essential for managing risk, particularly in volatile markets, by ensuring that losses do not exceed a predetermined threshold. However, because stop-loss

orders become market orders once triggered, there is a risk of slippage, especially in fast-moving markets.

Trailing Stop Orders: A trailing stop order is a variation of the stop-loss order that moves with the market price. The trailing stop is set at a fixed percentage or dollar amount below (for a sell order) or above (for a buy order) the current market price. As the price of the security moves in your favor, the trailing stop adjusts accordingly, locking in profits while still allowing for potential gains. If the market price reverses by the specified amount, the trailing stop is triggered, and a market order is executed. Trailing stops are particularly useful for protecting profits in a winning trade without the need to constantly monitor the position.

Each type of market order has its own advantages and disadvantages, and the choice of which to use depends on the specific trading strategy, market conditions, and risk tolerance. Understanding how and when to use each type of order is critical for optimizing trade execution and managing risk effectively.

3.2 Order Execution

Order execution refers to the process by which a broker fulfills a market order, limit order, or any other type of trade instruction. The speed, accuracy, and efficiency of order execution can significantly impact the outcome of a trade, particularly in fast-moving or highly liquid markets.

When a trader places an order, the broker or trading platform routes it to the appropriate exchange or market maker for execution. The goal is to match the order with a counterparty willing to take the opposite side of the trade. For example, a buy order is matched with a seller, while a sell order is matched with a buyer. In the case of a market order, execution occurs at the best available price, while limit orders are executed only when the market reaches the specified price.

This entire process isn't something we control as traders, but

several factors can impact the execution of an order. These elements are crucial to consider when developing our trading system:

Slippage: Slippage occurs when an order is executed at a price different from the one expected, usually due to rapid price changes in the market. Slippage is common with market orders, especially during periods of high volatility or low liquidity. For example, if you place a market order to buy a stock at $50, but the price jumps to $51 before the order is executed, you will experience $1 of slippage. While slippage is often unavoidable, understanding how we can include it in our backtesting can help us assess its effect on our trading strategy.

Fill Rates: Fill rate refers to the percentage of an order that is successfully executed. Market orders usually have a high fill rate because they are executed at the best available price. However, limit orders can have lower fill rates, especially if the specified price is far from the current market price. A low fill rate means only part of the order was filled, leaving the trader with an incomplete position. This issue is less common in high liquidity markets like Forex, where orders are typically filled more easily.

Partial Fills: A partial fill occurs when only a portion of an order is executed. This can happen in cases of low liquidity or when the order size exceeds the number of shares or contracts available at the desired price. While partial fills are often completed as more liquidity becomes available, they can be frustrating for traders, especially if market conditions change before the rest of the order is filled.

The role of brokers in order execution is also critical. Brokers can vary significantly in their execution speeds, order routing practices, and fees. Some brokers offer direct market access (DMA), allowing traders to interact directly with the order book and control the execution of their trades. Others use internal matching engines or route orders to third-party market makers.

Understanding the execution process and choosing a broker with reliable execution capabilities can have a significant impact on trading performance.

4 Liquidity and Spread

Liquidity and spread are two fundamental concepts that play a crucial role in determining trading costs, execution quality, and overall strategy performance. Liquidity refers to the ease with which an asset can be bought or sold without significantly affecting its price. The bid-ask spread is the difference between the highest price a buyer is willing to pay (bid) and the lowest price a seller is willing to accept (ask).

Liquidity: A highly liquid market is characterized by a large number of buyers and sellers, high trading volumes, and narrow bid-ask spreads. In such markets, traders can execute large orders quickly and with minimal price impact. Examples of highly liquid markets include major currency pairs in the forex market and large-cap stocks like those in the S&P 500. High liquidity is advantageous because it reduces the risk of slippage and ensures that orders are filled promptly.

In contrast, low liquidity markets have fewer participants, lower trading volumes, and wider bid-ask spreads. In these markets, executing large orders can be challenging, as it may require moving the price significantly to find a willing counterparty. Low liquidity increases the risk of slippage and partial fills, making it harder to execute trades efficiently. Examples of low liquidity markets include small-cap stocks, certain commodities, and exotic currency pairs.

Bid-Ask Spread: The bid-ask spread represents the cost of trading and is a key factor in determining the profitability of a trade. In highly liquid markets, the spread is usually narrow, meaning that the cost of entering and exiting a position is relatively low. Conversely, in less liquid markets, the spread can

be wide, increasing the cost of trading and reducing potential profits.

For example, if the bid price for a stock is $100 and the ask price is $100.10, the spread is $0.10. If a trader buys the stock at the ask price and immediately sells it at the bid price, they incur a $0.10 loss due to the spread. Therefore, understanding and managing the spread is crucial for optimizing trading performance, particularly for high-frequency trading strategies where even small differences in execution cost can have a significant impact on overall profitability.

Liquidity and spread are also influenced by market conditions, such as economic data releases, geopolitical events, and market sentiment. During periods of high volatility, liquidity may dry up, and spreads can widen dramatically, making it more difficult to execute trades at favorable prices. Conversely, in stable market conditions, liquidity is typically higher, and spreads are narrower.

For algorithmic traders, understanding liquidity and spread is essential for developing strategies that can adapt to changing market conditions. Algorithms need to monitor real-time market data, and traders can optimize their order placement to minimize trading costs and improve overall execution quality. For example, a simple algorithmic setup can stop generating trading signals if the spread is wider than a threshold that is a certain percentage of the initial price, this way trading is only triggered when the spread is narrower than the set threshold.

5 The Role of Market Data in Algorithmic Trading

Market data is the lifeblood of algorithmic trading, supplying the crucial information required to develop, backtest, and execute trading strategies. Even the most advanced algorithms

can fail without accurate and timely market data. In fact, one of the key reasons we consider equities and forex as top choices for algorithmic trading is the ease of access to reliable and abundant data.

In this section, we will introduce the different types of market data, the role of data providers, the mechanics of data feeds and APIs, and the techniques for analyzing market data. We will also discuss briefly the critical issues of data accuracy and latency, particularly in high-frequency trading scenarios.

5.1 Market Data Types

Market data can be broadly categorized into three types: real-time data, historical data, and derived data. Each type of data serves a specific purpose in the world of algorithmic trading:

Real-Time Data: Real-time data provides a continuous flow of up-to-the-second information on prices, trade volumes, order book depth, and other relevant market activity. This data is crucial for traders who need to make immediate decisions based on current market conditions. In algorithmic trading, real-time data triggers buy or sell signals, adjusts positions, and monitors market trends. The accuracy and speed of real-time data are vital, as delays can lead to missed opportunities or suboptimal trades. However, the importance of real-time data depends on the time frame your algorithm operates on. For instance, if your strategy uses a daily time frame, you're only checking the market once per day, at the end of the day or the next morning. In this case, you don't need fast data streaming; you have a full 24 hours between data queries. This is advantageous because you can use free data sources, like Yahoo Finance, which provide daily prices without needing expensive data feeds or broker services. The challenge grows as you increase the resolution of your time frame. For example, on a one-minute time frame, your algorithm queries the market every minute for each asset. If

you're trading 10 different stocks, that's 10 data queries every minute, all of which need to occur at the close of each minute to make trading decisions based on the closing price. Here, reliable data streaming becomes essential. The complexity increases further with tick data, which captures every single price movement. Tick data is not only more expensive but also comes with limitations because brokers' hardware can only handle a certain number of queries per second. Even if you have access to live tick data, it's usually not unlimited. To manage this, traders often run a preliminary algorithm on a low time frame, such as one minute, to identify assets of interest. Only those selected assets are then subjected to tick data queries, conserving resources and managing the data load efficiently. High-frequency trading introduces additional complexities, but since these are beyond the scope of this book, we will focus on higher time frames, anything above five minutes. For these time frames, data is generally available for free, and the quality is sufficient. Even if there is some uncertainty in the opening and closing prices, it won't significantly impact trading outcomes on time frames longer than one hour. The price movements on these longer time frames are large enough that small inaccuracies in data do not materially affect the results.

Historical Data: Historical data is an essential component in the toolbox of any algorithmic trader. It includes past market data, such as prices, volumes, and other relevant metrics over a specified period. This data is invaluable for backtesting trading strategies, allowing us to simulate how our algorithms would have performed under various historical market conditions. We can analyze historical data to identify recurring patterns, test different scenarios, and refine strategies before deploying them in live markets. This approach helps minimize risk and enhances the potential for profitability, ensuring that the strategy is robust across different market environments. Python, with its powerful libraries such as Pandas and NumPy, handles historical data exceptionally well. These libraries enable traders to efficiently

manage and analyze large datasets, making it easy to work with different time frames, apply various filters, and perform complex calculations. One particularly useful feature in Python is its ability to handle multi-timeframe indexing. This means we can easily aggregate data from different time frames, such as daily, hourly, and minute intervals, within the same dataset. This capability is essential for developing multi-timeframe strategies, where decisions on a shorter time frame might be influenced by trends observed on a longer time frame. Beyond basic backtesting, Python's extensive ecosystem of libraries, such as Matplotlib for visualization and SciPy for statistical analysis, allows traders to perform advanced analysis on historical data. For instance, we can conduct sentiment analysis by integrating historical data with text data from news sources or apply machine learning techniques using libraries like Scikit-learn to predict future price movements based on historical patterns. Additionally, Python's flexibility enables the integration of historical data with external databases or APIs, facilitating real-time updates and continuous backtesting. This means we can create dynamic models that evolve with the market, constantly refining strategies based on the most recent data. Python's ability to seamlessly work with historical data makes it an indispensable tool for algorithmic traders, offering both the power and versatility needed to succeed in fast-paced markets.

Derived Data: Derived data is created by processing raw market data to generate new metrics or indicators that provide deeper insights into market behavior. Common examples of derived data include classic technical indicators such as moving averages, Relative Strength Index (RSI), and volatility measures. These indicators are integral to technical analysis, helping traders identify trends, assess momentum, and spot potential reversal points in the market. Transforming raw data into meaningful metrics, allows us to use derived data to enhance the decision-making process in trading, making it easier to develop and refine trading strategies. Python offers a range of powerful

libraries that simplify the generation of technical indicators and derived data. For instance, **Pandas_ta** and **TA-Lib** are popular libraries that provide a comprehensive suite of technical analysis tools. With these libraries, traders can easily compute a wide array of indicators such as Bollinger Bands, MACD, and Fibonacci retracements, among others, without the need to manually code these calculations from scratch, or with minimal codding in the case of Fibonaccis depending on the indicator, the point is you can customize these indicators in so many ways to fit your trading strategy. Additionally, libraries like **Backtesting**, **Backtrader** and **Zipline** not only support the integration of these indicators but also facilitate the backtesting of strategies that rely on derived data. For more advanced analysis, Python's **NumPy** and **SciPy** can be used to customize and create unique indicators tailored to specific trading strategies, offering endless possibilities for innovation. These tools empower traders to transform raw market data into actionable insights, enabling more informed and precise algorithmic trading decisions.

Volume and Level 2 Data in Trading: Beyond price charts, open, high, low and closing prices, understanding volume and level 2 data can open the door to additional trading strategies that can be easily coded and automated to enhance your trading experience. Volume data refers to the total number of shares or contracts traded during a specific period. In the stock market, volume is a key indicator of market activity, showing how much interest there is in a particular stock. High volume often signifies strong investor interest and can confirm the strength of a price movement, whether it's a breakout or a trend continuation. Conversely, low volume may indicate weak interest, making price movements less reliable. Traders use volume data to assess the validity of trends and to gauge the potential for price reversals.

Level 2 data, also known as market depth, provides a more detailed view of the order book by showing the prices at which other market participants are willing to buy (bids) or sell (asks) a

security, along with the volume available at each price level. This data allows traders to see the supply and demand at different price points, offering insights into potential support and resistance levels. In the stock market, Level 2 data is particularly valuable for short-term traders and scalpers who need to understand the market's microstructure and anticipate price movements.

The forex market, however, operates differently. Unlike the stock market, forex does not have a centralized exchange, which means there isn't a single, unified source for volume or Level 2 data. Forex brokers may provide their own volume data, but this represents only a portion of the overall market, as it is limited to the trades conducted through that broker! This makes forex volume data less reliable compared to the stock market, where all trades are recorded on exchanges. Similarly, Level 2 data in forex is often limited or non-existent, as the market's decentralized nature means there isn’t a consolidated order book visible to traders.

A Word Around Cryptocurrencies: Volume data in the cryptocurrency market indicates the total amount of a particular cryptocurrency traded across exchanges during a specific period. Just like in stock trading, volume in the crypto market helps traders gauge the strength or weakness of a price movement. High volume often confirms the legitimacy of a price trend, whether it’s a surge or a decline, and can be a signal of strong market interest. Conversely, low volume might suggest that a price movement lacks conviction, making it less reliable for traders to act upon. However, unlike stocks, cryptocurrency volume data can vary significantly between different exchanges because the crypto market is decentralized. This means there isn’t a single, unified source for volume data—each exchange reports its own figures, which can sometimes lead to discrepancies in perceived market activity.

One key difference in Level 2 data between cryptocurrencies

and traditional markets like stocks is that cryptocurrency markets operate 24/7 across multiple exchanges worldwide. As a result, the order book on one exchange might differ significantly from another. This can create opportunities for arbitrage, where traders buy on one exchange and sell on another to profit from price discrepancies. However, it also means that the data may be less consistent, and traders need to be aware of the liquidity and depth available on the specific exchange they are using. Moreover, while some major crypto exchanges provide detailed Level 2 data, not all do, and the quality and availability of this data can vary. Additionally, as with forex, the decentralized nature of the crypto market means there is no single, consolidated order book across all exchanges, which can make it harder to get a complete picture of the market.

In short, I would approach cryptocurrencies similarly to forex, avoiding heavy reliance on volume and market depth data.

A final word around this, everything you will be building starts from the data you have so accurate and timely market data is the foundation of any successful trading strategy. The quality of the data directly impacts the effectiveness of your algorithms, so when it comes to choosing your data source it's better to spend some time doing your research to choose a reliable source and ensure that your data is up-to-date.

5.2 Data Providers

Market data providers are companies or platforms that supply the data needed for trading. These providers range from large, established firms like Bloomberg and Reuters to free sources such as Yahoo Finance and Alpha Vantage. Each provider offers different levels of data quality, coverage, and pricing, so it's important to choose the one that best suits your needs.

Bloomberg: Bloomberg is one of the most well-known and widely used data providers in the financial industry. It offers

comprehensive market data, news, analytics, and tools through its Bloomberg Terminal, a powerful platform used by professionals worldwide. Bloomberg's data is known for its accuracy, depth, and timeliness, making it a top choice for institutional traders and large firms. However, the Bloomberg Terminal comes with a high price tag, which may be prohibitive for individual traders or small firms.

Reuters: Reuters, another major player in the financial data industry, provides a wide range of market data, news, and analysis through its Eikon platform. Like Bloomberg, Reuters is known for its reliability and breadth of coverage. Reuters' data is used by professionals for trading, risk management, and research. The Eikon platform offers a user-friendly interface and extensive data coverage, but like Bloomberg, it is relatively expensive.

Yahoo Finance: Yahoo Finance is a popular free source of market data, offering real-time quotes, historical data, and financial news. While it lacks the depth and sophistication of Bloomberg or Reuters, Yahoo Finance is a good starting point for individual traders or those with limited budgets. It provides access to basic data and news, making it suitable for developing and testing simple trading strategies.

Alpha Vantage: Alpha Vantage is a free API provider that offers real-time and historical market data. It is particularly popular among algorithmic traders and developers because of its easy integration with Python and other programming languages. Alpha Vantage offers data on stocks, forex, and cryptocurrencies, making it a versatile option for those looking to build and test trading algorithms without incurring significant costs.

Each data provider has its pros and cons, and the choice depends on your trading needs, budget, and the level of sophistication required for your strategies. For institutional traders, the comprehensive coverage and reliability of Bloomberg or Reuters may be worth the investment. For individual traders

or those just starting, free sources like Yahoo Finance or Alpha Vantage offer a more accessible entry point.

5.3 Data Feeds and APIs

Data feeds and APIs (Application Programming Interfaces) are the means by which market data is delivered to traders and their algorithms. Understanding how data feeds work and how to integrate them into your trading systems is crucial for ensuring that your algorithms receive the information they need to make timely decisions.

Streaming Data Feeds: Streaming data feeds provide continuous, real-time data to your trading system. This data includes live price quotes, trade volumes, and order book updates. Streaming data is essential for high-frequency trading (HFT) and other strategies that require instant decision-making based on the latest market information. The speed and reliability of streaming data feeds are critical, as even a slight delay can result in missed trading opportunities.

API Access: APIs allow traders to access market data programmatically, enabling them to integrate data directly into their trading algorithms. APIs provide a flexible and efficient way to retrieve data, execute trades, and manage positions. Many data providers, including Bloomberg, Reuters, and Alpha Vantage, offer APIs that can be used to access their data services. APIs are particularly useful for algorithmic traders who need to automate the process of data retrieval and trade execution.

RESTful APIs: RESTful APIs are a common type of API that uses standard web protocols to access and interact with data services. RESTful APIs are widely used because they are simple, scalable, and easy to integrate with existing systems. They allow traders to request specific data, such as historical prices or real-time quotes, and receive it in a format that can be easily processed by their algorithms.

WebSocket APIs: WebSocket APIs provide a more interactive and real-time method of accessing data compared to RESTful APIs. They establish a continuous connection between the client and the server, allowing data to be pushed to the client as soon as it is available. This makes WebSocket APIs ideal for applications that require low-latency data, such as high-frequency trading.

Integrating data feeds and APIs into your trading systems requires a good understanding of the technical aspects involved. This includes handling data in different formats (e.g., JSON, CSV), managing API authentication, and ensuring that your system can process and act on the data in real-time. The choice of data feed and API will depend on your trading strategy, the level of automation you require, and the speed at which you need to receive and process data.

If you're unfamiliar with these technical terms, there's no need to worry. The examples in this book are designed so that you won't need to differentiate between these concepts. What matters is that we'll be writing a few functions to retrieve live market data for free. All the examples provided in this book won't require any special memberships or additional costs, and you can use these functions to get any data needed for your algorithmic trading.

These definitions are included in this section for those who may want to expand their knowledge on data retrieval later on. However, the main focus of this book is on the algorithmic side of trading, specifically, how to design, backtest, and implement trading strategies.

5.4 Data Accuracy and Latency

As we mentioned earlier, data accuracy and latency are two critical factors that can significantly impact trading performance. Ensuring that your data is accurate and minimizing latency in data processing are essential for executing trades effectively,

particularly in high-frequency trading environments.

Data Accuracy In Algorithmic Modelling: Inaccurate or incomplete data can lead to incorrect signals, missed opportunities, and potentially significant losses. Ensuring data accuracy is crucial for successful trading. This involves verifying the integrity of the data, cross-referencing it with multiple sources, and relying on trustworthy data providers. It's also essential to regularly monitor for discrepancies or errors in the data that could impact your trading decisions. Implementing quality control measures and auditing your data feeds consistently will help maintain the accuracy of your market data. In my experience, I've encountered odd data points or "weird candles" when plotting charts using data from free sources like Yahoo Finance and Dukascopy. While these sources are valuable, especially for those who want to avoid additional costs, errors can occur. It's the trader's responsibility to verify the accuracy and correctness of the data before using it in their strategies.

When designing an algorithmic trading strategy, we can account for these uncertainties by using techniques such as random sampling and Monte Carlo simulations. For example, we can blur the values of opening and closing prices by a small, random amount, following a uniform or normal distribution. This approach helps simulate real-world conditions where data might not be perfect, and prices could vary slightly due to slippage or other factors. Additionally, by blurring the entry prices, we can further mimic the effects of slippage and other variables that might influence trading outcomes. This method allows us to generate results that include combined uncertainties, providing a more realistic picture of potential performance. For instance, instead of simply expecting a 50% yearly return, we might estimate it as 50% ±10% to account for these uncertainties.

In the coming chapters, we'll explore these concepts in greater depth, with a dedicated section on Monte Carlo simulations. This section will focus on how to incorporate

different types of noise and uncertainty into your trading models, helping you better prepare for the realities of live trading.

Latency: Latency refers to the delay between the moment market data is generated and when it is received and processed by your trading system. In high-frequency trading (HFT), where decisions are made and executed within milliseconds, even a slight delay can significantly impact performance. High latency can lead to trades being executed at less favorable prices, reducing profitability or increasing losses. To minimize latency, traders can use low-latency data feeds, move their trading servers near exchange data centers, and optimize their algorithms for speed. Additionally, using direct market access (DMA) can reduce the number of intermediaries between the trader and the market, further reducing latency. While low latency is critical for high-frequency trading, it is also important for other types of trading strategies where timely execution can affect the outcome of a trade.

However, for the purposes of this book, latency is not a critical issue. The strategies and examples provided here are designed to operate on higher time frames, generally above 5 minutes, where latency becomes negligible. When you're working with time frames like 5 minutes or longer, there is ample time for your algorithm to query data, perform computations, and generate a trading signal. In fact, using higher time frames, such as the daily or 4-hour charts, not only avoids latency problems but also filters out much of the noise present in lower time frames. This noise, often caused by market indecision, is smoothed out, making the signals more reliable and reducing the likelihood of false alarms.

Trading on lower time frames, such as minutes, introduces additional challenges. The higher frequency of trades and the tight spreads can lead to significant transaction costs, which can erode profitability. In my experience, building a successful algorithmic strategy on such low time frames, especially for

scalping, is extremely difficult. The costs associated with frequent trading can turn a potentially profitable strategy into a losing one.

For most individual traders, it’s more practical to focus on higher time frames like daily or 4-hour charts. These time frames not only simplify the trading process but also improve the reliability of the signals generated by your algorithms. High-frequency trading, on the other hand, is generally out of reach for individual traders with standard home setups. It requires specialized hardware and ultra-fast connections typically available only to large institutions.

Therefore, latency is not a concern for the strategies covered in this book, as they operate on time frames where the impact of latency is minimal. If you ever encounter latency issues, such as delays caused by complex computations or data retrieval that exceed a minute or two, it may be necessary to reconsider your approach or optimize your code. And as long as you're working with time frames above 5 minutes there is always a workaround to make things work faster.

Finally, it's important to note that data accuracy and minimizing latency are ongoing challenges in algorithmic trading. To ensure success, robust data management practices and an optimized trading infrastructure are essential. The specific requirements will vary depending on your trading style and strategy, but it's crucial to tailor these practices to suit your approach. As a personal tip, if you’re still finding your footing in the world of algorithmic trading, start with higher timeframes. It’ll save you a ton of time and help you avoid a lot of technical headaches. Higher timeframes not only simplify the process but also give you more reliable signals, making it easier to develop and refine your strategies without getting bogged down by the complexities of high-frequency trading.

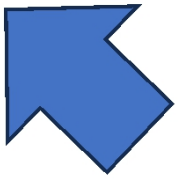
Chapter 3: Introduction to Python for Trading

1 Setting up your python environment

It's essential to set up your Python environment correctly. This section will guide you through the installation process, with a focus on two popular methods: downloading Python directly from the official website or using a package manager like Anaconda.

1.1 Direct installation from Python's Official Website

- Start by going to https://www.python.org/ This is the official website where you can download the latest version of Python.
- Navigate to the Downloads section, where you'll find options for various operating systems (Windows, macOS, Linux). Choose the appropriate version for your system. It's generally recommended to download the latest stable release.





- Once the download is complete, run the installer. On Windows, ensure you check the box that says "Add Python to PATH" before proceeding with the installation. This step is crucial as it allows you to run Python from the command line. Once Python is added to PATH in windows it's kind of accessible from any folder and any location on your drive.
- After installation, open a terminal or command prompt and type python --version. If the installation was successful, it should display the installed version of Python.

```
C:\>python --version
Python 3.11.4
```

Although this method installs Python correctly from the official website, coding directly from the command prompt isn't very practical. Additionally, using a simple text file and running it later through the command prompt can be cumbersome. Typically, we use Integrated Development Environments (IDEs) like VSCode or PyCharm. Think of these as user-friendly interfaces that make it easier to work with Python installed on your system. We'll dive into IDEs and how to use them in the following section.

1.2 Integrated Development Environments (IDEs)

Choosing the right Integrated Development Environment (IDE) is an important step in setting up your Python environment for algorithmic trading. An IDE serves as the interface where you write, test, and debug your code. While there are many IDEs available, each with its own set of features, the choice ultimately comes down to personal preference and the specific needs of your project. In this section, we'll discuss three popular IDEs: Jupyter Notebook, VSCode, and PyCharm, highlighting their strengths and how they can be used effectively in finance-related projects.

1.2.1 Why Choose an IDE?

An IDE provides a more efficient and organized workspace compared to coding directly in a text editor or command prompt. IDEs offer features like syntax highlighting, code completion, debugging tools, and integration with version control systems. These features streamline the coding process and help you avoid common mistakes, making it easier to develop and maintain your trading algorithms.

1.2.2 Jupyter Notebook: Interactivity and Visualization

Jupyter Notebook is a widely used IDE, especially popular in data science and financial analysis. It operates in a web browser, allowing you to write and execute code in a more interactive manner. The main advantage of Jupyter Notebook is its ability to mix code with rich text elements such as headings, links, and images, which makes it ideal for documenting your workflow and creating detailed analysis reports.

Interactivity: Jupyter allows you to run code in small, manageable chunks, and view the output immediately below the

code cell. This interactive nature is particularly useful when analyzing financial data, as you can make adjustments on the fly and see the results instantly.

Visualization: Jupyter Notebook integrates seamlessly with visualization libraries like Matplotlib and Plotly, making it easy to create and display charts directly within the notebook. This is invaluable when working with financial data, as visual representations of market trends, price movements, and other key indicators can greatly enhance your analysis.

Use Cases: Jupyter is ideal for exploratory data analysis, backtesting trading strategies, and creating presentations or reports that combine code, data, and visualizations.

1.2.3 VSCode: Flexibility and Extensibility

Visual Studio Code (VSCode) is a versatile and highly customizable IDE that has gained popularity among developers across various fields, including finance. It provides a balance of powerful features, flexibility, and ease of use, making it an excellent choice for both beginners and experienced traders. I use VSCode every day for my coding projects and I find that it offers everything I need as a developer.

Flexibility: VSCode is known for its lightweight nature and the ability to extend its functionality through a wide range of extensions. Whether you need advanced debugging tools, integrated terminal access, or specific Python utilities, VSCode can be tailored to meet your needs.

Code Management: VSCode offers robust support for managing and navigating through large codebases. Features like IntelliSense (code completion), code linting, and Git integration make it easier to write clean, efficient code and keep track of changes over time.

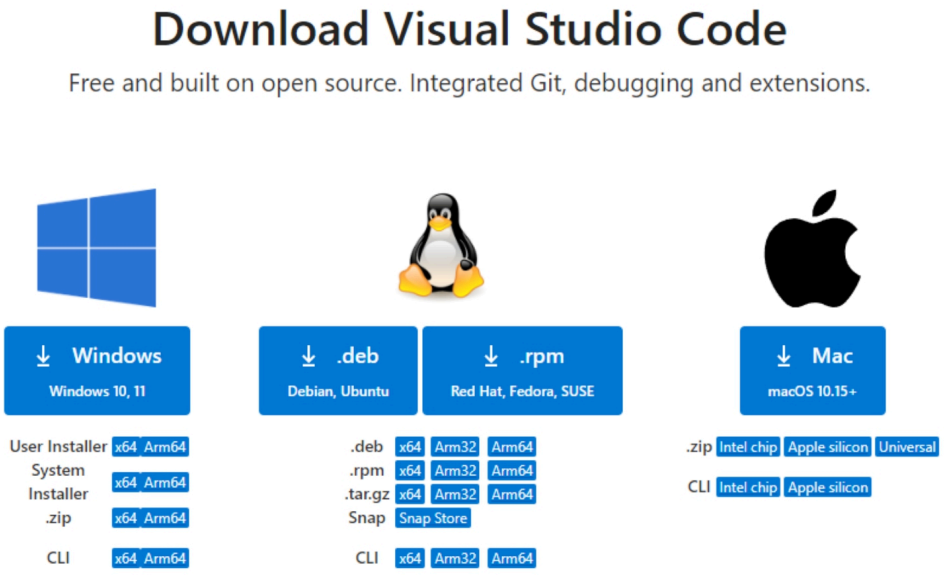
Extensibility: One of the key strengths of VSCode is its vast library of extensions. For finance-related projects, you can install extensions for Python, Jupyter, version control (Git), and

even specific finance tools.

Visualization and Debugging: While VSCode doesn't offer the same level of interactive visualization as Jupyter Notebook, it supports powerful debugging tools and can run Jupyter notebooks within the IDE itself using the Jupyter extension. This makes it a highly flexible option for those who want the best of both worlds, strong code management and access to interactive notebooks.

1.2.4 Installing VSCode:

Go to the https://code.visualstudio.com/Download and download the installer for your operating system (Windows, macOS, or Linux)



Run the installer and follow the on-screen instructions. The installation process is straightforward and should only take a few minutes.

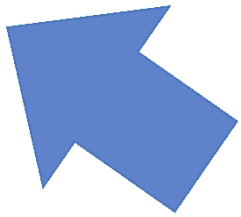
Once VSCode is installed, open the application and go to the Extensions view by clicking on the Extensions icon in the Activity Bar on the side of the window or pressing Ctrl+Shift+X.

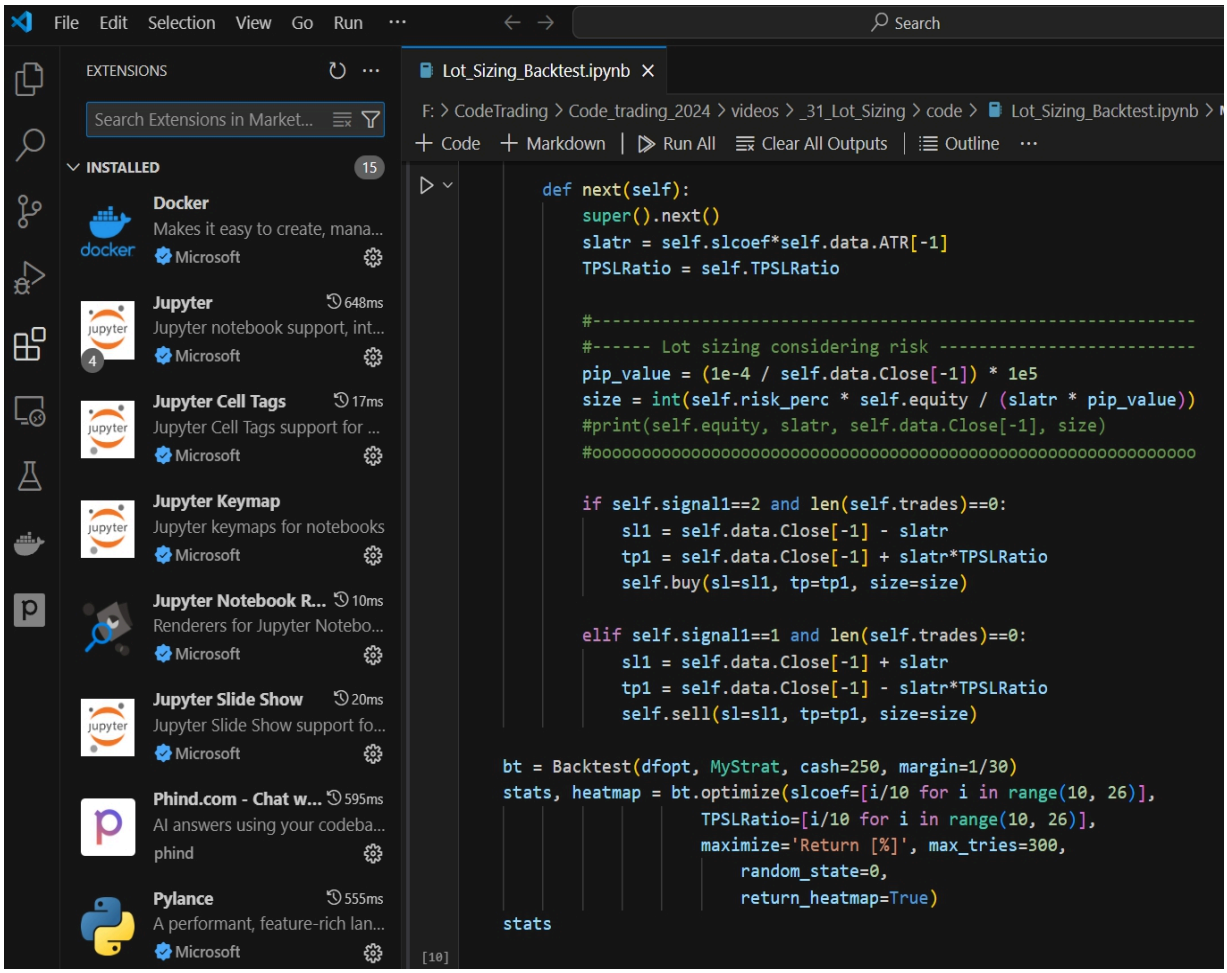
Search for "Python" and install the official Python extension provided by Microsoft. This extension adds support for Python

language features, including IntelliSense, linting, and debugging.

After installing the Python extension, you'll need to select the Python interpreter. Open the Command Palette (Ctrl+Shift+P), type "Python: Select Interpreter," and choose the environment you want to use (e.g., the base Python environment created earlier when you installed python from the original website, as explained in the previous section about python installation).

Depending on your needs, you might want to install additional extensions like Jupyter for running notebooks. Simply search for these extensions in the Extensions view and click install.





1.2.5 PyCharm: Comprehensive Features for Python Development

PyCharm is another popular IDE, particularly among developers who work extensively with Python. It offers a comprehensive set of tools and features designed to streamline the development process.

Full-Featured IDE: PyCharm comes with built-in support for everything from code analysis to project navigation, making it a powerful tool for managing complex Python projects.

Professional Version: While the free Community edition is sufficient for most needs, the Professional version offers advanced features like database tools, scientific tools, and more, which might be beneficial for extensive financial projects.

Refactoring and Testing: PyCharm excels in helping you write clean, maintainable code, with features for refactoring, testing, and debugging, all tightly integrated into the IDE.

Finally, Selecting the right IDE is a matter of personal preference and project needs. Whether you choose Jupyter Notebook for its interactivity and visualization capabilities, VSCode for its flexibility and extensibility, or PyCharm for its comprehensive Python development features, each IDE has its strengths. I provided detailed instructions on setting up VSCode, since it's the IDE I am using every day for my coding and I find it to be a versatile option that balances simplicity with powerful features, making it an excellent choice for a wide range of finance-related projects. However, feel free to explore and choose the IDE that best fits your workflow and preferences.

1.3 Installing Python Using the Anaconda Package Manager

While downloading Python directly from the official website is straightforward, using Anaconda offers several advantages, particularly for finance-related projects. Anaconda is a popular distribution of Python that simplifies package management and deployment, making it particularly well-suited for data science and finance-related projects. comes pre-installed with over 1,000 of the most commonly used Python libraries and tools, including essential ones like Pandas, NumPy, Matplotlib, and SciPy. These libraries are the backbone of financial data analysis, allowing you to handle large datasets, perform complex numerical computations, and create detailed visualizations, all crucial tasks in algorithmic trading. Instead of manually installing each library one by one, Anaconda provides them out-of-the-box, saving you time and ensuring that you have everything you need right from the start.

Anaconda includes Conda, a powerful package and environment manager that makes it easy to install, update, and

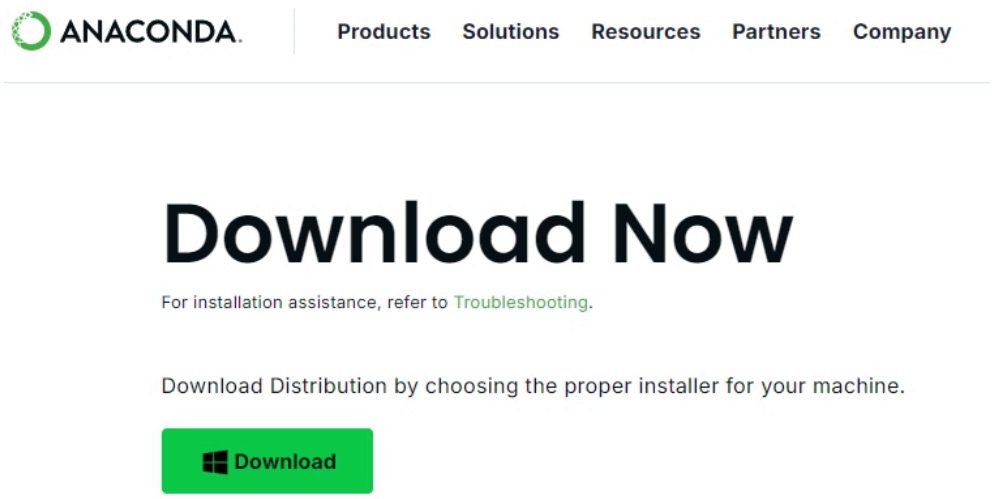
manage packages and their dependencies. Conda handles package conflicts automatically, ensuring that the libraries you install work well together. This is particularly important in finance-related projects, where different packages may have specific version requirements. Conda's ability to manage dependencies reduces the risk of errors and conflicts that could arise when manually installing packages.

Anaconda is designed to work seamlessly with Jupyter Notebook, an interactive development environment that's particularly useful for financial analysis and algorithmic trading. Jupyter Notebook allows you to write and run code in a browser-based interface, making it easy to document your work, visualize data, and test code snippets in real-time. This interactive approach is ideal for exploring financial datasets, backtesting trading strategies, and presenting results in a clear, understandable format.

So, if you decide to use Anaconda for your python installation you don't have to install any additional IDE such as VSCode or PyCharm. Instead, you can rely on Jupyter Notebook which comes preinstalled with Anaconda and it will be enough for the examples we will develop in this book.

To install the Anaconda package manager, you can follow these steps:

- Visit the https://www.anaconda.com/ and navigate to the Downloads section. Choose the version of Anaconda that matches your operating system.



- After downloading, run the installer and follow the on-screen instructions. The installation process is straightforward. On Windows, ensure you check the option to add Anaconda to your PATH environment variable.
- Open a terminal or Anaconda Prompt (which is installed with Anaconda) and type conda --version. This should display the installed version of Conda. Then, type python --version to check the Python version installed with Anaconda.

Tip: If you encounter an error message indicating that a package or module is missing while coding, you can easily install it directly within Jupyter Notebook. Simply type !pip install <PACKAGE NAME> in any cell, and it will install the missing package directly into your environment. For example to install Pandas execute the following code: !pip install pandas.

2 Key Python Libraries For Testing Strategies

In this brief section, we'll cover some of the essential

packages and modules you'll need for downloading data files, performing data wrangling and cleaning, adding technical indicators, and backtesting strategies. While this isn't an exhaustive list, there are many other libraries available for visualization and analysis, these are the core tools you'll need to complete a full project pipeline from start to finish.

2.1 Pandas

Pandas is a powerful library for data manipulation and analysis, built on top of NumPy. It provides data structures like Series and DataFrame, which are ideal for handling structured data, such as time series data often used in financial markets.

Uses in Financial Backtesting:

Data Importing and Cleaning: Pandas makes it easy to import data from various sources like CSV files, Excel sheets, SQL databases, and APIs. It also provides tools to clean and preprocess this data, such as handling missing values, filtering, and reformatting.

Time Series Analysis: Pandas excels at managing time series data, which is crucial for backtesting. You can easily resample, roll, and shift data to create moving averages, rolling statistics, and other time-dependent indicators.

Data Wrangling: Pandas allows you to merge, join, and pivot data, enabling you to structure your datasets in a way that is most useful for your analysis and backtesting.

2.2 NumPy

NumPy is the foundational package for numerical computing in Python. It provides support for arrays, matrices, and a wide range of mathematical functions, making it essential for high-performance computations.

Uses in Financial Backtesting:

Mathematical Operations: NumPy's array operations are

highly efficient, allowing for fast computation of large datasets, which is often required in backtesting.

Statistical Analysis: NumPy offers a variety of statistical functions that can be used to calculate returns, standard deviations, correlations, and other key metrics used in financial analysis.

Vectorization: One of the key strengths of NumPy is its ability to perform vectorized operations, which are much faster than traditional loops in Python. This is particularly useful when applying calculations across large datasets, such as calculating daily returns across multiple assets.

2.3 Matplotlib and Pyplot

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Pyplot is a module within Matplotlib that provides a MATLAB-like interface, simplifying the process of generating plots and charts.

Uses in Financial Backtesting:

Data Visualization: Matplotlib, along with Pyplot, is commonly used to create visualizations of financial data, such as line charts, candlestick charts, histograms, and more. These visualizations are crucial for understanding trends, patterns, and anomalies in your backtested results.

Overlaying Indicators: You can use Matplotlib to plot various technical indicators over price charts, such as moving averages, Bollinger Bands, and RSI, making it easier to visualize how these indicators perform relative to price movements.

Custom Visuals: Matplotlib allows for extensive customization of plots, enabling you to tailor visuals to highlight specific aspects of your data, which is particularly useful when presenting backtest results.

2.4 Statistical Packages (Scipy, Statsmodels)

Python's ecosystem includes several statistical packages, with SciPy and Statsmodels being two of the most commonly used for advanced statistical analysis.

Uses in Financial Backtesting:

Regression Analysis: Statsmodels provides robust tools for performing linear, logistic, and other types of regression analysis, which can be used to model relationships between different financial variables.

Correlation and Covariance: Both SciPy and Statsmodels offer functions to calculate correlation and covariance matrices, helping you understand the relationships between different assets or indicators.

Hypothesis Testing: These packages also provide tools for statistical hypothesis testing, which can be used to validate the performance of a trading strategy.

2.5 Pandas TA

Pandas TA is a technical analysis library built on top of Pandas. It provides a wide range of technical indicators that can be easily integrated into a Pandas DataFrame.

Uses in Financial Backtesting:

Technical Indicators: Pandas TA offers a comprehensive collection of indicators such as moving averages, RSI, MACD, Bollinger Bands, and many others. These indicators can be easily applied to your financial data to generate signals for backtesting.

Integration with Pandas: Since it's built on Pandas, it seamlessly integrates with your existing data pipelines, allowing you to apply technical analysis directly to your data with minimal code.

Custom Indicator Creation: Pandas TA also allows you to create custom indicators by combining existing ones, giving you the flexibility to tailor your analysis to specific needs.

2.6 TA-Lib

TA-Lib (Technical Analysis Library) is another widely used library for technical analysis. It provides over 150 indicators and functions that are commonly used in financial market analysis.

Uses in Financial Backtesting:

Breadth of Indicators: TA-Lib offers a broader range of indicators compared to most libraries, including some more advanced and lesser-known indicators.

Performance: TA-Lib is implemented in C, which makes it very fast and efficient, especially when dealing with large datasets.

Backtesting Flexibility: With TA-Lib, you can easily integrate technical indicators into your backtesting framework, providing robust tools for generating and analyzing trading signals.

2.7 Backtesting.py

Backtesting is a popular Python library specifically designed for backtesting trading strategies. It provides a flexible framework that allows you to test, optimize, and analyze trading strategies.

Uses in Financial Backtesting:

Strategy Development: Backtrader allows you to define and test trading strategies in an object-oriented way. You can specify the logic for entering and exiting trades, and Backtrader handles the execution and simulation of the strategy on historical data.

Performance Metrics: Backtrader automatically calculates

a wide range of performance metrics, including returns, drawdowns, Sharpe ratios, and more, providing a comprehensive overview of your strategy's performance.

Integration with Data Sources: Backtrader supports data from multiple sources, including CSV files, Pandas DataFrames, and online APIs, making it easy to integrate with your existing data infrastructure.

2.8 Scikit-learn

Scikit-learn is a powerful and versatile machine learning library in Python. Compared to previously mentioned libraries I would classify this one as more advanced and providing access to more complex solutions. It provides a wide range of algorithms and tools for classification, regression, clustering, and more. Scikit-learn is widely used in financial markets for tasks such as predictive modeling, risk assessment, and strategy optimization.

Uses in Financial Backtesting:

Feature Engineering: Scikit-learn provides numerous tools for preprocessing and feature engineering, such as scaling, encoding, and dimensionality reduction. These are essential when preparing your financial data for machine learning models.

Predictive Modeling: You can use Scikit-learn to build predictive models, such as logistic regression or random forests, which can be used to forecast market trends, asset prices, or to classify trading signals.

Model Evaluation: Scikit-learn offers a wide range of metrics for evaluating the performance of your models, such as accuracy, precision, recall, and ROC-AUC. These metrics help you assess how well your trading model is performing and where it might need improvements.

Cross-Validation: Scikit-learn's cross-validation tools allow you to test your trading strategy on multiple subsets of your

data, ensuring that your model generalizes well to unseen data and reducing the risk of overfitting.

Pipeline Integration: Scikit-learn's pipeline feature enables you to create end-to-end workflows that include data preprocessing, model training, and prediction, all in a single, streamlined process. This is particularly useful in backtesting to maintain a clean and reproducible workflow.

Scikit-learn's versatility and ease of use make it an essential tool for incorporating machine learning into your financial backtesting strategies.

3 Live Testing and Connecting to Broker APIs

When it comes to live testing your trading strategies, the choice of libraries will largely depend on the trading platform and broker you plan to connect to. Many brokers today offer Python APIs, including popular names like Binance, Interactive Brokers, and OANDA. These APIs allow you to connect your Python code directly to the broker's platform, enabling you to execute trades in real time, retrieve market data, and monitor your positions.

3.1 Choosing the Right Broker and API

The broker you choose will influence the specific libraries you need to install and the documentation you'll need to follow. Each broker provides its own set of Python libraries and API documentation, which are crucial for integrating your trading algorithms with their platform. It's essential to familiarize yourself with the API provided by your broker to effectively use the functions and features it offers.

In this book, we will focus on live testing using OANDA's

trading platform. OANDA is a well-known broker that provides a robust Python API, making it a great choice for algorithmic trading. Throughout this book, all examples related to live trading will utilize OANDA's libraries. However, once you're comfortable with the process, you can easily switch to a different broker that offers Python API support. Your choice might depend on factors like your geographic location, the broker's services in your country, or your specific trading preferences.

3.2 Key OANDA Python Libraries

OANDA offers a suite of Python libraries that facilitate interaction with their trading platform. Here's a brief introduction to the most important ones:

OANDA API v20:

- **Overview**: The OANDA API v20 is the primary API for interacting with OANDA's trading platform. It provides access to a wide range of functionalities, including placing trades, retrieving account information, and managing orders. This API is essential for executing your strategies live on the market.
- **Features**: With OANDA API v20, you can execute trades, set stop losses, and take profits, monitor your positions, and retrieve historical and real-time market data. The API is designed to be flexible and easy to use, making it a solid choice for both beginners and experienced traders.
- **Installation**: You can install the OANDA API v20 library using pip:

```
pip install oandapyV20
```

- **Documentation**: Comprehensive documentation is available online, providing detailed guidance on how to use each function and feature.

OANDA Candles:

- **Overview**: The OANDA Candles API allows you to retrieve historical price data in the form of candlestick charts. This data is crucial for backtesting and for making real-time trading decisions based on past price movements.
- **Features**: You can specify the instrument (e.g., currency pair), the granularity (e.g., minute, hourly, daily), and the time range for the candlestick data you want to retrieve. This flexibility enables you to tailor the data to match your specific trading strategy.
- **Usage**: The OANDA Candles API is integrated into the OANDA API v20, making it easy to retrieve and analyze candlestick data as part of your overall trading strategy.

OANDA Streaming API:

- **Overview**: The OANDA Streaming API provides real-time market data and trade updates. This is essential for implementing live trading strategies that need to react quickly to market changes.
- **Features**: The streaming API allows you to receive real-time price quotes, order book data, and updates on your open trades. This continuous stream of data is crucial for high-frequency trading and other strategies that depend on up-to-the-second market information.
- **Integration**: Like the Candles API, the Streaming API is part of the broader OANDA API v20. It integrates seamlessly with other OANDA services, allowing you to build a comprehensive live trading setup.

3.3 Getting Started with OANDA in Python

To start live trading with OANDA in Python, you'll first need to set up an account with OANDA and obtain an API key. This key is required to authenticate your requests to the OANDA servers. Once you have your API key, you can begin by installing the necessary Python libraries, as mentioned earlier.

Here's a simple example to get you started with OANDA in Python, that demonstrates how to use account details and stream real-time pricing data for the EUR/USD currency pair. Once you're comfortable with this setup, you can expand it to include more complex trading strategies, using the full range of features offered by OANDA's API:

```
import oandapyV20
from oandapyV20.endpoints.accounts import AccountDetails
from oandapyV20.endpoints.pricing import PricingStream

# Replace with your OANDA API key and account ID
api_key = 'YOUR_API_KEY'
account_id = 'YOUR_ACCOUNT_ID'

client = oandapyV20.API(access_token=api_key)

# Retrieve account details
account_details = AccountDetails(accountID)
response = client.request(account_details)
print(response)

# Stream real-time pricing data
params = {
    "instruments": "EUR_USD"
}
pricing_stream = PricingStream(accountID=accountID,
params=params)
response = client.request(pricing_stream)
```

```
for tick in response:
    print(tick)
```

4 Conclusion

These packages form the backbone of financial backtesting environment in Python. Pandas and NumPy provide the foundation for data manipulation and analysis, while Matplotlib and Pyplot handle visualization. Statistical packages like SciPy and Statsmodels offer advanced analysis tools, and specialized libraries like Pandas TA and TA-Lib provide the technical indicators needed for trading strategies. Finally, Backtesting.py ties everything together by offering a comprehensive framework for backtesting, optimizing, and even deploying trading strategies. Together, these tools empower you to develop, test, and refine your trading ideas.

At this point, I assume this introduction has provided a sufficient refresher on Python. While you don't need to be an expert in Python or an advanced software developer to follow along, it's important to have a basic understanding of the language. In the upcoming chapters, we'll assume you're familiar with Python basics. Additionally, for backtesting, you'll need to grasp the concepts of classes and instances, as well as have a basic familiarity with object-oriented programming, since the backtesting.py package relies on these concepts.

Since this book focuses primarily on trading strategies rather than being a Python course, we'll keep our attention on the trading aspects. If you find yourself struggling with the Python coding, I recommend taking some time to refresh your skills through another course or book dedicated to Python.

In the next chapter, we'll dive deeper into the technical side of things, including preparing and visualizing financial data, computing technical indicators using pandas_ta, and performing basic exploratory data analysis (EDA). This will set the stage for

developing and backtesting trading strategies effectively.

Books/References:

Python Crash Course by Eric Matthes - A beginner-friendly book that covers Python installation and basic setup.

Python for Data Analysis by Wes McKinney - Provides insight into setting up Python for data analysis, including package management.

Python Data Science Handbook by Jake VanderPlas - Covers Pandas, NumPy, and Matplotlib in the context of data science, with applications that are highly relevant to financial data.

Python for Finance by Yves Hilpisch - A comprehensive guide that covers Python basics, with a focus on financial data manipulation.

Hands-On Data Analysis with Pandas by Stefanie Molin - This book goes into detail on using Pandas for data manipulation, especially with time series data.

Chapter 4: Getting and Reading Financial Data, Adding Technical Indicators, and Visualization Tips

In this chapter, we will take a hands-on approach to working with financial data in Python. We will go through the foundational tools needed to retrieve, analyze, and visualize market data effectively.

We'll begin by exploring how to access and import financial data from various sources using Python, ensuring that you have

a solid understanding of the essential functions required for data retrieval. Next, we'll delve into computing and adding technical indicators that will be useful for any trading strategy. You'll learn how to compute these indicators using pandas_ta enabling you to create and refine trading signals.

Finally, we'll cover how to visualize this data, allowing you to gain insights through charts and graphs. Visualization is key in understanding market trends and validating your strategies. Additionally, we'll perform exploratory data analysis (EDA) to uncover hidden patterns and relationships within your data. Throughout the chapter, you'll be provided with practical Python code examples to help you implement these techniques in your own projects.

1 Importing Data

There are several ways to import data into Python, but to keep things simple, this chapter will focus on two free and straightforward methods. The first method involves downloading CSV files of historical data for a specific asset from a data provider. Many providers offer this data for free; for example, Yahoo Finance and Dukascopy are two excellent sources to start with. You can create a free account on their websites and begin downloading historical data as CSV files, which can then be easily read into Python using Pandas.

The second method involves downloading data through an API. To do this, you'll need a data provider that supports a Python API. The level of access you have may vary depending on your account type, whether it's a free account or a paid membership, but this largely depends on the provider and the platform. While I won't delve too deeply into the specifics, as they can be quite varied, I will walk you through examples using the yfinance package and OANDA’s broker API to demonstrate how this process works in practice.

1.1 Loading And Cleaning Data In Pandas

```
import pandas as pd
df = pd.read_csv("BTCUSD_Candlestick_1_D_ASK_08.05.2017-16.10.2021.csv")
df=df[df['Volume']!=0]
df=df[df["High"]!=df["Low"]]
df=df.reset_index(drop=True)
```

This code uses the read_csv function from the Pandas library to load a Bitcoin-to-U.S. dollar CSV file that has already been downloaded. The function reads the file and converts its contents into a Pandas DataFrame, which is a tabular data structure that makes it easy to manipulate and analyze the data.

Next, the code performs some basic data cleaning:

The first cleaning step removes any rows where the trading volume is equal to 0. These rows represent periods when no trades were made, and thus, they're not useful for analysis.

The second cleaning step filters out rows where the 'High' price is equal to the 'Low' price. These rows indicate that there was no price movement during that time, which often happens on weekends, holidays, or times when the market was closed or the data wasn't recorded due to technical issues.

After filtering out these unwanted rows, the code resets the DataFrame's index, ensuring that the index values are sequential integers. This makes the DataFrame easier to work with in subsequent analysis.

The resulting DataFrame is shown in the figure below. You can see it includes an integer index, followed by columns for GMT time, open, high, low, and closing values, as well as a volume column. With this setup, the DataFrame is now ready for further analysis.

	Gmt time	Open	High	Low	Close	Volume
0	07.05.2017 21:00:00.000	1559	1681	1557	1658	1.872044e+06
1	08.05.2017 21:00:00.000	1658	1753	1626	1747	1.330136e+06
2	09.05.2017 21:00:00.000	1713	1769	1677	1752	1.916895e+06
3	10.05.2017 21:00:00.000	1784	1887	1777	1857	1.613521e+06
4	11.05.2017 21:00:00.000	1836	1836	1651	1692	2.550428e+06

1.2 Verifying the Integrity of 'Gmt time' column

We prioritize verifying the 'Gmt time' column for missing values and gaps because it is the most critical indicator of data quality. Ensuring that the timestamps are complete and consistently ordered is crucial. While we can also check for missing values in prices and candle boundaries later, our main focus is on the time data. This verification helps us confirm that there are no gaps in our DataFrame and that the data is correctly ordered, with dates and times always increasing sequentially.

1.2.1 Check for missing values

```
df['Gmt time'] = pd.to_datetime(df['Gmt time'],
                                format='%d.%m.%Y %H:%M:%S.%f')
missing_values = df['Gmt time'].isnull().sum()
if missing_values == 0:
    print("No missing values in the GMT time column.")
else:
    print(f"There are {missing_values} missing values in the GMT
time column.")
```

First, make sure the GMT time column is in the correct datetime format. If it's not already in datetime format, you can

convert it, and this is done in the first line in this snippet before checking for any missing values.

1.2.2 Verify that the Time Values are Always Increasing

```
is_increasing = df['Gmt time'].is_monotonic_increasing
if is_increasing:
    print("The GMT time values are clean and always increasing.")
else:
    print("The GMT time values are not always increasing.")
```

You can check if the time values in the GMT column are always increasing by comparing the column with a shifted version of itself.

If the time values are not always increasing, you can identify where the problem occurs:

```
non_increasing = df[df['Gmt time'] <= df['Gmt time'].shift(1)]
if len(non_increasing) > 0:
    print("The following rows have non-increasing GMT time values:")
    print(non_increasing)
else:
    print("All GMT time values are correctly increasing.")
```

And we can also visualize the time differences between consecutive rows to see if there are any irregularities:

```
df['time_diff'] = df['Gmt time'].diff().dt.total_seconds()
df['time_diff'].plot(kind='line', title='Time Differences Between Consecutive GMT Values')
```

But since in my data set I used the daily timeframe I prefer to plot the differences in hours to make the inspection easier:

```
import matplotlib.pyplot as plt

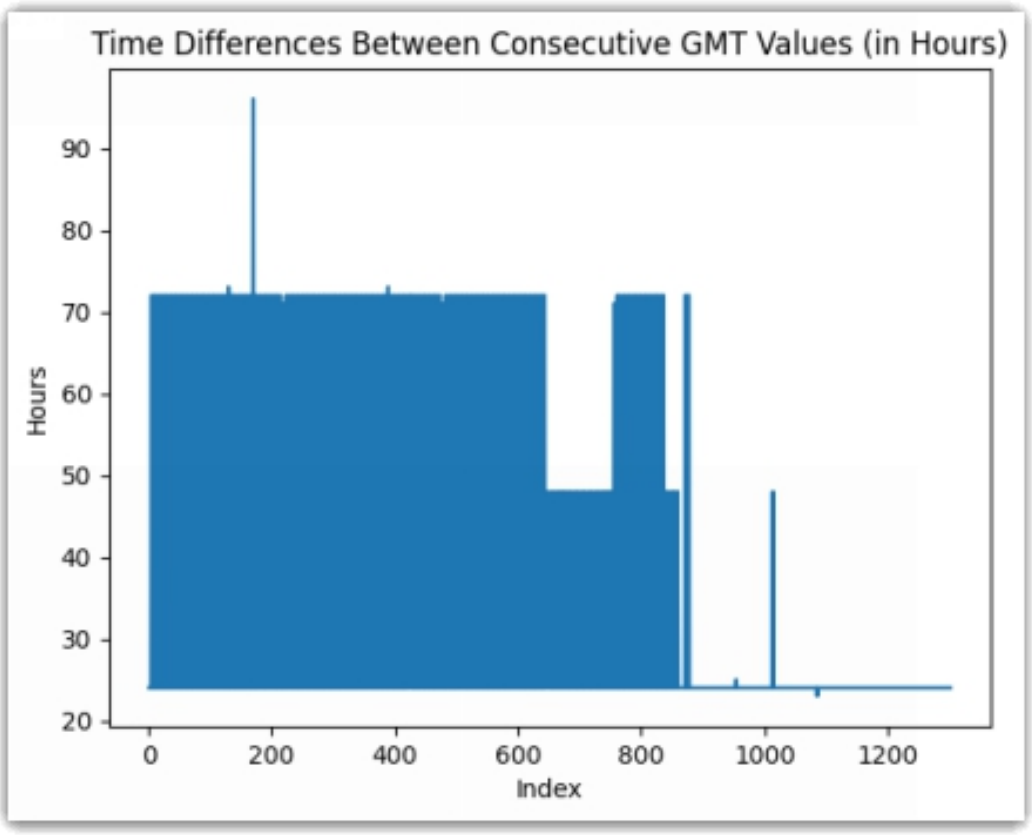
# Calculate the time differences in seconds
```

```
df['time_diff'] = df['Gmt time'].diff().dt.total_seconds()

# Convert the time differences from seconds to hours
df['time_diff_hours'] = df['time_diff'] / 3600

# Plot the time differences in hours
df['time_diff_hours'].plot(kind='line', title='Time Differences
Between Consecutive GMT Values (in Hours)')
plt.ylabel('Hours')
plt.xlabel('Index')
plt.show()
```

The results are displayed in the following figure:



If we examine the Y-axis, which shows the differences in hours, we notice a baseline around 24 hours. This is expected since we're dealing with daily candles, making 24 hours the most common interval. There's also a second baseline around 48

hours, likely corresponding to weekends when no trading occurs, resulting in two missing values. A third baseline appears at 72 hours, indicating three-day breaks, probably a weekend plus an additional holiday. We also see a spike above 90 hours, which is an outlier. While it doesn't seem critical, it’s isolated to a single data point around index 200. We could write a small piece of code to identify the exact row and investigate further by checking the date, possibly referencing a calendar to see if it coincides with bank holidays or other events.

On the X-axis, the range starts at zero and extends to about 1300, indicating the number of candles, or days, in our dataset. This suggests we have roughly four years of data, accounting for the removal of holidays and weekends.

You might wonder why we still see these spikes in the graph if we've already removed holidays and weekends. Remember, the plot reflects the differences between the indexes of the remaining candles. The dates of these candles remain unchanged, but by excluding weekends and holidays, we create these gaps in the GMT time column. This is important to keep in mind when interpreting the graph.

In conclusion, the data appears clean, with no significant anomalies that would likely impact our trading strategies. Any irregularities present are minimal and not clearly visible on the graph, suggesting they won't meaningfully alter our analysis or the strategies we plan to code based on this dataset.

1.3 Loading And Cleaning Data with Yfinance

yfinance is a popular Python library that simplifies the process of downloading historical market data from Yahoo Finance. Whether you're working with stocks, currencies, or other financial instruments, yfinance provides a convenient and efficient way to access data directly within your Python

environment. The library allows you to retrieve a wide range of data, including historical prices, trading volumes, dividends, stock splits, and even fundamental data like earnings and financial ratios.

One of the key advantages of yfinance is its ease of use. With just a few lines of code, you can download data for any asset available on Yahoo Finance and directly integrate it into your data analysis or backtesting pipeline. This makes yfinance an invaluable tool for both beginners and experienced traders who need reliable, up-to-date market data for their projects. Additionally, because Yahoo Finance is a free service, yfinance offers a cost-effective solution for accessing financial data without the need for expensive data subscriptions.

If you don't have the package installed you can execute a cell in Jupyter Notebook with the following content: !pip install yfinance

1.3.1 Downloading Data

Then to download data straight from python you can use the 'download' function. This snippet downloads the Bitcoin USD exchange data for the past month using 15-minute intervals. It then filters out candles where there was no price movement, specifically removing those where the High and Low values are equal.

```
import yfinance as yf
data = yf.download(tickers='BTC-USD', period='1mo',
interval='15m')
data = data[data["High"]!=data["Low"]]
```

1.3.2 Downloading Data for Multiple Tickers

You can download data for multiple tickers at once by passing a list of ticker symbols to the tickers parameter. This is useful if you want to compare different assets over the same period.

```
import yfinance as yf
# Download data for multiple tickers: Bitcoin, Ethereum, and Tesla
data = yf.download(tickers=['BTC-USD', 'ETH-USD', 'TSLA'],
period='1mo', interval='1d')
```

- **tickers=['BTC-USD', 'ETH-USD', 'TSLA']**: Specifies the list of tickers you want to download data for. In this case, Bitcoin, Ethereum, and Tesla.
- **period='1mo'**: Fetches data for the last month.
- **interval='1d'**: Sets the timeframe to daily candles.

This will return a multi-level DataFrame where each asset's data is grouped under its respective ticker symbol.

1.3.3 Specifying a Custom Date Range

If you need data for a specific date range, you can use the start and end parameters instead of the period parameter.

```
import yfinance as yf
# Download data for Bitcoin from January 1, 2021, to December
31, 2021
data = yf.download(tickers='BTC-USD', start='2021-01-01',
end='2021-12-31', interval='1d')
data.head()
```

- **start='2021-01-01'**: Specifies the start date for the data.
- **end='2021-12-31'**: Specifies the end date for the data.
- **interval='1d'**: Uses a daily timeframe.

This allows you to pull data for any custom date range, which is useful for analyzing specific periods of interest.

1.3.4 Downloading Intraday Data with Different Timeframes

yfinance supports various intraday timeframes such as 1 minute, 5 minutes, 15 minutes, 1 hour, etc. However, there are some limitations based on how far back you can go with these smaller intervals:

- **1-minute interval**: Limited to 7 days of data.
- **5-minute interval**: Limited to 60 days of data.
- **15-minute interval**: Limited to 60 days of data.
- **1-hour interval**: Limited to 730 days of data.

Here's how to download 1-minute interval data for the past week:

```
import yfinance as yf
# Download 1-minute interval data for Bitcoin for the last 7 days
data = yf.download(tickers='BTC-USD', period='7d', interval='1m')
print(data.head())
```

- **period='7d'**: Fetches data for the last 7 days.
- **interval='1m'**: Uses a 1-minute timeframe.

Keep in mind that for these smaller intervals, yfinance only provides data for the most recent days due to limitations in Yahoo Finance's data availability.

1.3.5 Handling Missing Data and Adjustments

Sometimes, financial data may have missing values, especially when dealing with intraday data or less liquid assets. yfinance offers parameters to handle these issues:

```
import yfinance as yf
# Download adjusted close prices and fill missing values forward
data = yf.download(tickers='BTC-USD', period='1mo', interval='1d',
auto_adjust=True, prepost=True)
```

```
print(data.head())
```

- **auto_adjust=True** : Automatically adjusts prices for dividends, stock splits, etc.
- **prepost=True** : Includes pre-market and after-hours data, if available.
- **data.fillna(method='ffill')** : Fills in missing data by carrying forward the last known value, if needed this function can be used to fill in missing values if you want to avoid removing rows from your data.

This helps in cleaning up the data and ensuring that any missing points don't disrupt your analysis.

1.3.6 Understanding the Limitations

While yfinance is a powerful tool, it does have some limitations:

- **Data Availability**: Not all timeframes are available for all periods. For instance, 1-minute data is only available for the past 7 days, while daily data is available for many years.
- **API Rate Limits**: Yahoo Finance imposes rate limits on how frequently you can request data, which can be a constraint if you're trying to download large datasets or frequent intraday data.
- **Data Gaps**: Intraday data might have gaps during non-trading hours or on holidays, which could require additional handling in your analysis.

1.4 Streaming Data Using Oanda's API

Using OANDA as a broker for algorithmic trading in Python provides traders with a powerful and flexible way to stream live

market data or download historical data directly into their trading systems. OANDA offers a comprehensive API that allows seamless integration with Python, enabling traders to access real-time market information and historical data for analysis or live trading execution. I've personally used this setup for my live trading and automated trading programs, and it has performed reliably as long as you avoid querying the market multiple times per second. So far, I haven't encountered any issues, making it a dependable option if you're planning to use it for your trading bots in the future.

The following Python code demonstrates how to connect to OANDA's API, retrieve the latest market data, and organize it into a Pandas DataFrame for further analysis.

```
from oanda_candles import Pair, Gran, CandleClient
import pandas as pd

access_token = "XXX"  # Your OANDA API token
accountID = "XXX"     # Your OANDA account ID

def get_candles(n):
    client = CandleClient(access_token, real=False)
    collector = client.get_collector(Pair.EUR_USD, Gran.M5)
    return collector.grab(n)

def get_candles_frame(n):
    candles = get_candles(n)

    dfstream = pd.DataFrame([{
        'Open': float(candle.bid.o.value),
        'Close': float(candle.bid.c.value),
        'High': float(candle.bid.h.value),
        'Low': float(candle.bid.l.value)
    } for candle in candles])

    return dfstream

get_candles_frame(5)
```

This code snippet starts by importing necessary modules and classes from the oanda_candles package. The access_token and accountID are placeholders where you need to insert your unique OANDA API token and account ID. These credentials are required to authenticate your connection to OANDA's servers.

The get_candles function sets up a connection to OANDA using the CandleClient class, specifying that the connection is to OANDA's demo environment (real=False). The function then collects a specified number (n) of candlestick data points for the EUR/USD currency pair using a 5-minute granularity (Gran.M5).

The get_candles_frame function calls get_candles to retrieve the data and then constructs a Pandas DataFrame from the retrieved candlesticks. It extracts and converts the bid prices (open, close, high, and low) from the candles into floating-point numbers, organizing them into a structured DataFrame. This DataFrame can then be used for further analysis, such as calculating technical indicators, backtesting strategies, or visualizing price movements.

The n parameter determines the number of candlestick data points you retrieve from the market. When you pass a value for n, the get_candles function will collect that specific number of recent candlesticks (or "candles") from OANDA's market data, each representing a set period (in this case, 5 minutes due to the Gran.M5 setting). This flexibility allows you to decide how much historical data you want to download, whether it’s a few recent candles for short-term analysis or a larger dataset for more comprehensive studies.

If you intend to use the data later on for different tests, you can save the DataFrame in CSV format and load it whenever needed. This approach allows you to automate data collection, for example, daily, and store the data for multiple assets. By doing so, you can easily access historical data without repeatedly querying the market, making your analysis and backtesting processes more efficient. The following snippet shows how to

save the data into a csv file:

```
# Retrieve the candles data and save it to a CSV file
dfstream = get_candles_frame(5)
dfstream.to_csv('candles_data.csv', index=False)
```

2 Charting And Visualization

Data visualization is a crucial aspect of analyzing financial data, as it allows traders and analysts to see trends, patterns, and anomalies that might not be immediately apparent from raw data alone. Python offers a variety of powerful libraries for visualizing data, each with its own strengths and use cases. Whether you're looking to create simple line charts, detailed candlestick charts, or interactive visualizations, Python provides the tools needed to turn your data into meaningful insights.

One of the most commonly used libraries for data visualization in Python is Matplotlib, which allows for the creation of static, interactive, and animated visualizations. For more advanced and interactive charts, Plotly is another excellent option, offering a wide range of features for creating visually appealing and informative charts that can be easily shared or embedded in web applications.

In financial analysis, candlestick charts are particularly valuable, as they provide a visual representation of price movements over time. These charts display the open, high, low, and close prices of an asset, making it easier to spot trends, reversals, and other key market signals.

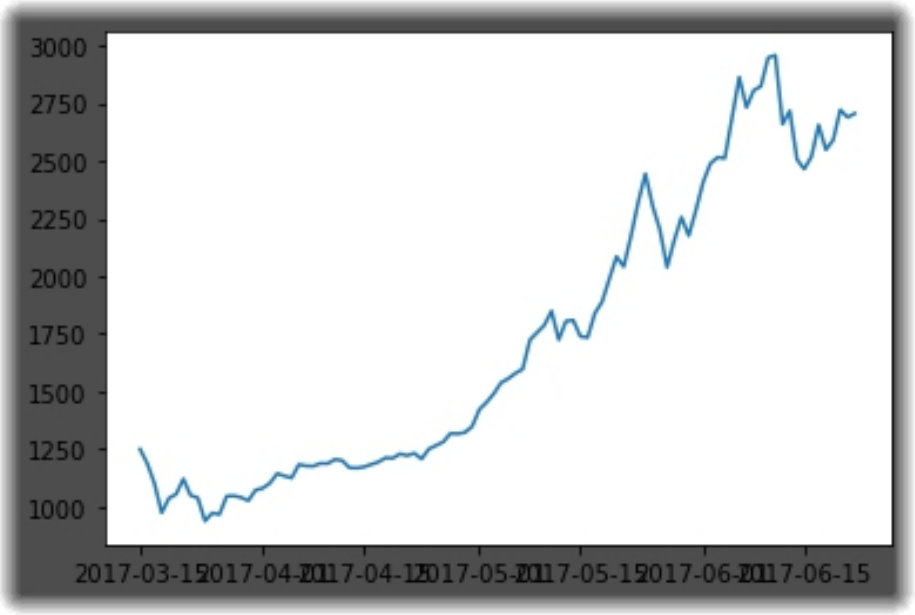
2.1 Simple Plot Line

One simple and quick way to plot a price is using the closing price versus the index, for example the following python snippet:

```
dfpl = data[0:100]
import matplotlib.pyplot as plt
```

```
plt.plot(dfpl.index, dfpl.Close) #,go
#color='green', marker='o', linestyle='dashed',linewidth=2,
markersize=12
plt.show()
```

Produces the following figure:



As you can see, this is a basic plot, it's not particularly aesthetic or impressive. However, it effectively charts price movements over time and serves its purpose if you need something quick without getting into too much detail.

2.2 Candlestick Chart

There is a better way visualizing the price with more information using candlesticks that include the open high low and closing prices for each candle. The following example shows how you can create a candlestick chart using the Plotly library:

```
dfpl = data[0:100]
import plotly.graph_objects as go

fig = go.Figure(data=[go.Candlestick(x=dfpl.index,
                open=dfpl['Open'],
```

```
                high=dfpl['High'],
                low=dfpl['Low'],
                close=dfpl['Close'])])

fig.show()
```

- dfpl = data[0:100] selects the first 100 rows from the DataFrame data. This subset of data is used to create a manageable and focused chart.
- import plotly.graph_objects as go imports the necessary components from the Plotly library. Plotly is a powerful visualization tool that allows for the creation of interactive charts, including candlestick charts.
- fig = go.Figure(data=[go.Candlestick(...)]) initializes a new figure for the candlestick chart.

The go.Candlestick() function takes several arguments:

x=dfpl.index: Specifies the x-axis values, which are typically the index or time.

open=dfpl['Open'], high=dfpl['High'], low=dfpl['Low'], close=dfpl['Close']: These arguments specify the open, high, low, and close prices for each candlestick.

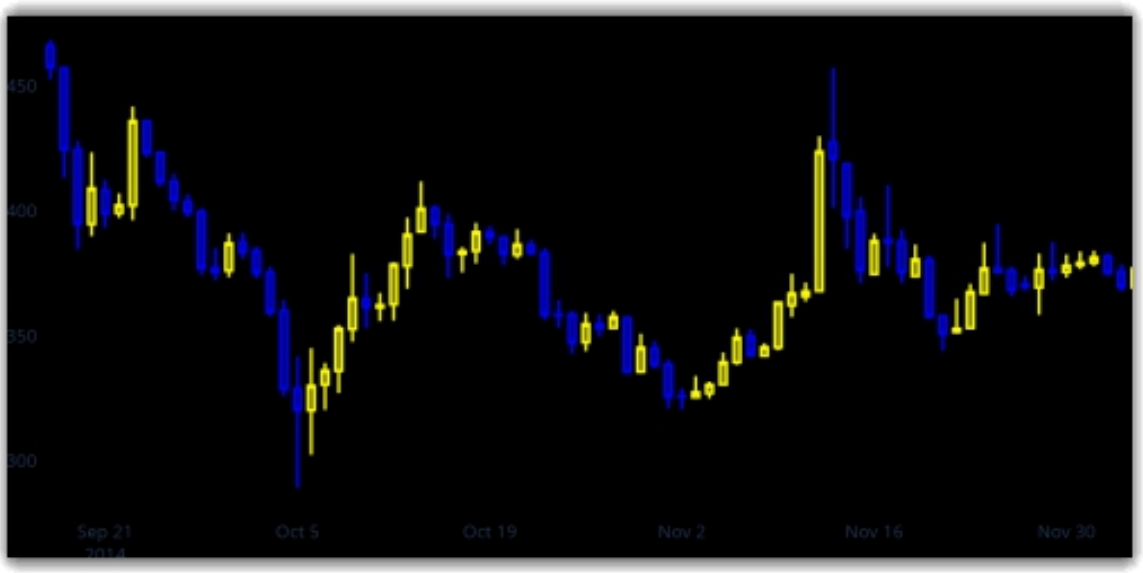
fig.show() renders the chart, allowing you to interact with it directly in your Python environment.

And the prices are now represented on the following chart:



It's also possible to change the design and the colors of the chart using additional functions and parameters. The following snippet for example plots candles with a darker style as seen on the figure below:

```
dfpl = data[0:100]
import plotly.graph_objects as go
fig = go.Figure(data=go.Candlestick(x=dfpl.index,
                open=dfpl['Open'],
                high=dfpl['High'],
                low=dfpl['Low'],
                close=dfpl['Close'],
                increasing_line_color= 'yellow', decreasing_line_color=
'blue'))
fig.update_layout(paper_bgcolor="black", plot_bgcolor="black",
                  margin_l=10, margin_b=0, margin_r=0, margin_t=0,
                 grid_columns=1,grid_rows=1)
fig.update_xaxes(showline=True, linewidth=2, linecolor='black',
gridcolor='black')
fig.update_yaxes(showline=True, linewidth=2, linecolor='black',
gridcolor='black')
fig.show()
```



Notice in the code how we used the increasing and decreasing line color parameters to change the colors of the candles. We also changed to background color and the margins. Then we also updated the axis attributes changing their line and grid colors to black matching with the background so they became invisible.

There's no right or wrong way to set up the appearance of your chart; it's all a matter of personal preference. You can find more examples with various color schemes and styles on the Plotly documentation pages.

In the next section we will see how to compute classic technical indicators and how to add these on the visualization charts.

3 Computing Technical Indicators

Technical indicators are essential tools in the arsenal of any trader or analyst, offering insights into market trends, momentum, volatility, and potential reversal points. These indicators help traders make informed decisions based on historical price data, enhancing the ability to detect current price

movements (notice I didn't say to predict future price movements, I don't want any trouble… more on this later in the strategies). In Python, the pandas_ta package provides a powerful and flexible framework for calculating a wide range of technical indicators with minimal effort. It allows you to compute indicators directly on your financial data. This makes it particularly useful for backtesting strategies, exploring new trading ideas, and conducting detailed market analysis.

One of the key advantages of using pandas_ta is its comprehensive selection of built-in indicators. The package includes a wide array of indicators such as moving averages, relative strength index (RSI), Bollinger Bands, MACD, and many more. Each of these indicators can be easily applied to your data, enabling you to analyze market conditions and refine your trading strategies.

Additionally, pandas_ta allows for customization and flexibility. You can fine-tune the parameters of each indicator to suit your specific needs, or even combine multiple indicators to create custom trading signals. It's a nice level of control and it's easily accessible for both beginners and experienced traders looking to implement sophisticated technical analysis in their trading systems.

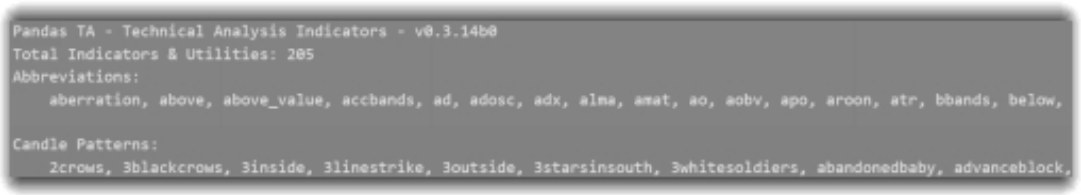
In the following sections, we will walk through examples of how to compute various technical indicators using the pandas_ta package. These examples will demonstrate how to apply the indicators to your data, visualize the results, and use them to enhance your trading strategies. In other words, it's a simple yet powerful way to leverage technical analysis in Python.

3.1 Accessing Pandas_TA functions

Suppose df a pandas dataframe with candlesticks data, meaning it contains the Open, High, Low and Close prices. We can list the indicators available in Pandas_TA calling the indicators() function:

```
df.ta.indicators()
```

Which will return the following result:



Then we can use the help function for any of the listed indicators to get information about the input paratmeters and the returned values, for example to get help for the rsi function we can run the following code:

```
help(ta.rsi)
```

And the returned result will detail the properties of the function and a small documentation explaining the computation behind, the input parameters and their default values:

```
Help on function rsi in module pandas_ta.momentum.rsi:

rsi(close, length=None, scalar=None, talib=None, drift=None, offset=None, **kwargs)
    Relative Strength Index (RSI)

    The Relative Strength Index is popular momentum oscillator used to measure the
    velocity as well as the magnitude of directional price movements.

    Sources:
        https://www.tradingview.com/wiki/Relative_Strength_Index_(RSI)

    Calculation:
        Default Inputs:
            length=14, scalar=100, drift=1
        ABS = Absolute Value
        RMA = Rolling Moving Average

        diff = close.diff(drift)
        positive = diff if diff > 0 else 0
        negative = diff if diff < 0 else 0

        pos_avg = RMA(positive, length)
        neg_avg = ABS(RMA(negative, length))

        RSI = scalar * pos_avg / (pos_avg + neg_avg)
...

    Returns:
        pd.Series: New feature generated.
```

We can observe that the rsi function requires the 'close' parameter, representing the closing prices, as its input. Additionally, it accepts a length parameter, which defaults to 14, along with other optional parameters. The function returns a Pandas Series containing the calculated RSI values, which can then be added to the DataFrame.

3.2 On Chart Indicators

In this paragraph, we'll explore how to compute and plot indicators directly on the candlestick chart, focusing on indicators like moving averages and Bollinger Bands. It's important to differentiate these indicators from others, such as RSI and volume, which require a separate scale due to their values not

aligning with the price scale on the same chart. We'll cover how to plot those in a separate frame in the following section. For now, we'll focus on examples involving moving averages and Bollinger Bands.

3.2.1 Computing and Visualizing Moving Averages

To compute the exponential moving average and add it to the dataframe we can run the following code:

```
import pandas_ta as ta
df["EMA"] = ta.ema(df.Close, length=10)
```

Notice how we provided the Close price column as input and we also changed the length of the candles window to 10 candles. The resulting dataframe has the following shape with a new column named "EMA":

	Gmt time	Open	High	Low	Close	Volume	EMA
0	07.05.2017 21:00:00.000	1559	1681	1557	1658	1.872044e+06	NaN
1	08.05.2017 21:00:00.000	1658	1753	1626	1747	1.330136e+06	NaN
2	09.05.2017 21:00:00.000	1713	1769	1677	1752	1.916895e+06	NaN
3	10.05.2017 21:00:00.000	1784	1887	1777	1857	1.613521e+06	NaN
4	11.05.2017 21:00:00.000	1836	1836	1651	1692	2.550428e+06	NaN
...	...	...	...	...	...	...	...
1301	12.10.2021 21:00:00.000	55433	57543	54328	57047	2.013182e+03	54074.141410
1302	13.10.2021 21:00:00.000	57078	58568	56906	57505	2.001020e+03	54697.933881
1303	14.10.2021 21:00:00.000	57533	62922	56940	62587	2.235774e+03	56132.309539
1304	15.10.2021 21:00:00.000	62587	62608	60223	61077	1.943311e+03	57031.344168
1305	16.10.2021 21:00:00.000	61077	61498	59043	59467	2.443833e+03	57474.190683

The first 10 rows have NaN as EMA values because since the length of the candles window is set to 10 we need at least 10 candles at the beginning of the dataframe to compute the first price average.

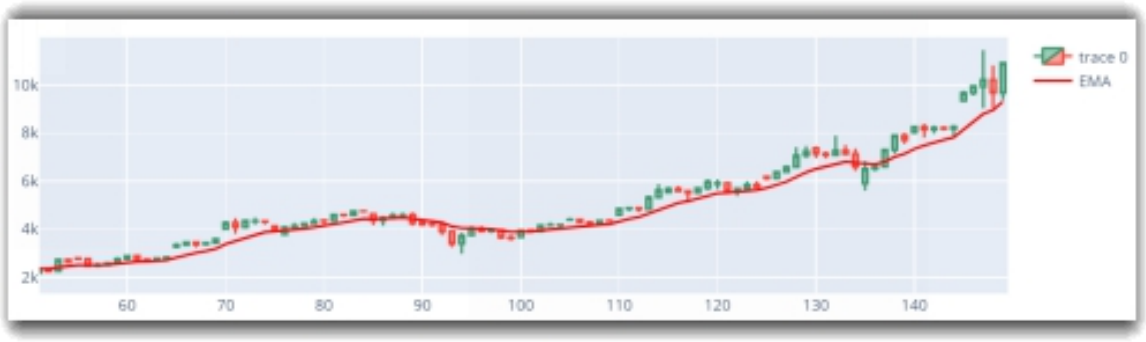
And now we can plot the curve on top of the candles chart using the following example:

```
dfpl = df[50:150]
import plotly.graph_objects as go

fig = go.Figure(data=[go.Candlestick(x=dfpl.index,
                open=dfpl['Open'],
                high=dfpl['High'],
                low=dfpl['Low'],
                close=dfpl['Close']),
                go.Scatter(x=dfpl.index, y=dfpl.EMA,
line=dict(color='red', width=2), name="EMA")])

fig.show()
```

Notice how we added the go.Scatter() function and we provided the index as well as the name of the column with the values of the moving average and then we provided the color and width parameters for drawing the curve. The result is the following:



3.2.2 Computing and Visualizing Bollinger Bands

To compute the Bollinger Bands we can use Pandas TA like in the following example:

```
import pandas_ta as ta
my_bbands = ta.bbands(data.Close, length=15, std=1.5)
```

```
data=data.join(my_bbands)
```

The dataframe we obtain has initial content with the addition of the Bollinger data:

BBL_15_1.5	BBM_15_1.5	BBU_15_1.5
55502.741482	62648.778646	69794.815809
55263.467513	62322.304427	69381.141341
55154.210233	61857.887760	68561.565288
54905.794326	61251.630990	67597.467653
54907.179869	60635.183854	66363.187839

Note that BBL represents the lower curve, BBM is the middle curve, and BBU is the upper curve. And we can visualize the curves using this example:

```
import plotly.graph_objects as go

df = data[100:500]
fig = go.Figure()

# Candlestick chart
fig.add_trace(go.Candlestick(x=df.index,
                open=df['Open'],
                high=df['High'],
                low=df['Low'],
                close=df['Close']))

# Upper Bollinger Band (BBU)
```

```
fig.add_trace(go.Scatter(x=df.index, y=df['BBU_15_1.5'],
name='BB_UPPER',
                         line=dict(color='blue')))

# Middle Bollinger Band (BBM)
fig.add_trace(go.Scatter(x=df.index, y=df['BBM_15_1.5'],
name='BB_MIDDLE',
                         line=dict(color='blue')))

# Lower Bollinger Band (BBL)
fig.add_trace(go.Scatter(x=df.index, y=df['BBL_15_1.5'],
name='BB_LOWER',
                         line=dict(color='blue')))

# Layout settings
fig.update_layout(
    xaxis=dict(rangeslider=dict(visible=False))
)

fig.show()
```

Note that we used the add_trace function this time to add curves on top of the candles chart and we also provided the name of the curve as well as the color, of course these attributes can be skipped, and default values will be used instead.



3.3 Different Scale Indicators

For some indicators, like the RSI, the ATR, or other oscillators, the scale doesn't match well with the candlestick chart. For example, the RSI ranges from 0 to 100, while Bitcoin prices in the sample chart vary by thousands. Because of this, it's not practical to plot both the price and RSI on the same chart. Instead, it's better to display them on separate grids, which you can easily do using Plotly.

The following code is an example how to split the canvas into two plotting areas one where we plot the price and another one for whichever indicator we want to add.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots
fig = make_subplots(rows=2, cols=1, subplot_titles=['Price',
'ATR'], shared_xaxes=True)

fig.add_trace(go.Candlestick(x=df.index,
                open=df['Open'],
                high=df['High'],
                low=df['Low'],
                close=df['Close']), row=1, col=1)
fig.add_trace(go.Scatter(x=df.index, y=df['ATR'], name='ATR'),
row=2, col=1)

fig.update_layout(
    xaxis=dict(rangeslider=dict(visible=False))
)

fig.show()
```

Note how in the first line of code after importing the needed libraries and packages we created sub plots using the make_subplots function and we provided the number of rows which is 2 and the number of columns which is one and we even provided the subplot titles as price and ATR. The shared xaxes

true parameter will actually make both plots share only the X axis The Y axis scales will be different.

Then we used the add_trace function and so we provided the X&Y values for the ATR and we also provided where this trace will be added, precisely at row 2 and column 1.

And this is how we can split the canvas into different plotting areas if we have different indicators or different variables that we need to plot independently using different axes. The result is shown on the following figure:



So far, we've covered how to compute technical indicators using pandas_ta and how to plot these indicators on candlestick charts. The method described in this chapter is quite flexible, meaning you can apply it to any technical indicator you want to compute, add, and visualize on your charts. The process is straightforward: first, print the list of available indicators in pandas_ta. Then, choose the indicator and review its documentation using the help function. After that, you can compute the indicator values and visualize them by using the add_trace method, which is the simplest way to add any curve, point, or series of values to the chart. It's also a good idea to review the pandas_ta documentation and check out their forums for additional tips and ideas on how to use and combine indicators effectively.

This brings us to the end of this chapter, where we've learned how to query data from different sources, clean it, ensure it's ready for analysis or algorithmic trading tests, and visualize the data and technical indicators for better inspection.

Chapter 5: Advanced Indicators And Automated Patterns Detection

Before we dive into this chapter, it's important to address a common concern. I often hear feedback that classic indicators are "lagging" and therefore not effective for predicting future price movements or trends, leading some to believe they shouldn't be used for trading. But here's the reality: everything on a chart is based on past data, including the price itself. What you see on the chart has already happened; you're never looking at something that's about to happen in the future. Whether it's a candlestick, a line chart, or a classic technical indicator, it all represents historical data.

While it's true that analyzing historical data won't give you a crystal-clear view of the future, it's the only data we have to work with. Even approaches like price action, support and resistance levels, and other techniques that claim not to be lagging are still based on past events. In essence, all trading decisions are based on past data, making them inherently lagging.

However, this doesn't render them useless. We can still use these classic technical indicators to detect the current or most recent trends, and from there, we can make educated guesses about where the market might head in the near future. This is the risk we take as traders. Trading, at its core, is about placing informed bets.

Despite the inherent lag, these indicators can be valuable tools when used correctly. With today's tools in algorithmic trading, we can calculate probabilities and apply statistical methods to determine if certain rules and configurations offer a statistical edge in predicting short-term price movements, assuming no major events disrupt the current market conditions.

So, keep in mind that while these indicators are based on past data, they are no less effective than any other technical analysis method. The only way to truly predict the future would be to have insider information, which is not feasible in a purely technical approach. In trading, we make the best decisions we can based on the information available, which is always rooted in the past.

As you gain more experience in trading and coding, it becomes clear that basic indicators alone may not be enough. It's time to get creative by combining various indicators and automating pattern detection in the market. This chapter focuses on pushing the boundaries of what you can do with the tools provided in Python. By mixing different indicators, applying advanced techniques, and leveraging automation, you can develop more sophisticated trading strategies.

1 Introduction

This chapter aims to inspire experimentation and exploration of new possibilities. Whether you want to enhance your current strategies or develop entirely new ones, the tools and concepts discussed here will help you approach the market with a fresh perspective. It's not just about following the rules, it's about seeing the market in a more nuanced way and finding creative solutions that work for you.

The power of technical analysis lies in how creatively you can combine indicators and automate pattern detection to uncover trading opportunities that might not be immediately

obvious. To stand out and compete in the market, you'll need more than just the classic technical indicators; you'll need something different, something better.

In this chapter, we'll explore how to build on what you already know by using advanced indicators and automated pattern detection to take your trading to the next level.

Keep in mind that these methods will significantly impact your trading strategy, as they are closely tied to the signals for entering positions.

2 Combining and layering classic indicators

2.1 Windowed Moving Averages

In this paragraph, I'll use the example of two moving averages for trend detection. Notice that I'm saying "trend detection" and not "trend forecasting" because we're focusing on identifying the current and past trends with these indicators. Predicting or forecasting future trends isn't really possible, we can only hope that the trend, which has been in place for a few hours, will continue for at least another hour. We base our trading decisions on this assumption for the next hour. This mindset is key when dealing with algorithmic trading.

We're all familiar with the moving average crossover strategy, where a fast moving average crossing above a slow moving average signals an uptrend, and the reverse signals a downtrend. In algorithmic trading, the program will detect every single crossover, which can happen frequently, especially when the market is ranging. In such conditions, moving averages might cross each other multiple times in a short period and in opposite directions, simply because there's no clear trend.

If you look at the left side of the following figure, you'll see

the moving averages crossing several times without a clear price trend, which can be challenging for trading algorithms and may generate false signals, leading to unnecessary losses. However, on the right side of the figure, there's a clearer trend, with the moving averages spreading apart as the momentum becomes more defined.



To address the issue of frequent crossover signals in ranging markets, we need to examine the positions of the moving averages over a slice of past candles. For example, if the fast moving average stays below the slow moving average for five or ten consecutive candles, we can confirm a downtrend. Conversely, if the fast moving average remains above the slow moving average within that slice of 10 candles, we can confirm an uptrend. While trends can change, this approach helps filter out those uncertain periods in ranging markets, where the trend is indecisive, something we want to avoid if we're targeting trending markets.

In the following code, the ema_signal() function compares two columns in the DataFrame, EMA_fast and EMA_slow, and tests the positions of these moving averages over a slice of past candles. The length of this slice is defined by the input parameter backcandles. The function takes two input parameters: the DataFrame and the number of back candles. For each current candle, it looks back at the previous candles within the slice to compare the moving averages' positions. It generates

a signal: 1 for a confirmed downtrend, 2 for an uptrend, and 0 if no trend is confirmed within that slice. The function then returns the DataFrame with an additional column called EMA_signal that contains the trend category result.

```
def ema_signal(df, backcandles):
    # Create boolean Series for conditions
    above = df['EMA_fast'] > df['EMA_slow']
    below = df['EMA_fast'] < df['EMA_slow']

    # Rolling window to check if condition is met consistently over the window
    above_all = above.rolling(window=backcandles).apply(lambda x: x.all(), raw=True).fillna(0).astype(bool)
    below_all = below.rolling(window=backcandles).apply(lambda x: x.all(), raw=True).fillna(0).astype(bool)

    # Assign signals based on conditions
    df['EMASignal'] = 0  # Default no signal
    df.loc[above_all, 'EMASignal'] = 2  # Signal 2 where EMA_fast consistently above EMA_slow
    df.loc[below_all, 'EMASignal'] = 1  # Signal 1 where EMA_fast consistently below EMA_slow

    return df
```

Needless to say the dataframe has to contain the 2 moving average columns:

```
import pandas_ta as ta
df["EMA_slow"]=ta.ema(df.Close, length=50)
df["EMA_fast"]=ta.ema(df.Close, length=30)
```

2.2 Layering Windowed RSI

We can apply the same method to calculate a windowed RSI signal. For example, if the RSI has been above 50 for the last

few candles, we can confirm an uptrend in momentum. Conversely, if the RSI has been below 50 for the last few candles, we confirm a downtrend in momentum. While 50 is a simple threshold to use in this example, it's possible to adjust it for both upward and downward momentum. For instance, we might confirm an uptrend if the moving averages signal an uptrend and the RSI has been consistently above 45 for the last few candles. Similarly, in a downtrend, we could confirm the momentum if the RSI has been consistently below 55 for the last few candles.

The following function demonstrates how to calculate windowed RSI signals using a threshold of 50 and a default of 5 backcandles. The function adds a new column to the DataFrame called RSISignal, which can have three possible values: 0 if there's no clear signal, 1 if there's downward momentum (meaning the RSI has been consistently below 50 for the last few candles), and 2 if the RSI indicates an uptrend.

```
def rsi_signal(df, backcandles=5):
    rsi_series = df['RSI']

    # Create boolean Series for conditions
    above_50 = rsi_series.gt(50.1)
    below_50 = rsi_series.lt(49.9)

    # Rolling window to check if condition is met consistently over
the window
    above_all =
above_50.rolling(window=backcandles).apply(lambda x: x.all(),
raw=True).fillna(0).astype(bool)
    below_all =
below_50.rolling(window=backcandles).apply(lambda x: x.all(),
raw=True).fillna(0).astype(bool)

    # Initialize the RSISignal column in the DataFrame with 0 (no
signal)
    df['RSISignal'] = 0
```

```
    # Assign signals based on conditions
    df.loc[above_all, 'RSISignal'] = 2  # Signal 2 where RSI
consistently above 50.1
    df.loc[below_all, 'RSISignal'] = 1  # Signal 1 where RSI
consistently below 49.9

    return df
```

To combine and layer the EMA signal with the RSI, we can use both in a single conditional statement. Assuming df is our DataFrame, we can apply the functions as shown in this example:

```
df = ema_signal(df, 7)
df = rsi_signal(df, 7)
df['TotalSignal'] = df.apply(lambda row: row['EMA_Signal']
                if row['EMASignal'] == row['RSISignal']
                else 0, axis=1)
```

First, we apply the ema_signal function to the DataFrame df with a window of 7 candles. The function calculates and adds an EMASignal column to df, indicating whether the market is in an uptrend, downtrend, or no clear trend based on the EMA. Then we apply the rsi_signal indicating the RSI-based trend. Then the last line creates a new column TotalSignal in the dataframe, for each row, it checks if the EMASignal matches the RSISignal. If they match, the value in TotalSignal is set to the value of EMASignal; otherwise, it's set to 0. This effectively filters the signal, ensuring that only when both the EMA and RSI signals agree, the TotalSignal is confirmed, otherwise, it resets the signal to 0.

3 Regression And Slopes Measurements

Comparing the levels of two indicators is a classic method for

assessing trend momentum, as we did in the previous section by comparing two moving averages and checking RSI values against a certain threshold. Another powerful approach is to use regression to measure the rate of change of any variable over time. For example, we can use regression to determine how a moving average is increasing or decreasing and at what speed. Similarly, this method can be applied to measure how quickly the price, middle price, or closing price of candles is changing within a specific time window. In this section, we will focus on using regression in Python to calculate these rates of change and explore how this technique can be leveraged to enhance trend analysis.

3.1 Linear Regression

Regression is a fundamental statistical tool used to model the relationship between a dependent variable and one or more independent variables. In the context of trading and financial analysis, regression is often used to understand and quantify how various factors influence market trends or to predict future price movements based on historical data.

In its simplest form, linear regression fits a straight line through a set of data points, providing a clear view of the trend over time. Essentially, it finds the line that best represents the data by being as close as possible to all the points simultaneously. The slope of this line represents the rate at which the dependent variable is increasing or decreasing, offering valuable insight into the strength and direction of a trend.

For our purposes, we will focus on linear regression, as it is particularly well-suited for measuring the rate of change in variables and indicators over time. By calculating the slope of a linear fit, we can efficiently determine whether an indicator is trending upward or downward and at what rate. This information is crucial for developing and refining trading strategies.

A simple example of how to use linear regression in Python to calculate the slope of a trend:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Example data: closing prices over time
data = {
    'time': np.arange(10),  # Time points (e.g., days)
    'close_price': [100, 102, 104, 103, 105,
                    107, 106, 108, 110, 109]  # Example closing prices
}
df = pd.DataFrame(data)

# Reshape time data to fit the model
X = df['time'].values.reshape(-1, 1)  # Independent variable (time)
y = df['close_price'].values  # Dependent variable (price)

# Apply linear regression
model = LinearRegression()
model.fit(X, y)

# Get the slope of the fitted line
slope = model.coef_[0]

print(f'The slope of the trend is: {slope}')
# The slope of the trend is: 1.0181818181818183
```

We start with a simple dataset where time represents the time points (e.g., days) and close_price represents the corresponding closing prices of an asset. The time data (X) is reshaped to fit the linear regression model. This is necessary because the model expects a 2D array as input. We use the LinearRegression class from the sklearn.linear_model module to fit a linear model to the data. This model will calculate the best-fit line through the data points. The slope of the fitted line is obtained using the model.coef_[0] attribute. The code prints the slope, which quantifies the trend's strength and direction. A

positive slope indicates an upward trend, while a negative slope indicates a downward trend. It indicates the rate at which the closing price is increasing or decreasing over time.

I ran the example and got a slope of 1.018 which is positive value indicating an increasing overall trend. We can visualize this plotting the points and the fitted line for further inspection. To do this we will modify the code adding the intercept parameter and the predicting the trend line. Then we add the plotting part in the code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Example data: closing prices over time
data = {
    'time': np.arange(10),  # Time points (e.g., days)
    'close_price': [100, 102, 104, 103, 105, 107, 106, 108, 110,
109]  # Example closing prices
}
df = pd.DataFrame(data)

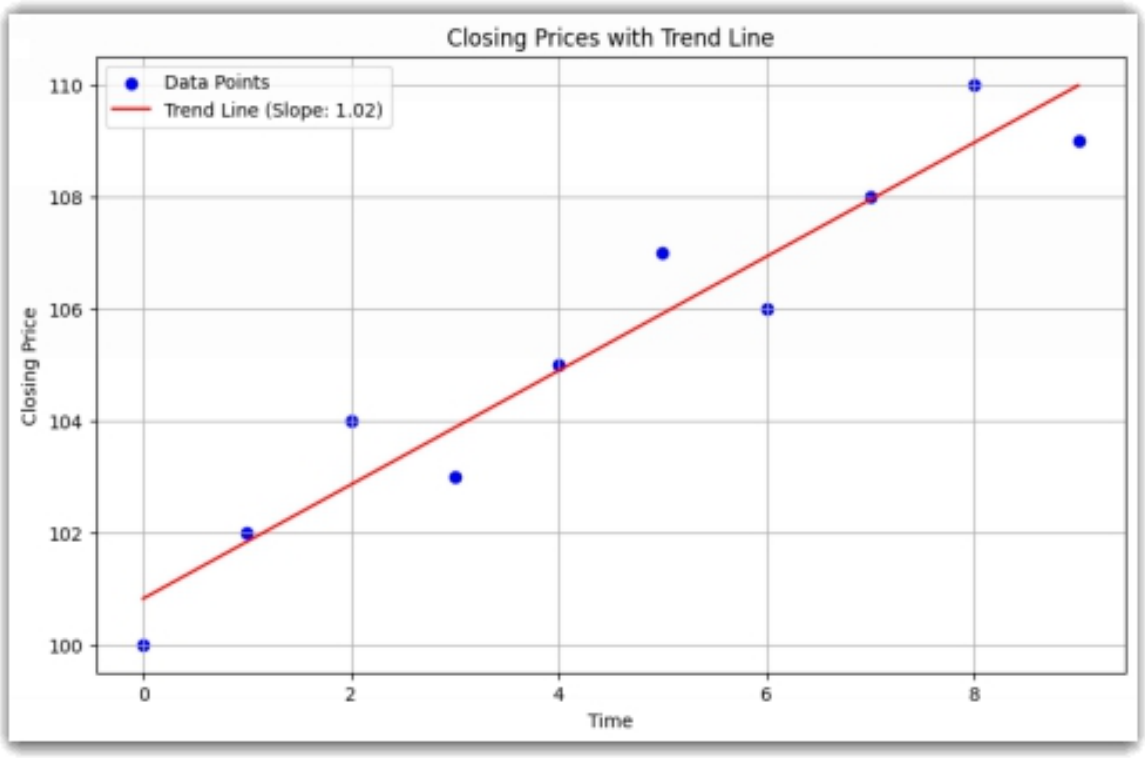
# Reshape time data to fit the model
X = df['time'].values.reshape(-1, 1)  # Independent variable (time)
y = df['close_price'].values  # Dependent variable (price)

# Apply linear regression
model = LinearRegression()
model.fit(X, y)

# Get the slope and intercept of the fitted line
slope = model.coef_[0]
intercept = model.intercept_

# Predict the trend line
trend_line = model.predict(X)
```

```
# Plotting the data points and the trend line
plt.figure(figsize=(10, 6))
plt.scatter(df['time'], df['close_price'], color='blue', label='Data
Points')
plt.plot(df['time'], trend_line, color='red', label=f'Trend Line
(Slope: {slope:.2f})')
plt.title('Closing Prices with Trend Line')
plt.xlabel('Time')
plt.ylabel('Closing Price')
plt.legend()
plt.grid(True)
plt.show()
```



Now we can visually confirm the positive trend obtained in numbers.

3.2 Using Regression For Trend And Momentum Detection

As noted in the previous paragraph, regression allows us to calculate the slopes of any indicator or series, such as closing or middle prices. This enables us to estimate whether the trend is moving in a positive or negative direction and to gauge the momentum of this movement.

In this section, we'll apply regression to exponential moving averages (EMAs). We'll plot three EMAs of different lengths and compute their slopes at each point or candle. The idea is that if all three slopes are positive, we can confirm an uptrend. Conversely, if all three slopes are negative, indicating they are pointing downward, we can confirm a downtrend.

It's important to note that this is not a complete trading strategy but rather an example of how regression can be used on indicators to assess the trend.

```
import pandas as pd
import pandas_ta as ta
from sklearn.linear_model import LinearRegression
import numpy as np

# Function to calculate the slope of a series over a specified
number of backcandles
def calculate_slope(series, backcandles=7, startindex=87):
    slopes = [np.nan] * startindex  # Initialize with NaN for the first
rows

    for i in range(startindex, len(series)):
        # Select the range of data to compute the slope
        y = series[i-backcandles:i].values
        x = np.arange(backcandles).reshape(-1, 1)

        # Perform linear regression to get the slope
        model = LinearRegression()
```

```
        model.fit(x, y)
        slopes.append(model.coef_[0])

    return slopes

# Function to add EMA slopes to the DataFrame
def add_ema_slopes(df, backcandles):
    df['EMA_20'] = ta.ema(df['Close'], length=20)
    df['EMA_50'] = ta.ema(df['Close'], length=50)
    df['EMA_80'] = ta.ema(df['Close'], length=80)

    # Calculate slopes and add them as new columns
    df['Slope_EMA_20'] = calculate_slope(df['EMA_20'],
backcandles)
    df['Slope_EMA_50'] = calculate_slope(df['EMA_50'],
backcandles)
    df['Slope_EMA_80'] = calculate_slope(df['EMA_80'],
backcandles)

    return df

# Example usage
df = pd.read_csv("BTCUSD_Candlestick_1_D_ASK_08.05.2017-
16.10.2021.csv")
df = df[df['Volume'] != 0]
df = df[df["High"] != df["Low"]]
df = df.reset_index(drop=True)

# Add EMA slopes with a specified number of backcandles (e.g., 7)
df = add_ema_slopes(df, backcandles=7)

df.tail(15)
```

The calculate_slope function takes three input parameters: a series of values (representing the coordinates of the points we want to fit the slope to), the number of back candles to consider for each fitting operation, and a starting index. The back candles parameter defines the width of the sliding window used to compute the slopes at each step, allowing us to find the slope

for the specified number of back candles.

The starting index is necessary because the initial values of the moving averages are NaN, depending on the length of the moving average. For instance, with an EMA of length 80, we need at least 80 values before we can calculate the first EMA value. Therefore, the slope calculation needs to start at index 80 plus the number of back candles.

The function utilizes linear regression from the Scikit-learn package and returns a list of slopes that can be added to the DataFrame. This way, we obtain the slope of the last few back candles at each position in the DataFrame, or for each candle, and this process is repeated for all three moving averages, resulting in three slopes in total. Below is a small sample of the resulting DataFrame:

	Gmt time	Open	High	Low	Close	Volume	EMA_20	EMA_50	EMA_80	Slope_EMA_20	Slope_EMA_50	Slope_EMA_80
1288	02.10.2021 21:00:00.000	48077	49263	47185	48516	2105.904	45323.452957	45079.076568	44310.649299	-67.201230	-34.370309	-2.879167
1289	03.10.2021 21:00:00.000	48514	49551	46989	49062	2246.872	45679.505057	45235.269643	44427.966600	64.175077	18.438331	29.593158
1290	04.10.2021 21:00:00.000	49087	51831	48641	51534	2166.265	46237.076004	45482.278677	44603.424215	197.069968	74.953354	64.901263
1291	05.10.2021 21:00:00.000	51564	55461	50481	55000	2615.271	47071.640194	45855.522650	44860.129790	322.716366	131.478993	100.739680
1292	06.10.2021 21:00:00.000	54998	55813	53467	54247	2455.656	47755.007794	46184.600194	45091.904363	435.325011	185.346763	135.415420

As we mentioned earlier, slopes can be utilized in various ways. One straightforward approach is to check if all three slopes are pointing in the same direction at a given moment, which can help us estimate the current trend. Later in this chapter, we'll also use regression to detect price channels by fitting maxima and minima into two separate slopes. Additionally, regression can be applied to identify different patterns, such as flag or triangle patterns. The possibilities for using regression to detect price patterns and incorporate them into your systematic trading strategy are virtually limitless.

4 Fractals, Minimas And Maximas

In financial trading, fractals are used as technical indicators to identify potential reversals or continuation patterns in the market. A fractal occurs when a specific candle stands out as

either a high point (bearish fractal) or a low point (bullish fractal) compared to the surrounding candles.

For a bearish fractal, the candle's high must be higher than the highs of the two candles before it and the two candles after it. Conversely, a bullish fractal occurs when a candle's low is lower than the lows of the two candles before and after it. Traditionally, two candles on each side are used for comparison, but in this example, we'll create a more flexible function that allows you to specify the number of candles before and after to compare.

It's important to note that when we refer to the "current candle," we're not talking about the most recent candle in real-time, because we can't compare it with future candles that haven't formed yet. Instead, we're looking at a specific candle in the past, with its "left" being the earlier candles and its "right" being the candles that follow.

This concept is crucial when using fractal detection in real-time. To accurately identify fractals, you need to leave a gap and start testing candles from a few periods back, ensuring there are enough candles on both sides for comparison.

In the following example, we'll define a function to detect fractal candles and visualize them on a chart. We'll mark bearish fractals with a triangle above the candle and bullish fractals with a triangle below the candle.

```
def isPivot(candle, window):
    """
    function that detects if a candle is a pivot/fractal point
    args: candle index, window before and after candle to test if
pivot
    returns: 1 if pivot high, 2 if pivot low, 3 if both and 0 default
    """
    if candle-window < 0 or candle+window >= len(df):
        return 0
```

```
    pivotHigh = 1
    pivotLow = 2
    for i in range(candle-window, candle+window+1):
        if df.iloc[candle].Low > df.iloc[i].Low:
            pivotLow=0
        if df.iloc[candle].High < df.iloc[i].High:
            pivotHigh=0
    if (pivotHigh and pivotLow):
        return 3
    elif pivotHigh:
        return pivotHigh
    elif pivotLow:
        return pivotLow
    else:
        return 0
```

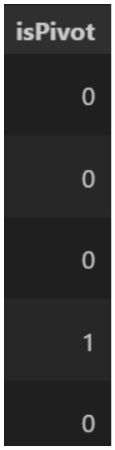
This function evaluates whether a given candle in a series forms a fractal or pivot point. It examines a specified number of candles before and after the current candle within a window to determine if the current candle qualifies as a pivot. The function returns:

- 1 (Bearish Pivot): If the current candle's high is greater than the highs of its neighboring candles, indicating a potential reversal to the downside.
- 2 (Bullish Pivot): If the current candle's low is lower than the lows of its neighboring candles, signaling a possible reversal to the upside.
- 3 (Extreme Candle): If the current candle's high is higher and its low is lower than those of its neighbors, indicating an extreme fluctuation.
- 0 (No Pivot): If none of these conditions are met.

To utilize this function, it can be applied directly to a DataFrame or a Series within a DataFrame, as shown in the following example:

```
window=3
df['isPivot'] = df.apply(lambda x: isPivot(x.name,window), axis=1)
```

And now the data frame will contain a column named isPivot that has one of four values either 0, 1, 2 or 3:



Now in order to be able to plot or to mark these candles on a chart we will generate one more column where if the isPivot has a bearish signal (value 1) we will place a coordinate of a point just above the high of the candle, and if the isPivot has a bullish signal (value 2) we will place a coordinate of a point just below the low of the candle. This can be done using the following function:

```
def add_pointpos_column(df, signal_column):
    """
    Adds a 'pointpos' column to the DataFrame to
    indicate the position of support and resistance points.

    Parameters:
    df (DataFrame): DataFrame containing the stock data
                    with the specified SR column, 'Low', and 'High'
columns.
    sr_column (str): The name of the column to consider
                     for the SR (support/resistance) points.

    Returns:
```

```
    DataFrame: The original DataFrame with an additional 'pointpos' column.
    """
    def pointpos(row):
        if row[signal_column] == 2:
            return row['Low'] - row['Low']*0.06
        elif row[signal_column] == 1:
            return row['High'] + row['High']*0.06
        else:
            return np.nan

    df['pointpos'] = df.apply(lambda row: pointpos(row), axis=1)
    return df
```

This function processes a DataFrame and uses a specified signal column to generate position coordinates for plotting points on a chart. Within this function, there's an inner function called pointpos(), which calculates the exact position of these points based on the signal values.

Here's how pointpos() works:

- If the signal column value is 2, the function returns a coordinate equal to the low of the candle minus 6% of the low value, placing the point slightly below the candle.
- If the signal column value is 1, the function returns a coordinate equal to the high of the candle plus 6% of the high value, placing the point slightly above the candle.

These calculated coordinates are stored in a new column named pointpos in the DataFrame. The apply() function is used to apply pointpos() across all rows, generating the required coordinates for each candle. These coordinates are then used to plot symbols on the chart, positioned just above or below the corresponding candles.

If this concept seems unclear, don't worry, once you see the visualization generated by the next function, it will make the purpose and functionality of this process much clearer.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

def plot_candlestick_with_signals(df, start_index, num_rows):
    """
    Plots a candlestick chart with signal points.

    Parameters:
    df (DataFrame): DataFrame containing the stock data with
                    'Open', 'High', 'Low', 'Close', and 'pointpos' columns.
    start_index (int): The starting index for the subset of data to
plot.
    num_rows (int): The number of rows of data to plot.

    Returns:
    None
    """
    df_subset = df[start_index:start_index + num_rows]

    fig = make_subplots(rows=1, cols=1)

    fig.add_trace(go.Candlestick(x=df_subset.index,
                                 open=df_subset['Open'],
                                 high=df_subset['High'],
                                 low=df_subset['Low'],
                                 close=df_subset['Close'],
                                 name='Candlesticks'),
                  row=1, col=1)

    # Flags to control legend display
    added_above_legend = False
    added_below_legend = False

    for i in range(len(df_subset)):
        if df_subset['pointpos'].iloc[i] > df_subset['High'].iloc[i]:
```

```
        # Plot downward triangle above the candle
        fig.add_trace(go.Scatter(x=[df_subset.index[i]],
                                 y=[df_subset['pointpos'].iloc[i]],
                                 mode="markers",
                                 marker=dict(size=12,
                                             color="MediumPurple",
                                             symbol='triangle-down'),
                                 name="Signal Above",
                                 showlegend=not added_above_legend),
                      row=1, col=1)
        # Disable further legend entries for above signals
        added_above_legend = True

    elif df_subset['pointpos'].iloc[i] < df_subset['Low'].iloc[i]:
        # Plot upward triangle below the candle
        fig.add_trace(go.Scatter(x=[df_subset.index[i]],
                                 y=[df_subset['pointpos'].iloc[i]],
                                 mode="markers",
                                 marker=dict(size=12,
                                             color="MediumPurple",
                                             symbol='triangle-up'),
                                 name="Signal Below",
                                 showlegend=not added_below_legend),
                      row=1, col=1)
         # Disable further legend entries for below signals
        added_below_legend = True

fig.update_layout(
    width=1200,
    height=800,
    plot_bgcolor='black',
    paper_bgcolor='black',
    font=dict(color='white'),
    xaxis=dict(showgrid=False, zeroline=False),
    yaxis=dict(showgrid=False, zeroline=False),
    showlegend=True,
```

```
    legend=dict(
        x=0.01,
        y=0.99,
        traceorder="normal",
        font=dict(
            family="sans-serif",
            size=12,
            color="white"
        ),
        bgcolor="black",
        bordercolor="gray",
        borderwidth=2
    )
)

fig.show()
```

This function plots the chart of the candles and it overlays the pivots or fractal candles signals using triangles above and below the candles. The results are shown in the figure below:



As you've noticed, the candles marked with triangles have either high or low extremes. This means they are either higher or lower than their neighboring candles on both the left and right sides. We're interested in these particular candles because they help us identify price patterns, which are crucial in many advanced indicators and trading strategies. In this book, we'll use them in upcoming sections to detect specific patterns such as price channels, flags, and triangular patterns.

The plot_candlestick_with_signals() function takes three input parameters: the DataFrame where prices are stored, the start index of the candle (which will be the first candle on our chart), and the number of rows or candles to be plotted after the start index (e.g., 50, 100, or 200 candles). The function slices the DataFrame accordingly, plots the candlesticks, and overlays the triangles based on the coordinates from the pointpos column. The final part of the function is purely aesthetic—it adjusts the background color to black, sets font sizes, configures the legend, and manages the gridlines on the X and Y axes.

A quick reminder: the isPivot function uses a parameter called window, which we've set to 3 for these examples. However, if you want to be more selective and plot only strong fractals or strong pivot points, you can increase this window to 5 or 7. In this case, the function will only identify pivot points that are higher or lower than seven neighbors on both sides.

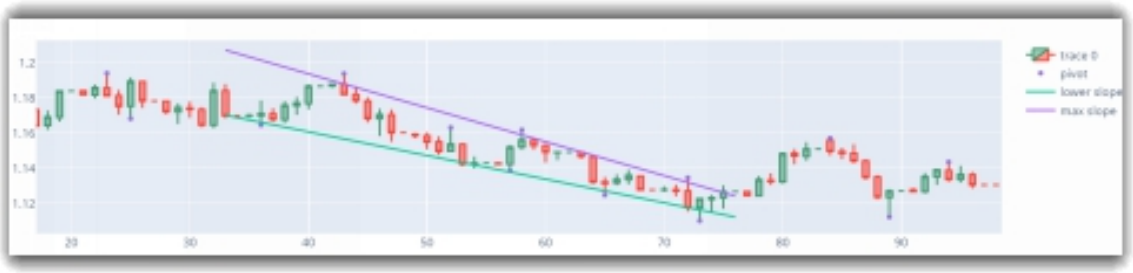
The figure below shows the results with a window set to 7. As you can see, the function only highlighted the most significant pivot point, resulting in a single pivot being detected within the same DataFrame slice, where a true price swing occurred:



In the next section, we'll combine fractals with regression analysis to identify complex price patterns, including channels, flags, triangles, ascending triangles, and descending triangles. You can also expand this approach to detect any specific pattern you wish.

5 Price Patterns Detection

5.1 Price Channels



When coding algorithms or algorithmic solutions, there are often multiple ways to address the same problem. This variety is beneficial because it means there is always room for

improvement, regardless of what others are using. In this section, we'll focus on detecting price channels, a pattern of interest in trading. Identifying a valid price channel allows traders to either trade within the channel or wait for a breakout and trade in the direction of the breakout. Several strategies rely on channel detection.

In the accompanying figure, a price channel was detected by the algorithm. As you can see, it's not perfect, but this approximation is intentional. We've allowed for some flexibility in our program to decrease the selectivity of the model, recognizing that the market is not perfect. The level of imperfections and parameters you choose to incorporate into your algorithm is up to you and depends on how you design your own intelligent detectors.

The algorithm we used for channel detection is straightforward. First, we detect fractals, represented by the purple points on the chart. After identifying these key points, we fit the maxima into one slope and the minima into another. There are a few conditions to consider. We need at least three points on each side to apply the fitting, or at least three points on one side and a minimum of two on the other if we want the model to be more sensitive. Increasing the minimum number of points to four on each side will result in fewer detected channels. Essentially, the more conditions you add, the fewer signals you will detect. Once we have identified the two slopes, we compare their values. The slopes need to be nearly parallel; ideally, they should be completely parallel. If the difference between the slopes is within a small percentage, we have detected a parallel price channel.

There are two important considerations to keep in mind. First, the detection process works in a windowed approach, meaning that for each candle, we analyze a slice of candles before the current one, which we consider the present candle. The window needs to be wide enough to include at least five

extrema points to apply the fitting.

Second, it's crucial to avoid a look-ahead bias. Since fractal detection compares each candle with past and future candles to check if it's an extrema, we need to shift the window for fractal detection and fitting into the past by the same number of candles used for fractal detection. This adjustment ensures that, when trading in real-time, we only use past data and not future candles, aligning our coding with real-world trading scenarios.

In the following example, I will demonstrate how to implement this in Python.

```
def collect_channel(candle, backcandles, window):
    """
    Collect pivot points data for the specified
    candle index and calculate linear regressions.

    Parameters:
    candle (int): The current index of the candle.
    backcandles (int): Number of candles to look back.
    window (int): Window size to identify pivot points.

    Returns:
    tuple:  Slope and intercept of lows,
            slope and intercept of highs,
            and R-squared values for both.
    """
    local_df = df.iloc[candle - backcandles - window : candle -
window].copy()
    local_df['isPivot'] = local_df.index.map(lambda idx: isPivot(idx,
window))

    highs = local_df[local_df['isPivot'] == 1]['High']
    lows = local_df[local_df['isPivot'] == 2]['Low']

    if len(lows) >= 2 and len(highs) >= 2:
        sl_lows, interc_lows, r_value_l, _, _ =
stats.linregress(lows.index, lows)
```

```
        sl_highs, interc_highs, r_value_h, _, _ =
stats.linregress(highs.index, highs)

        return sl_lows, interc_lows, sl_highs, interc_highs,
r_value_l**2, r_value_h**2

    return 0, 0, 0, 0, 0, 0
```

This function is designed to collect pivot points for a specific candle or the candles preceding it, and it also applies linear regression fitting. The first line of the code slices a portion of the DataFrame to create the local_df variable. We begin this slice at the current candle's index minus the back candles and minus a window. This might seem confusing because we're working with two different windows. The window is used for detecting fractals, while backcandles refers to the wider window within which we search for extremes and apply the fitting. The slicing starts at candle - backcandles - window and, importantly, stops at candle - window to avoid look-ahead bias. It's crucial to understand that we are not slicing up to the current candle but rather stopping at the candle before it, at candle - window. This approach prevents the look-ahead bias.

Once we have the local_df, we apply the isPivot function to detect extrema, splitting them into highs and lows, stored in the highs and lows variables. The function requires at least two lows and two highs to proceed with the fitting using linear regression. We provide the indices and values of the minima and maxima to the regression function.

The function then returns the slopes and intercepts for the minima and maxima, along with the R-squared values. The R-squared values, which range from 0 to 1, indicate the goodness of fit. A value of 0 indicates a poor fit, while a value of 1 indicates a perfect fit where the slope passes through all the points. These values help us filter out poor fits—if the R-squared values are below a certain threshold, such as 0.9, we can discard the detected channel, as it suggests the points are not well-

aligned, and the fit is not reliable for identifying a proper price channel.

If the conditions for fitting are not met, the function returns zeros by default.

Now, we can use this function to plot the results. We'll define a new function called plot_candlestick, which takes three parameters: the index of the current candle, the number of backcandles, and the window parameter. This function will plot the candlesticks and use the collect_channel function to detect and fit the extrema slopes. These slopes are then plotted as lines on the chart within the backcandles slice.

By calling this function for each candle and providing the candle's index along with the other parameters, any detected channels will be plotted on the chart. If no channel is detected, the chart will simply display without the channel lines. The function is defined in the following example:

```
def plot_candlestick(candle, backcandles, window):
    """
    Plots a candlestick chart with additional lines
    based on the given candle, backcandles, and window parameters.

    Parameters:
    candle (int): The index of the candle to plot.
    backcandles (int): The number of candles to look back for calculations.
    window (int): The window size for rolling calculations.
    """

    fig = go.Figure(data=[go.Candlestick(x=dfpl.index,
                    open=dfpl['Open'],
                    high=dfpl['High'],
                    low=dfpl['Low'],
                    close=dfpl['Close'])])
```

```
    fig.add_scatter(x=dfpl.index, y=dfpl['pointpos'],
mode="markers",
                    marker=dict(size=5, color="MediumPurple"),
                    name="pivot")

    sl_lows, interc_lows, sl_highs, interc_highs, r_sq_l, r_sq_h =
collect_channel(candle, backcandles, window)
    print("R-squared values for low and high slopes are displayed
above.", r_sq_l, r_sq_h)
    x = np.array(range(candle-backcandles-window, candle+1))
    fig.add_trace(go.Scatter(x=x, y=sl_lows*x + interc_lows,
mode='lines', name='lower slope'))
    fig.add_trace(go.Scatter(x=x, y=sl_highs*x + interc_highs,
mode='lines', name='max slope'))
    #fig.update_layout(xaxis_rangeslider_visible=False)
    fig.show()

plot_candlestick(76, 40, 3)
```

I've added a print statement within the function that outputs the R-squared values for both the lows and highs. For the data I used, I got values of 0.99 and 0.94, indicating that both fits are quite strong. If we compare the low and high points with their respective slopes in the figure mentioned earlier, we can see that the lows are slightly closer to their slope than the highs. Notably, there's a particular high point, the second one, that appears to be slightly farther from its slope.

One of the challenges with this approach to detecting channels is finding the optimal values for the window and backcandles, as well as determining the minimum number of points or fractals to consider when applying the fitting. Other parameters, like the goodness of fit, are less problematic, ideally, you'd want an R-squared value above 0.9, as anything below this suggests a poor fit.

For parameters that might require some experimentation, like the backcandles slice, you can call the function with different

values. For each candle, you could run the plot_candlestick function with three different backcandles values, such as 20, 40, and 60, to examine short-range, medium-range, and long-range channels. Based on the R-squared values and the quality of the detection, you can then select the most appropriate channel to either detect a breakout or to trade within.

5.2 Triangle Patterns

If the previous paragraph is well clear and understood the rest of this section is now straightforward and very simple because we're going to use exactly the same approach to detect different patterns in prices. This paragraph detects the triangle pattern. A triangle pattern is a continuation or reversal pattern that occurs during a period of price consolidation, where the range of price movement narrows over time, forming a triangular shape.

- **Ascending Triangle:** Characterized by a horizontal resistance line and a rising support line. It typically indicates a bullish continuation, where a breakout above the resistance line suggests the resumption of the upward trend.
- **Descending Triangle:** Defined by a horizontal support line and a descending resistance line. It usually signals a bearish continuation, where a breakdown below the support line indicates the continuation of the downward trend.
- **Symmetrical Triangle:** Formed by a converging trendline of lower highs and higher lows, indicating a period of consolidation where the direction of the breakout can signal either a continuation or reversal, depending on the prior trend and the breakout direction.

The algorithm for detecting triangle patterns begins by

identifying pivot candles, or fractals, and then fitting the highs and lows with slopes, similar to the approach described in the previous section. To detect a triangle pattern, we require the highs to have a negative slope and the lows to have a positive slope. Given that the shape of a triangle can vary, we set specific thresholds: the slope of the highs must be below a certain threshold, and the increasing slope of the lows must be above the absolute value of that same threshold.

To simplify this process, we can express the slopes as percentages of price change rather than using their absolute values. This can be easily achieved by adding two lines of code which will convert the slopes into percentages of price change.

```
sl_lows_pct = (sl_lows / start_price_low) * 100
sl_highs_pct = (sl_highs / start_price_high) * 100
```

Our code is getting larger so let's redefine the functions. We will discuss three essential functions that work together to detect and plot triangle patterns within candlestick data. Each function plays a specific role in the overall process, from calculating slopes and R-squared values to plotting the detected pattern on a chart.

```
calculate_slopes(df, candle, backcandles, window)
```

This function is the backbone of our analysis, designed to calculate the slopes and R-squared values for a given candle. It takes in the DataFrame containing candlestick data, the index of the candle under scrutiny, the number of candles to look back, and the window size for rolling calculations. By evaluating the high and low prices, it returns the percentage slopes of the highs and lows along with their respective R-squared values. These metrics are essential for identifying whether a triangle pattern is forming.

```
plot_triangle(df, candle, backcandles, window, slope_threshold,
r2_threshold=0.9)
```

Once the slopes and R-squared values are calculated, this function steps in to visualize the detected triangle pattern. It checks if the slopes meet the defined thresholds for a valid triangle pattern, based on the provided slope_threshold and r2_threshold. If a triangle is detected, the function identifies the first minima and maxima within the relevant data range and plots only the section of the chart where the pattern occurs. This targeted plotting ensures a clear and focused view of the triangle pattern, making it easier to analyze and interpret.

```
find_first_triangle(df, backcandles, window, slope_threshold=0.01, r2_threshold=0.9, start_index=None)
```

The final function in our sequence is designed to loop through the DataFrame and find the first occurrence of a triangle pattern. Starting from a user-defined index (or a default starting point), it applies the calculate_slopes function to each candle. When a triangle pattern that meets the specified thresholds is detected, it triggers the plot_triangle function to visualize the pattern. The search halts as soon as the first triangle is found and plotted, providing a quick and efficient way to identify significant patterns in the data. If no triangle is detected, the function simply informs the user.

Together, these functions provide our toolkit for detecting and visualizing triangle patterns in financial data. The Python functions are defined as follows:

```
import numpy as np
from tqdm import tqdm

def calculate_slopes(df, candle, backcandles, window):
    """
    Calculate slopes and R-squared values for a given candle.

    Parameters:
    df (pd.DataFrame): The dataframe containing candle data.
    candle (int): The index of the candle to calculate for.
```

```
    backcandles (int): The number of candles to look back for
calculations.
    window (int): The window size for rolling calculations.

    Returns:
    tuple: sl_lows_pct, sl_highs_pct, r_sq_l, r_sq_h
    """
    sl_lows, interc_lows, sl_highs, interc_highs, r_sq_l, r_sq_h =
collect_channel(candle, backcandles, window)

    start_price_low = df['Low'].iloc[candle - backcandles - window]
    start_price_high = df['High'].iloc[candle - backcandles - window]

    sl_lows_pct = (sl_lows / start_price_low) * 100
    sl_highs_pct = (sl_highs / start_price_high) * 100

    return (sl_lows_pct, sl_highs_pct,
            r_sq_l, r_sq_h,
            sl_lows, interc_lows,
            sl_highs, interc_highs)

def plot_triangle(df, candle, backcandles,
                  window, slope_threshold, r2_threshold=0.9):
    """
    Plots a candlestick chart showing only the area where the
triangle pattern is detected.

    Parameters:
    df (pd.DataFrame): The dataframe containing candle data.
    candle (int): The index of the candle where the triangle is
detected.
    backcandles (int): The number of candles to look back for
calculations.
    window (int): The window size for rolling calculations.
    slope_threshold (float): The threshold for detecting triangle
patterns (in percentage).
    r2_threshold (float): The minimum R-squared value required for
a valid fit (default is 0.9).
```

```
    """
    sl_lows_pct, sl_highs_pct, r_sq_l, r_sq_h, sl_lows, interc_lows,
sl_highs, interc_highs = calculate_slopes(
        df, candle, backcandles, window
    )

    if (sl_highs_pct < -slope_threshold and
        sl_lows_pct > slope_threshold and
        r_sq_l >= r2_threshold and
        r_sq_h >= r2_threshold):

        print(f"Triangle pattern detected at candle index {candle}.")

        # Find the first minima and maxima indices
        local_df = df.iloc[max(0, candle - backcandles -
window):candle + 1]
        first_min_idx = local_df[local_df['Low'] ==
local_df['Low'].min()].index[0]
        first_max_idx = local_df[local_df['High'] ==
local_df['High'].max()].index[0]
        start_idx = min(first_min_idx, first_max_idx)

        # Define the range to plot
        plot_start_idx = max(0, start_idx - 50)
        plot_end_idx = min(len(df), candle + 1)

        # Slice the DataFrame to only include the relevant range
        df_slice = df.iloc[plot_start_idx:plot_end_idx]

        fig = go.Figure(data=[go.Candlestick(x=df_slice.index,
                                             open=df_slice['Open'],
                                             high=df_slice['High'],
                                             low=df_slice['Low'],
                                             close=df_slice['Close'])])

        fig.add_scatter(x=df_slice.index, y=df_slice['pointpos'],
mode="markers",
                        marker=dict(size=5, color="MediumPurple"),
```

```
                    name="pivot")

        # Limit the x values for the lines to the detected triangle
range
        x = np.array(range(start_idx, candle + 1))
        fig.add_trace(go.Scatter(x=x, y=sl_lows * x + interc_lows,
                                 mode='lines',
                                 name='lower slope (triangle)'))
        fig.add_trace(go.Scatter(x=x, y=sl_highs * x + interc_highs,
                                 mode='lines',
                                 name='upper slope (triangle)'))

        fig.show()
    else:
        print("No triangle pattern detected.")

def find_first_triangle(df, backcandles, window,
slope_threshold=0.01, r2_threshold=0.9, start_index=None):
    """
    Loops through the DataFrame to find and plot the first triangle
pattern.

    Parameters:
    df (pd.DataFrame): The DataFrame containing candlestick data.
    backcandles (int): The number of candles to look back for
calculations.
    window (int): The window size for rolling calculations.
    slope_threshold (float): The threshold for detecting triangle
patterns (in percentage).
    r2_threshold (float): The minimum R-squared value required for
a valid fit (default is 0.9).
    start_index (int, optional): The index in the DataFrame where the
search starts.
                                 If None, it starts at `backcandles +
window`.
    """
    if start_index is None:
```

```
        start_index = backcandles + window

    start_index = max(start_index, backcandles + window)

    for candle in tqdm(range(start_index, len(df))):
        sl_lows_pct, sl_highs_pct, r_sq_l, r_sq_h, *_ =
calculate_slopes(df, candle, backcandles, window)

        if (sl_highs_pct < -slope_threshold and
            sl_lows_pct > slope_threshold and
            r_sq_l >= r2_threshold and
            r_sq_h >= r2_threshold):
            plot_triangle(df, candle, backcandles, window,
slope_threshold, r2_threshold)
            break
    else:
        print("No triangle pattern detected in the dataset.")
```

And the results on a subset of data that I've been trying the functions on is shown in the figures below:





Although the first example might seem acceptable, the issue becomes more apparent in the second figure, where we observe two highs followed by two lows. These points do not alternate as required for a valid triangular pattern. For a proper pattern, we need a sequence where a high is followed by a low, then another high followed by another low. In this case, the lack of alternation is causing the detection of a false triangle pattern.

To address this problem, we can modify the calculate_slopes function to filter out cases where the pivots do not alternate between highs and lows. Additionally, we need to update the find_first_triangle function to skip cases where calculate_slopes returns None, indicating that the pivots did not alternate.

To avoid making the algorithm too selective and missing out on potentially valid patterns, we'll modify calculate_slopes to account for scenarios where there is at least one alternation, such as a high-low-high or low-high-low pattern among the pivot points. We can also consider a more flexible approach where we only filter out cases where all the high pivots are grouped together and all the low pivots are grouped together (e.g., two highs followed by two lows, or vice versa). This condition ensures some alternation in the pivots without strictly requiring every point to alternate.

The updated functions are as follows:

```
def calculate_slopes(df, candle, backcandles, window):
```

```
    """
    Calculate slopes and R-squared values for a given candle,
    ensuring that high and low pivots are not grouped
    together (e.g., avoiding sequences like low low high high).

    Parameters:
    df (pd.DataFrame): The dataframe containing candle data.
    candle (int): The index of the candle to calculate for.
    backcandles (int): The number of candles to look back for
calculations.
    window (int): The window size for rolling calculations.

    Returns:
    tuple: sl_lows_pct, sl_highs_pct,
    r_sq_l, r_sq_h, sl_lows, interc_lows, sl_highs, interc_highs
    """
    sl_lows, interc_lows, sl_highs, interc_highs, r_sq_l, r_sq_h =
collect_channel(candle, backcandles, window)

    start_price_low = df['Low'].iloc[candle - backcandles - window]
    start_price_high = df['High'].iloc[candle - backcandles -
window]

    sl_lows_pct = (sl_lows / start_price_low) * 100
    sl_highs_pct = (sl_highs / start_price_high) * 100

    # Access the isPivot column to check the sequence of pivots
    local_df = df.iloc[candle - backcandles - window:candle-window]
    pivots = [p for p in local_df['isPivot'].values if p not in [0, 3]]

    # Ensure no grouping of all highs or all lows
    if len(pivots) >= 3:
        if (all(p == 2 for p in pivots[:len(pivots)//2]) and
            all(p == 1 for p in pivots[len(pivots)//2:])):
            return None, None, None, None, None, None, None, None
        if (all(p == 1 for p in pivots[:len(pivots)//2]) and
            all(p == 2 for p in pivots[len(pivots)//2:])):
            return None, None, None, None, None, None, None, None
```

```
    return (sl_lows_pct, sl_highs_pct,
            r_sq_l, r_sq_h,
            sl_lows, interc_lows,
            sl_highs, interc_highs)
```

```
def find_first_triangle(df, backcandles, window,
                        slope_threshold=0.01, r2_threshold=0.9,
start_index=None):
    """
    Loops through the DataFrame to find and plot
    the first triangle pattern, ensuring alternating high/low pivots.

    Parameters:
    df (pd.DataFrame): The DataFrame containing candlestick data.
    backcandles (int): The number of candles to look back for
calculations.
    window (int): The window size for rolling calculations.
    slope_threshold (float): The threshold for detecting triangle
patterns (in percentage).
    r2_threshold (float): The minimum R-squared value required for
a valid fit (default is 0.9).
    start_index (int, optional): The index in the DataFrame where
the search starts.
                                 If None, it starts at `backcandles +
window`.
    """
    if start_index is None:
        start_index = backcandles + window

    start_index = max(start_index, backcandles + window)

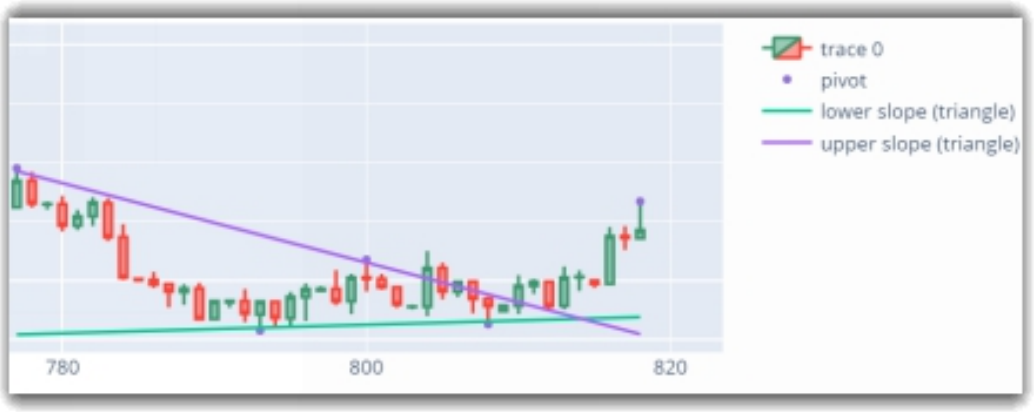
    for candle in tqdm(range(start_index, len(df))):
        results = calculate_slopes(df, candle, backcandles, window)

        if results[0] is None:  # if the calculation was skipped due to
non-alternating pivots
            continue
```

```
        sl_lows_pct, sl_highs_pct, r_sq_l, r_sq_h, *_ = results

        if (sl_highs_pct < -slope_threshold and
            sl_lows_pct > slope_threshold and
            r_sq_l >= r2_threshold and
            r_sq_h >= r2_threshold):
            plot_triangle(df, candle, backcandles,
                          window, slope_threshold, r2_threshold)
            break
    else:
        print("No triangle pattern detected in the dataset.")
```

And the detected patterns are now the following:







The results have shown some improvement, and there are many ways to further enhance fractal detection and refine this system. The examples presented here serve as a solid starting point, demonstrating how you can combine fractals and regression to identify patterns relevant to your trading strategy. One key aspect to consider is the sensitivity of the algorithm. For instance, we used a selective set of parameters with an R^2 threshold set at 0.9 and a back-candles window of 40. However, you could lower the R^2 threshold and increase the number of back-candles to allow for more fractals to form, making the system less selective about the goodness of fit. These are just a few ideas to consider when building your own system.

It's also worth noting that we're currently detecting all types of triangles, regardless of their shape—whether they are symmetrical, ascending, or descending. This is because we didn't impose any specific conditions related to the slopes of the high and low fractals. However, this can easily be adjusted. For example, if you want to detect only ascending triangles, you could require that the slope of the high fractals is nearly horizontal (close to 0), while the slope of the low fractals is strictly positive. For descending triangles, you'd look for a negative slope in the high fractals and a near-horizontal slope for the lows.

To detect divergent triangles that start from a tight formation and expand in volatility, you could add conditions where the slope of the highs is positive, and the slope of the lows is

negative. For parallel price channels, you would want the slopes of the high and low fractals to be nearly equal.

These are just a few ways you can customize this approach to create your own technical indicators based on the specific patterns you want to detect.

6 Candles Patterns Detection

In this section, we'll explore candlestick patterns, focusing particularly on the engulfing and rejection patterns. These two patterns are selected for their distinct characteristics, providing a foundational understanding of pattern detection. We will go through the coding techniques used to identify these patterns. You can use the methods presented here to develop detection algorithms for any pattern you wish to incorporate into your trading system. We will go through examples that illustrate the simplicity and adaptability of our approach, showing how you can apply and extend these concepts to recognize related patterns in your own trading strategies.

Candlestick patterns are used in technical analysis to predict future market movements based on past price action. Let’s take a detailed look at the engulfing pattern and the rejection pattern, including why they occur and the underlying market actions they reflect:

An engulfing pattern is a two-candle reversal pattern that signals a potential shift in market direction. It consists of two opposite-colored bodies, where the second candle's body completely engulfs the first candle's body.

- **Bullish Engulfing**: Occurs in a downtrend. The first candle is bearish (red), followed by a larger bullish (green) candle. It suggests buyers have overtaken sellers, potentially leading to a reversal upward.

- **Bearish Engulfing**: Occurs in an uptrend. The first candle is bullish (green), followed by a larger bearish (red) candle. This indicates sellers have overcome buyers, potentially leading to a reversal downward.

Engulfing patterns occur due to a sudden and significant shift in market sentiment. In the case of a bullish engulfing, the buying pressure overwhelms the prior selling pressure, suggesting a strong interest in pushing the price up. Conversely, a bearish engulfing shows that selling pressure has overtaken buying interest, pushing prices lower.

The rejection pattern, often referred to as a "Pin Bar" (Pinocchio Bar), is a single candlestick pattern that indicates a rejection of a price level and a potential reversal. It features a small body at one end of the candle and a long wick (or shadow) which is typically at least three times the length of the body.

- **Bullish Rejection**: The long wick is below the body, showing that while prices were pushed down, buyers entered the market aggressively and pushed prices back up, rejecting lower levels.
- **Bearish Rejection**: The long wick is above the body, indicating that prices were pushed up, but sellers came in strongly and pulled the prices back down, rejecting higher levels.

Rejection patterns form when prices are pushed to a certain level that is either not justified by market conditions or is at a significant support or resistance point. The long wick represents an initial push in one direction, while the small body shows that the initial push was not sustained and the opposite force prevailed. This pattern reflects the testing of market sentiment at a particular price level, often leading to strong reversals when combined with high volume.

Both engulfing and rejection patterns reflect underlying actions such as:

- **Shifts in Supply and Demand**: Sudden changes in volume and open interest can indicate shifts in market dynamics.
- **Psychological Levels**: Support and resistance levels often play a crucial role in the formation of these patterns.
- **Market News and Events**: Economic announcements or geopolitical events can dramatically alter trader sentiment and market direction.

These patterns are effective across various timeframes, but the price reaction tends to be more sustained on larger timeframes, such as daily or weekly charts. Additionally, candle patterns become a strong indicator of price reversal when they occur near key levels. We can keep these ideas in mind when developing strategies relying on candles indicators.

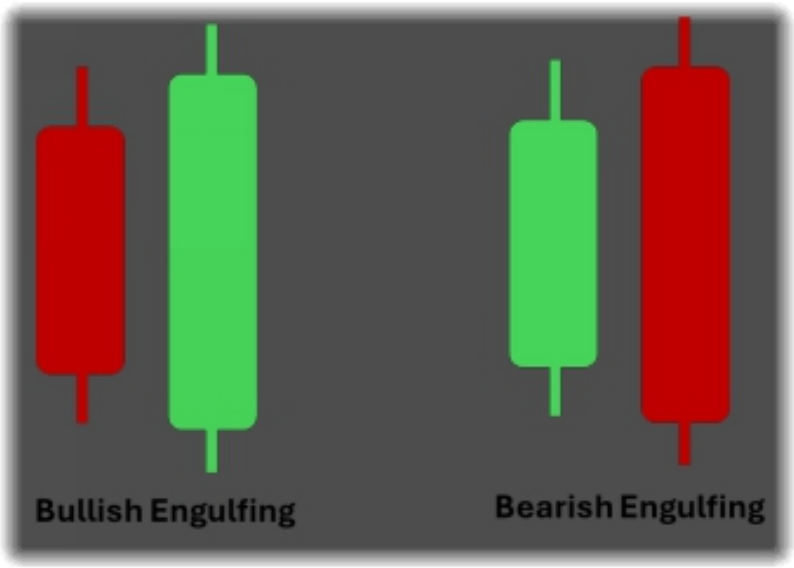
6.1 Coding The Engulfing Pattern

The basic structure of the engulfing pattern involves identifying a candle with a body wider than the previous candle's body, while the two candles are of opposite colors. The figure below illustrates the typical bullish and bearish engulfing patterns.

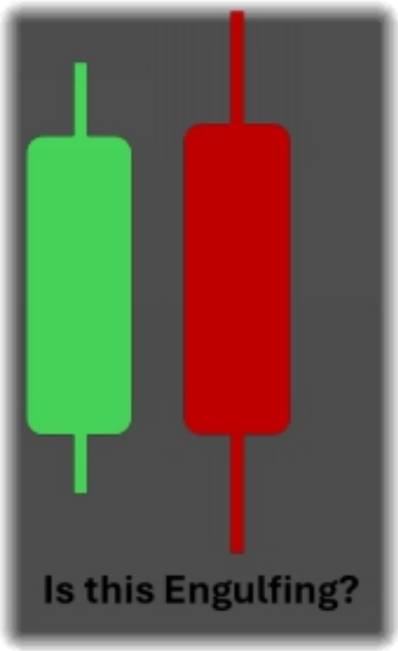
Coding this pattern is straightforward. To detect a bullish engulfing pattern, you simply need to check if the current candle's opening price is below the previous candle's closing price and if the current candle's closing price is higher than the previous candle's opening price.

For the bearish engulfing pattern, the conditions are symmetrical: check if the current candle's closing price is below the previous candle's opening price, and if the current candle's

opening price is above the previous candle's closing price. Translating these conditions into Python or any other programming language involves just four simple checks, giving you a basic engulfing pattern detector.



In algorithmic trading, there's a subtlety that often gets overlooked by traders. As humans, we typically assess the engulfing pattern visually, relying on our eyes to detect whether one candle engulfs another. However, to visually perceive this engulfing, there needs to be a noticeable difference between the opening and closing levels of the current and previous candles. What I'm trying to emphasize is that if the current candle engulfs the previous one by just a few pips, an amount too small to be easily seen by the trader, this pattern might go undetected because the candles will appear almost equal. To illustrate this point, take a look at the following figure.

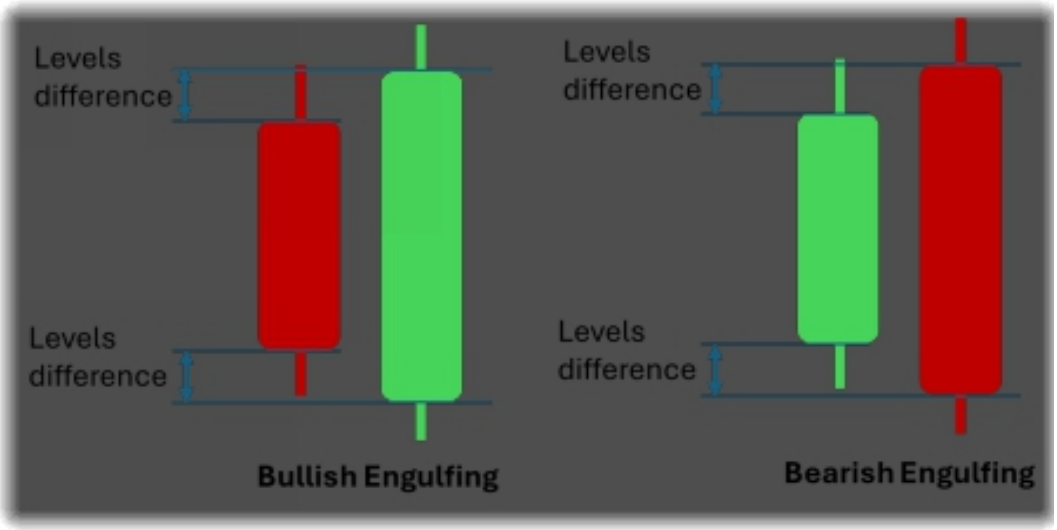


The red candle closely resembles the green candle, with only a slight difference at the top end. Now, the question is whether this qualifies as an engulfing candle. You might typically look for additional confirmation from other indicators, but in the context of algorithmic trading, even a small difference of one pip could trigger a valid engulfing candle signal. If the signal isn't strong enough, it can be filtered out, and to achieve this, I'll introduce a new variation of the engulfing pattern.

In this variation, we'll not only check if the current candle engulfs the previous one, but we'll also ensure that the difference between the levels of the current and previous candles exceeds a certain threshold. This threshold will be set by the user as a percentage of the price.

In the following figure, you can observe the level differences at the top and bottom ends of the candles. These differences must surpass the specified threshold. Any engulfing candle that doesn't meet this criterion will be discarded, and the signal will be invalidated. This approach helps filter out ambiguous signals that don't clearly indicate strong market sentiment. Remember, this pattern is all about understanding the underlying market

psychology, whether buyers or sellers are exerting greater control at that moment. If the market is showing hesitation, we want to avoid taking risks when the direction or state of the market is uncertain.



In the following example, we present a Python function that determines whether a candle is part of a bearish or bullish engulfing pattern. The function returns 1 for bearish candles, 2 for bullish candles, and 0 if no pattern is detected. It also applies an engulfing difference percentage threshold, along with all the necessary conditions for accurate pattern identification:

```
def is_engulfing(df, candleidx, engulfdiff_pct=0.01):
    """
    Determine if a given candle is part of a bullish or bearish engulfing  pattern.

    Parameters:
    candleidx (int): The index of the current candle in the DataFrame.
    df (pandas.DataFrame): The DataFrame containing the OHLC (Open, High, Low, Close) data.
    engulfdiff_pct (float): The minimum percentage difference
                            used to confirm an engulfing pattern.
```

```
    Returns:
    int: Returns 2 if a bullish engulfing pattern is detected,
                1 if a bearish engulfing pattern is detected,
                and 0 if neither pattern is found.
    """
    # Ensure we're not at the start of the DataFrame
    if candleidx < 1:
        return 0

    # Calculate the difference thresholds based on the
    # percentage of the previous candle's closing price
    engulfdiff = df['Close'][candleidx - 1] * engulfdiff_pct/100

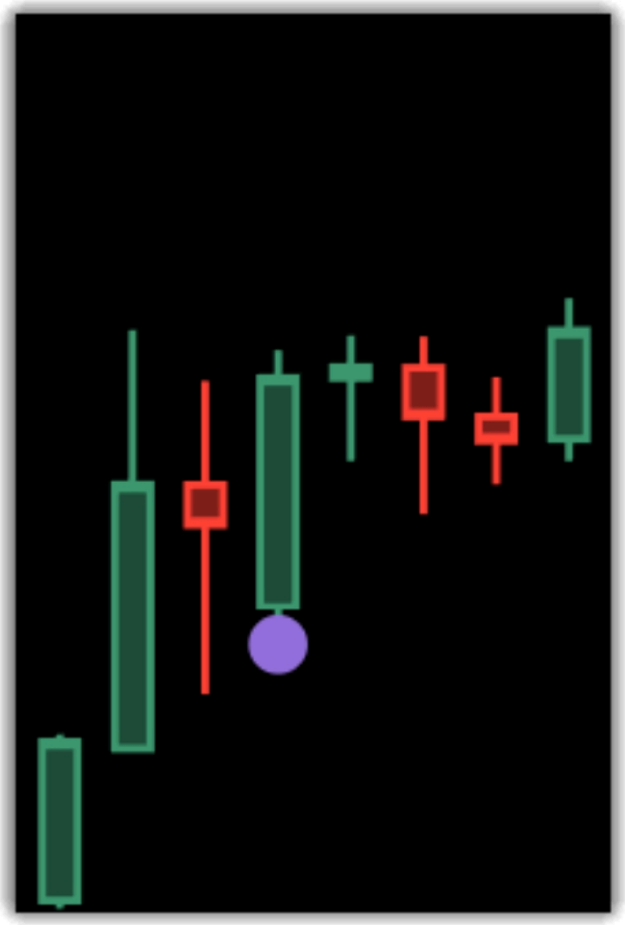
    # Check for a bullish engulfing pattern
    if (df['Close'][candleidx - 1] < df['Open'][candleidx - 1] and
        df['Close'][candleidx] > df['Open'][candleidx] and
        df['Close'][candleidx - 1] - df['Open'][candleidx] > engulfdiff
and
        df['Open'][candleidx - 1] - df['Close'][candleidx] < -
engulfdiff):
        return 2

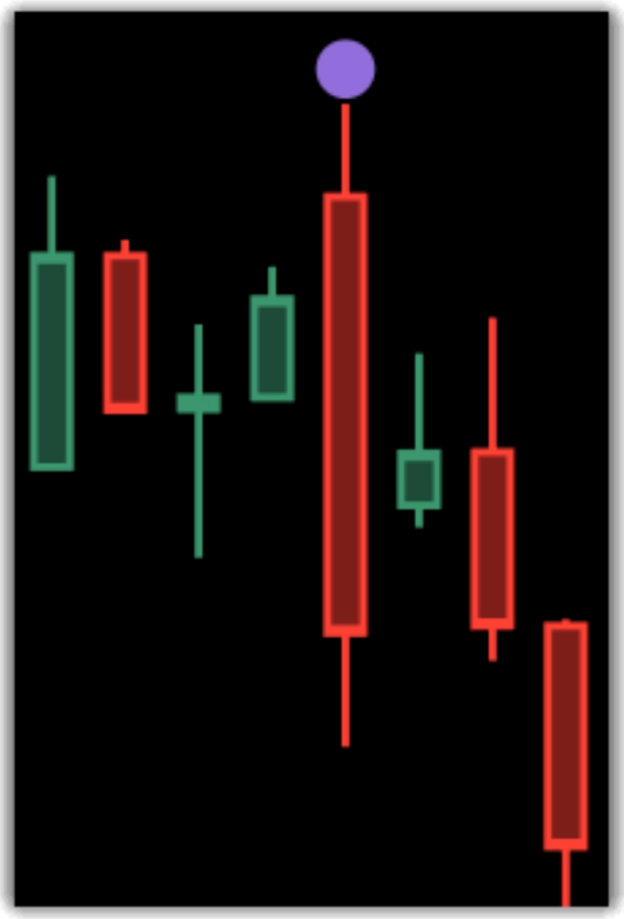
    # Check for a bearish engulfing pattern
    if (df['Close'][candleidx - 1] > df['Open'][candleidx - 1] and
        df['Close'][candleidx] < df['Open'][candleidx] and
        df['Open'][candleidx] - df['Close'][candleidx - 1] > engulfdiff
and
        df['Close'][candleidx] - df['Open'][candleidx - 1] < -
engulfdiff):
        return 1

    # If no engulfing pattern is found
    return 0
```

Applying this function to a DataFrame will identify bullish and bearish engulfing candles. In the sample data shown in the figure below, a purple signal point below a candle indicates a

bullish engulfing pattern, while a purple signal point above a candle indicates a bearish engulfing pattern. The visualization is primarily used to verify that the algorithm is functioning correctly and accurately identifying the intended patterns in our data.

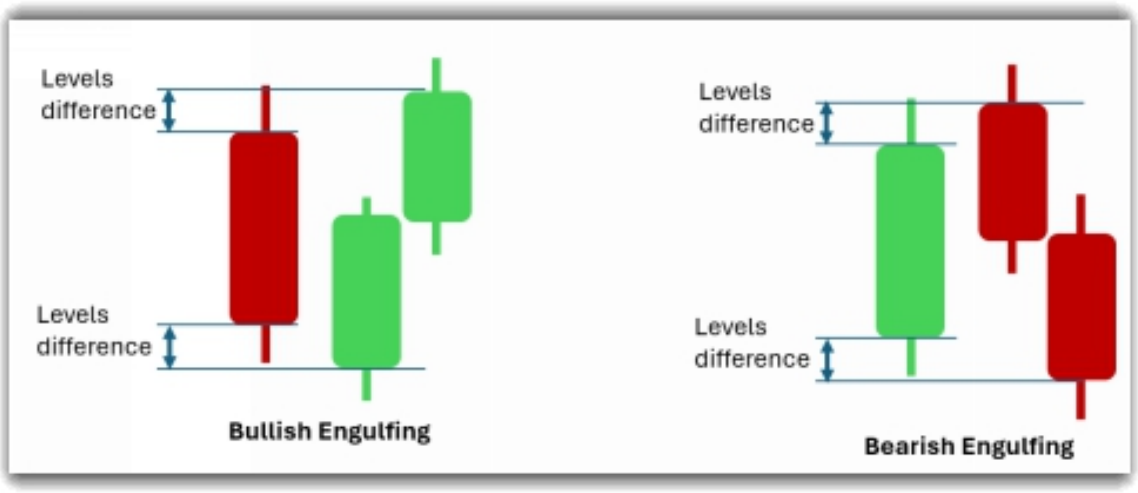




6.2 Engulfing Variation

In some cases, the engulfment doesn't occur with just one candle. Instead, two consecutive candles moving in the opposite direction can collectively engulf the first candle. We can code this variation in Python, but we need to consider the number of candles allowed for engulfment and the percentage difference in closing prices, as discussed earlier.

The figure below illustrates this variation. In the first example, two green candles engulf the initial red candle, and in the second example, two red candles engulf the initial green candle. While it's possible to extend this concept to allow engulfment over three or four subsequent candles, doing so may lose its relevance from a trading perspective.



Here's an example of our Python function, which uses the same set of parameters as before: the DataFrame, candle index, and the threshold for the level difference in percentage of the last closing price. Additionally, we've introduced a new parameter that specifies the number of candles allowed to complete the engulfing pattern.

```
def is_engulfing(df, l, engulfdiff_pct=0.0001,
engulfing_candles=1):
    """
    Determine if a given candle is an engulfing candle pattern.

    Parameters:
    df (DataFrame): DataFrame containing Open and Close prices.
    l (int): Current position in the DataFrame to evaluate.
    engulfdiff_pct (float, optional):   Percentage difference to
consider
                                        for engulfing. Default is 0.0001.
    engulfing_candles (int, optional): Number of candles to check
for
                                       the engulfing pattern. Default is 1.

    Returns:
    int: 0 if not engulfing, 1 if bearish engulfing, 2 if bullish
engulfing.
```

```
    """
    if l < engulfing_candles:
        return 0

    # Check if all recent engulfing_candles are of same color
    # and the candle just before is an opposite color
    if (not all((df.Open[l-i] > df.Close[l-i]) == (df.Open[l] >
df.Close[l])
                for i in range(engulfing_candles)) or
        not ( (df.Open[l-engulfing_candles] < df.Close[l-
engulfing_candles])
            == (df.Open[l] > df.Close[l]) ) ):
        return 0

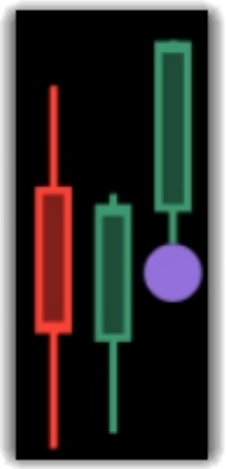
    engulfdiff = df.Close[l] * engulfdiff_pct

    # Check if last candle is green then check if its closing price is
greater
    # than the open price of l-engulfing_candles by at least
engulfdiff
    # and apply symmetrical configuration for bearish engulfing
detection
    if (df.Close[l] > df.Open[l] and df.Close[l] >=
        df.Open[l-engulfing_candles] + engulfdiff and
        df.Close[l-engulfing_candles] - df.Open[l-
engulfing_candles+1] >
        engulfdiff):
        return 2

    if (df.Close[l] < df.Open[l] and df.Close[l] <=
        df.Open[l-engulfing_candles] - engulfdiff and
        df.Close[l-engulfing_candles] - df.Open[l-
engulfing_candles+1]
        < -engulfdiff):
        return 1

    return 0
```

I applied the function shown in this example to a subset of data to visualize some detections, specifically where two candles were needed to engulf a previous candle. The figure below shows an example of such a detection. We can see that the first green candle did not have enough power to overcome and close above the previous red candle; however, with the second green candle, this gap was closed, and the engulfing pattern was completed.



6.3 Coding The Rejection Pattern

The rejection candle, also known as a pin bar, has a distinctive structure that typically features a long wick extending from a relatively small body, indicating a significant rejection of certain price levels. This candle can appear in both bullish and bearish forms depending on the direction of the price rejection relative to the trend. The figure below illustrates typical bullish and bearish rejection candles.

Coding this pattern is straightforward as well. To detect a bullish rejection candle, you need to check if the lower wick is significantly longer than the body and if the body closes near the top of the candle's range. This indicates that despite dropping to lower prices during the session, buying pressure pushed the price back up, rejecting lower levels.

For the bearish rejection candle, the criteria are inverted: the upper wick should be significantly longer than the body, and the body should close near the bottom of the candle's range. This suggests that selling pressure overcame buying pressure after prices peaked during the session, rejecting higher levels. Implementing these checks in Python or any other programming language typically requires evaluating the proportions of the candle's wicks relative to its body, resulting in a simple yet effective rejection candle detector.

The following function detects rejections pattern candles is returns 2 for a bullish rejection and 1 for a bearish rejection.

```
def is_rejection(df, current_candle, body_perc = 0.01, wick_ratio=3):
    """
    Determine if a given candle is a rejection candle based on body percentages and wick ratios.

    Parameters:
    df (pandas.DataFrame): DataFrame containing the candlestick data.
    current_candle (str or int): Identifier for the current candle (index or label).
    body_perc (float, optional): Threshold for the minimum candle body percentage (default is 0.1).
    wick_ratio (float, optional): Arbitrary ratio value for wick comparison (default is 3).

    Returns:
    int: 2 if the candle is a bullish rejection, 1 if bearish rejection, 0 otherwise.
    """
    current_pos = df.index.get_loc(current_candle)
    current_price = df['Close'].iloc[current_pos]

    body_length_percentage = ( abs(df['Open'].iloc[current_pos] - df['Close'].iloc[current_pos]) / current_price * 100)
```

```
    lower_wick_percentage = ( (min(df['Open'].iloc[current_pos],
df['Close'].iloc[current_pos])
                    -df['Low'].iloc[current_pos]) / current_price * 100 )
    upper_wick_percentage = ( (df['High'].iloc[current_pos] -
                            max(df['Open'].iloc[current_pos],
df['Close'].iloc[current_pos])) / current_price * 100 )

    # Buy conditions
    wick_to_body_ratio = lower_wick_percentage /
body_length_percentage
    c1 = body_length_percentage > body_perc  # Minimum
threshold for body
                                            length in percentage
    c2 = lower_wick_percentage >
upper_wick_percentage*wick_ratio
    c3 = wick_to_body_ratio > wick_ratio  # Arbitrary ratio value

    if c1 and c2 and c3:
        return 2

    # sell conditions
    wick_to_body_ratio = upper_wick_percentage /
body_length_percentage
    c1 = body_length_percentage > body_perc # Minimum
threshold for
                                            body length in percentage
    c2 = upper_wick_percentage >
lower_wick_percentage*wick_ratio
    c3 = wick_to_body_ratio > wick_ratio  # Arbitrary ratio value

    if c1 and c2 and c3:
        return 1

    return 0
```

The function accepts a DataFrame and an index corresponding to the current candle, determining whether it forms a rejection pattern. Two additional parameters, body

percentage and wick ratio, refine the detection criteria. All measurements within the function are expressed as percentages of the current asset price, including the body length, lower wick, and upper wick. This approach simplifies calculations and standardizes them across different assets.

The body percentage sets a minimum threshold for the candle's body as a percentage of the price, filtering out candles with excessively thin bodies. This ensures that only candles with sufficiently substantial bodies are considered, as thinner candles can too easily meet the other criteria.

The wick ratio defines the relationship between the length of the candle's longer wick and its body. By default, a wick ratio of three indicates that to qualify as a rejection candle, the longer wick must be at least three times the length of the body. Additionally, the shorter wick must be less than one-third the length of the longer wick (or whatever ratio value we use for the input parameter).

This method effectively identifies rejection candles, categorizing them as bullish or bearish based on their characteristics and returning appropriate indicators: 2 for bullish, 1 for bearish, or 0 if neither condition is met.

The results are displayed on the chart below with the purple points marking the bullish rejections if the point is below the candle and the bearish rejections if the point is above the candle.



7 Conclusion

In this chapter, we explored various ways to customize indicators, overlay them, and develop new measures to enhance our trading strategies. We covered a broad spectrum of techniques, from combining multiple indicators to creating entirely new ones, all with the goal of improving our decision-making in the markets. The flexibility and creativity that algorithmic trading allows are immense, and as we've seen, these ideas can be implemented and tested using relatively simple tools, such as a laptop. This is one of the great advantages of learning algorithmic trading: any indicator or strategy you can conceive can be coded, backtested, and refined to determine its potential effectiveness.

While we have covered a significant amount of ground, there are still many avenues you could explore further. For instance, you might want to delve into price action patterns like identifying sequences of higher highs and higher lows, followed by a low that breaks below a previous low, signaling a potential reversal. This type of analysis can be powerful when combined with other indicators to confirm signals or to provide additional context for

market moves.

Another intriguing area to investigate is the use of Fibonacci retracement levels to trigger entry or exit positions. Fibonacci levels are widely used by traders to predict potential reversal points, and integrating them into your algorithmic strategies could add a valuable layer of precision. You could simulate how the market reacts at these levels and develop custom rules for entering or exiting trades based on how price interacts with the retracement zones.

Moreover, you could expand your exploration of candlestick patterns. While basic patterns involving two or three candles are commonly used, there is a vast array of more complex patterns that can provide deeper insights into market sentiment. Developing functions that detect these patterns and integrating them into your trading strategy could lead to more nuanced and effective decision-making.

In addition to these ideas, consider experimenting with custom indicators that combine elements of both technical analysis and fundamental analysis. For example, you could create an indicator that adjusts its signals based on key economic data releases or company earnings reports, thereby providing a more holistic view of the market.

The key takeaway from this chapter is that the possibilities are virtually limitless. Whether you're refining existing strategies or creating new ones from scratch, the ability to test and iterate quickly is a powerful advantage in algorithmic trading. As you continue to develop your skills, remember that the market rewards creativity and innovation. The more you experiment and push the boundaries of what's possible with your indicators and strategies, the more likely you are to find a unique edge in the markets. So, keep exploring, keep testing, and most importantly, keep learning. The tools and techniques you've learned in this chapter are just the beginning of your journey into the world of algorithmic trading.

Chapter 6: Support Resistance And Fibonacci Levels

1 Introduction

1.1 Key Levels

In trading, understanding and identifying key levels is important as it opens further technical options to improve trading systems. These levels, often referred to as support and resistance, play a pivotal role in determining the behavior of price action. Support levels are areas where the price tends to find a floor, preventing it from falling further, while resistance levels act as a ceiling, stopping or slowing upward momentum. These levels are not just arbitrary; they represent points where market participants have historically shown a strong willingness to buy or sell, making them vital markers on a trader's chart.

These levels can be viewed as a self-fulfilling prophecy. When many traders recognize a price consistently reacting at a certain level, they tend to take action at that point, whether by placing buy or sell orders, setting stop losses, or determining take-profit levels. This collective behavior influences the market, often leading to predictable patterns. As a result, at these key levels, the price may exhibit distinct behaviors, such as bouncing sharply or breaking through and moving swiftly to the next significant level.

The strength of these key levels can vary depending on the timeframe in which they are observed. Higher timeframes, such as daily, weekly, or monthly charts, reveal more significant levels of support and resistance compared to lower timeframes like the

5-minute or 15-minute charts. The reason is simple: higher timeframes encompass more data and reflect the decisions of a broader range of market participants, including institutional investors. As a result, key levels identified on higher timeframes are generally considered more reliable and influential, often dictating the overall trend direction and providing stronger areas of confluence for trading decisions.

Fibonacci levels are another important tool that traders use to identify key levels in the market. Derived from the Fibonacci sequence, these levels are based on mathematical ratios, such as 38.2%, 50%, and 61.8%, which are believed to represent natural retracement points in a market move. When applied to price charts, Fibonacci levels help traders identify potential reversal zones, continuation patterns, and areas where the price might encounter significant support or resistance.

1.2 Fibonacci Levels

Fibonacci levels are particularly useful in conjunction with other key levels, as they often coincide with existing support or resistance areas, enhancing their importance. Traders use these levels not only to anticipate market reversals but also to plan their trades more effectively by setting entry points, stop-loss orders, and take-profit targets. For example, a trader might enter a long position at a Fibonacci retracement level, place a stop-loss order just below the nearest support level, and set a take-profit target near a resistance level identified by a Fibonacci extension.

Both Support, Resistance and Fibonacci levels serve multiple functions in trading strategies. One of the primary uses is in determining entry and exit points. Traders often wait for the price to reach a key support or resistance level before entering a trade, as these levels tend to provide low-risk, high-reward opportunities. For instance, buying at a support level gives the trader a clear area to place a stop-loss order just below the support, minimizing potential losses if the trade doesn’t work

out. Similarly, selling or shorting at a resistance level provides a logical point to place a stop-loss order just above the resistance, protecting against unexpected breakouts.

Finally, and before we jump into the technical part, the risk-reward ratio is significantly influenced by the entry point's position relative to key levels. The distance to these levels plays a crucial role in shaping the potential outcome of a trade. Most trading systems and strategies today incorporate key levels, as they are among the most important technical indicators available. These levels are frequently respected by the price and can’t be ignored if we want to build a successful strategy.

2 Coding Support And Resistance

To develop an algorithm that detects support and resistance levels, we begin by utilizing fractals or pivot candles, as introduced in the previous chapter. Identifying these candles, where the price tends to bounce, is crucial because they provide the initial set of candidate levels that might be recognized as support or resistance. The simplest approach is to identify these levels where fractal candles form and then cluster them into zones based on their proximity. For instance, if there are several levels close to each other, they should be merged into a single level, creating a zone where the price consistently reacts. In summary, the process involves two key steps: first, gathering the levels, and second, clustering these levels to filter out or determine an average level for each cluster. The following code sample will demonstrate these steps in Python and visualize the results on a standard candlestick chart.

The first function we're using is the isPivot function that we have introduced in the previous chapter and it's how we will detect fractal candles or pivot candles. For the sake of avoiding you going back to the previous chapter I'll just include a copy of the same function right here we will save the explanations

because these were detailed in the previous chapter but just as a memory refresher you can find the code below:

```
def isPivot(candle, window):
    """
    function that detects if a candle is a pivot/fractal point
    args: candle index, window before and after candle to test if
pivot
    returns: 1 if pivot high, 2 if pivot low, 3 if both and 0 default
    """
    if candle-window < 0 or candle+window >= len(df):
        return 0

    pivotHigh = 1
    pivotLow = 2
    for i in range(candle-window, candle+window+1):
        if df.iloc[candle].Low > df.iloc[i].Low:
            pivotLow=0
        if df.iloc[candle].High < df.iloc[i].High:
            pivotHigh=0
    if (pivotHigh and pivotLow):
        return 3
    elif pivotHigh:
        return pivotHigh
    elif pivotLow:
        return pivotLow
    else:
        return 0
```

The next function, get_support_resistance_levels, takes three input parameters: the DataFrame, the starting row index, and the ending row index. This function identifies pivot candles within the specified range by checking the isPivot column, which marks candles as either pivot highs or pivot lows. The identified levels are then saved into a list, which we'll refer to as the support resistance list.

```
def get_support_resistance_levels(df, start_row=3, end_row=205):
```

```
    """
    Identify support and resistance levels based
    on pivot candles within a specified range.

    Parameters:
    df (pandas.DataFrame):  The DataFrame containing the price
                            data and the `isPivot` column.
    start_row (int): The starting row index for
                     identifying pivot candles. Default is 3.
    end_row (int): The ending row index for
                   identifying pivot candles. Default is 205.

    Returns:
    list: A list of tuples where each tuple contains:
        - The row index (int) where a support or
          resistance level was identified.
        - The price level (float) at that row.
        - An indicator (int) where 2 represents
          support and 1 represents resistance.
    """
    sr = []
    for row in range(start_row, end_row):
        if df.isPivot[row] == 2:  # 2 indicates a support level
            sr.append((row, df.Low[row], 2))
        elif df.isPivot[row] == 1:  # 1 indicates a resistance level
            sr.append((row, df.High[row], 1))
    return sr
```

Next, we can introduce a function called clean_support_resistance, which takes two parameters: the previously computed list of support and resistance levels and a threshold percentage. This threshold percentage determines how close two levels can be before they are considered for merging or removal. The threshold distance is calculated as a percentage of the current price level or key level being evaluated for removal. The function returns a tuple containing two lists, each list comprising tuples with three elements: the index, the price,

and the category (indicating whether it is a support or resistance level), just like in the previous function.

```
def clean_support_resistance(sr, threshold_percent=0.5):
    """
    Clean the support and resistance levels by removing levels that are
    close to each other based on a specified threshold percentage.

    Parameters:
    sr (list of tuples): A list of tuples where each tuple contains:
        - The row index (int) where a support or resistance level was identified.
        - The price level (float) at that row.
        - An indicator (int) where 2 represents support and 1 represents resistance.
    threshold_percent (float): The threshold percentage to determine how close
                               the levels can be before being considered for removal.
                               Default is 0.5%.

    Returns:
    tuple: Two lists of tuples:
        - plotlist1: Cleaned resistance levels.
        - plotlist2: Cleaned support levels.
    """
    # Extract resistance and support levels with their original format
    plotlist1 = [(x[0], x[1], x[2]) for x in sr if x[2] == 1]  # Resistance levels
    plotlist2 = [(x[0], x[1], x[2]) for x in sr if x[2] == 2]  # Support levels

    # Sort the lists by the price level
    plotlist1.sort(key=lambda x: x[1])
    plotlist2.sort(key=lambda x: x[1])

    # Remove close resistance levels based on percentage threshold
```

```
    i = 1
    while i < len(plotlist1):
        current_level = plotlist1[i][1]
        previous_level = plotlist1[i - 1][1]
        threshold = threshold_percent / 100 * current_level

        if abs(current_level - previous_level) <= threshold:
            plotlist1.pop(i)
        else:
            i += 1

    # Remove close support levels based on percentage threshold
    i = 1
    while i < len(plotlist2):
        current_level = plotlist2[i][1]
        previous_level = plotlist2[i - 1][1]
        threshold = threshold_percent / 100 * current_level

        if abs(current_level - previous_level) <= threshold:
            plotlist2.pop(i)
        else:
            i += 1

    # Return the cleaned lists with the original format
    return plotlist1, plotlist2
```

Now we can combine these functions to first compute the pivot, support, and resistance levels, and then merge and clean close levels for a more refined set of key levels.

```
window=3
df['isPivot'] = df.apply(lambda x: isPivot(x.name,window), axis=1)

support_resistance_levels = get_support_resistance_levels(df,
start_row=window, end_row=300)

cleaned_resistance, cleaned_support =
clean_support_resistance(support_resistance_levels,
threshold_percent=10)
```

Printing for example the cleaned resistance levels would show a list of tuples where each Topper represents a level and we have access to the index of the candle where the level occurred the price at which the level is detected and then a category 1 for resistance and 2 for support, the result look like this:

```
[(3, 1887, 1), (41, 2642, 1), (21, 2941, 1), (105, 4470, 1), (114, 5868, 1), (290, 6901, 1),
```

To plot the levels on a chart, we'll define a new function called plot_candlestick_with_levels. This function will take the DataFrame, the starting index for where the plot should begin, the ending index for where the plot should stop, a support_levels parameter for the list of support levels, and a resistance_levels parameter for the list of resistance levels.

```
import plotly.graph_objects as go

def plot_candlestick_with_levels(df, start=0, end=200,
support_levels=None, resistance_levels=None):
    """
    Plots a candlestick chart with support and resistance levels.

    Parameters:
    df (pandas.DataFrame): The DataFrame containing the OHLC
data (Open, High, Low, Close).
    start (int): The starting index for the plot. Default is 0.
    end (int): The ending index for the plot. Default is 200.
    support_levels (list of tuples): A list of tuples representing
support levels,
                                     where each tuple contains the index,
price level, and indicator.
    resistance_levels (list of tuples): A list of tuples representing
resistance levels,
                                        where each tuple contains the index,
price level, and indicator.

    Returns:
```

```
    None: Displays a candlestick chart with the specified support and resistance levels.
    """
    # Slice the DataFrame to the specified range
    dfpl = df[start:end]

    # Create the candlestick chart
    fig = go.Figure(data=[go.Candlestick(
        x=dfpl.index,
        open=dfpl['Open'],
        high=dfpl['High'],
        low=dfpl['Low'],
        close=dfpl['Close']
    )])

    # Add support levels as lines within the specified range
    if support_levels:
        for level in support_levels:
            if start <= level[0] < end:
                fig.add_shape(type='line',
                              x0=df.index[level[0]], y0=level[1],
                              x1=df.index[end-1], y1=level[1],
                              line=dict(color="MediumPurple", width=2)
                              )

    # Add resistance levels as lines within the specified range
    if resistance_levels:
        for level in resistance_levels:
            if start <= level[0] < end:
                fig.add_shape(type='line',
                              x0=df.index[level[0]], y0=level[1],
                              x1=df.index[end-1], y1=level[1],
                              line=dict(color="Green", width=2)
                              )

    # Display the chart
    fig.show()
```

Now calling this function is relatively simple:

```
plot_candlestick_with_levels(df, start=0, end=100,
support_levels=cleaned_support,
resistance_levels=cleaned_resistance)
```

And the result is shown in the following figures:





Note that the lines start at the candle and the level initiating the support or resistance, with two colors used: support levels are marked with purple lines, and resistance levels are marked with green lines. At this point, you can modify the function to suit your needs. You might also want to review the previous functions; perhaps you prefer to skip the merging step and keep all the levels at once. You can reproduce the code and experiment in your virtual lab before deciding how to integrate this tool into your trading system.

3 Coding Fibonacci Levels

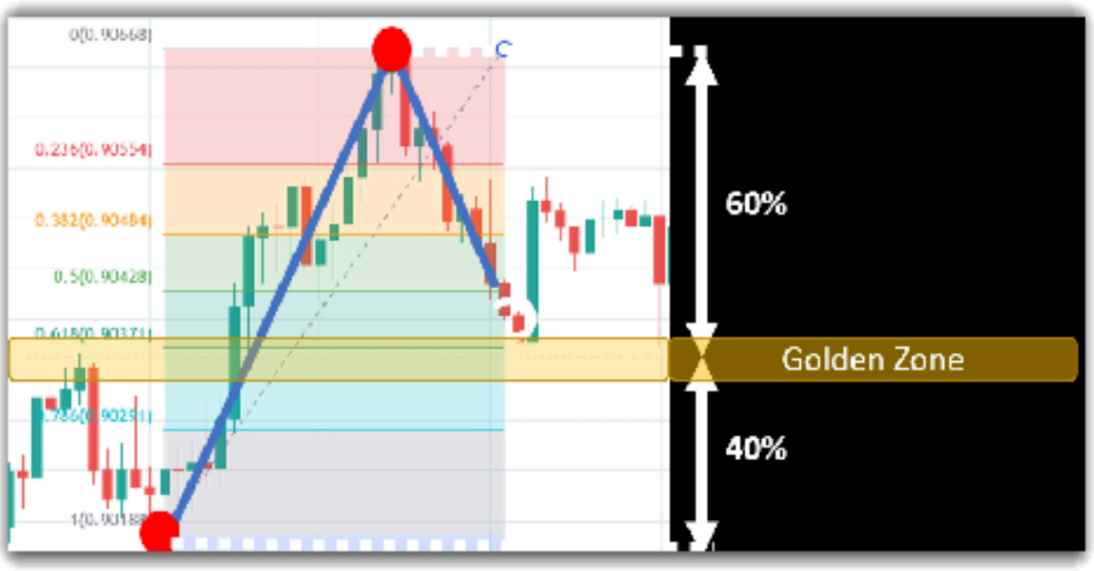
Fibonacci levels are a widely used tool in trading, derived from the Fibonacci sequence, which naturally appears in various aspects of life and nature. In trading, Fibonacci levels are calculated by taking significant price points, such as a high and a low, and dividing the vertical distance by key Fibonacci ratios: 23.6%, 38.2%, 50%, 61.8%, and 100%. Among these, the 61.8% level is often referred to as the "Golden Zone" because it is believed to be a critical area where the price is likely to retrace before continuing in the original direction.

One common trading strategy that relies on Fibonacci levels is to wait for the price to retrace to the Golden Zone (around 61.8%). Traders then place a stop loss just below the 100% level and set their take profit at the 0% level, creating a favorable risk-reward ratio. For example, if the price retraces to the 61.8% level, the distance to the stop loss (around 100%) is relatively small compared to the potential profit if the price moves back to the 0% level. This setup typically offers a risk-reward ratio of 1:2 or better, meaning that for every unit of risk, there is a potential reward of two units.

The success of this strategy is largely due to this favorable risk-reward ratio. By limiting potential losses and maximizing potential gains, Fibonacci levels help traders create setups where the rewards outweigh the risks, even if the strategy is only successful a fraction of the time. This is why Fibonacci levels, and particularly the Golden Zone, are highly regarded by traders, they help enforce disciplined trading and improve the likelihood of long-term success.

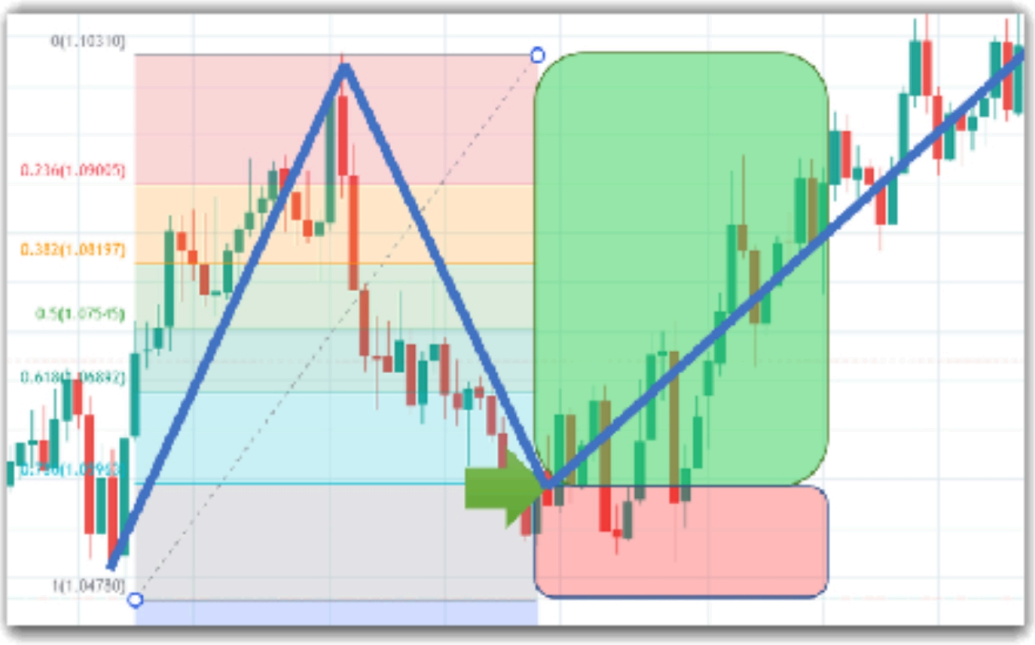
If you are unfamiliar with Fibonacci retracement, it involves identifying two extremes or clear pivot points, one pivot low and one pivot high. You then plot the Fibonacci levels between these two pivots, creating a matrix of levels. The strategy typically involves waiting for the price to retrace back to the Golden Zone,

which is around 61.8% or the 0.618 level, as illustrated in the figure below:



In this example, we focused on a particular configuration where we are anticipating an uptrend and looking for a long position. This configuration can be mirrored in a downtrend; in that scenario, we would plot the Fibonacci levels in reverse order and wait for the price to retrace up to the Golden Zone before taking a short position.

Ideally, the price would bounce back up, and the typical strategy uses the first Fibonacci level as the stop-loss point and the 0 level as the take-profit target, benefiting from a very advantageous reward-to-risk ratio. The figure below shows the stop-loss zone in red, the take-profit level in green, and the blue line representing the price retracement to guide the eye through the major movements.



When detecting pivot candles, it's important to consider that each candle requires checking both its preceding and following neighbors. To avoid lookahead bias when identifying two extreme pivot points for plotting Fibonacci levels, we should focus on a large window of past candles, or those preceding the current candle. Additionally, we need to account for a small gap, where we shift our computation backward. This gap is typically similar in size to the window used in the isPivot function when detecting pivot candles.

For the most recent candle, we limit our analysis to the current and past candles, ensuring we don't look into the future, because in real-time trading we don't have access to future data. This concept of a gap helps prevent lookahead bias and is illustrated in the figure below.



The generate_signal function takes a DataFrame, the index of the current candle, the number of back candles, and a gap parameter as inputs, along with two crucial thresholds: zone_threshold and price_diff_threshold. The zone_threshold ensures that the difference between the current closing price and the calculated entry level is within an acceptable range; if the distance is too large, the signal is invalidated. Additionally, the price_diff_threshold ensures that there is a significant distance between the identified minimum and maximum pivot points, which are used to calculate the Fibonacci levels for the entry, take profit, and stop-loss positions. If the price difference does not meet this threshold, the signal is also invalidated. Notice that in the function, only three levels are computed: the entry level, the stop-loss level, and the take-profit level. These levels are based on specific factors, 0.62 for the entry level and 0.78 for the stop-loss level, which can be easily adjusted by modifying these constants in the function. The take-profit level

currently uses a factor of 0, meaning it's set at the maximum or minimum price, but this can also be adjusted if you want to bring it closer to the entry price, such as by applying a factor of 0.1, for example.

Another key point within the function is that it accounts for the current trend based on the EMA (Exponential Moving Average) signal. This means that the function should be used after the moving average trend has already been determined. In the previous chapter, we discussed how to generate a trend signal by combining a fast and slow moving average, allowing us to easily identify the prevailing market trend.

In the function, you will find two configurations: one for a bullish market and another for a bearish market. The function returns a tuple of five values. The first value is the signal type, indicating whether the market is bullish or bearish, represented by an integer, 2 for bullish and 1 for bearish. The second element is the stop-loss level, which is a float, followed by the take-profit level, also a float. The last two values are integers representing the indexes of the minimum and maximum pivot points.

```
def generate_signal(df, l, backcandles, gap, zone_threshold,
price_diff_threshold):
    """
    Generates a trading signal based on EMA signals,
    Fibonacci retracement levels, and price action.

    Parameters:
    df (pandas.DataFrame): The DataFrame containing the relevant
                           market data (high, low, close prices, and
EMASignal).
    l (int): The current index or position in the
             DataFrame for which the signal is being generated.
    backcandles (int): The number of candles to look
                       back to find the pivot points (high and low).
    gap (int): The number of candles to look forward
```

```
            for confirming pivot points, avoiding lookahead bias.
    zone_threshold (float): The maximum allowed difference between
                            the current close price and the calculated entry level (l1).
    price_diff_threshold (float): The minimum price difference required
                            between the identified high and low for a valid signal.

    Returns:
    tuple: A tuple containing:
        - Signal type (2 for short, 1 for long, 0 for no signal)
        - Stop-loss level (float)
        - Take-profit level (float)
        - Index of the identified minimum price within the lookback window (int)
        - Index of the identified maximum price within the lookback window (int)
    """
    # Identify the maximum and minimum prices in the specified lookback window
    max_price = df.high[l-backcandles:l-gap].max()
    min_price = df.low[l-backcandles:l-gap].min()
    index_max = df.high[l-backcandles:l-gap].idxmax()
    index_min = df.low[l-backcandles:l-gap].idxmin()
    price_diff = max_price - min_price

    # Check for short signal
    if (df.EMASignal[l] == 2 and index_min < index_max and price_diff > price_diff_threshold):
        entry_level = max_price - 0.62 * price_diff  # Position entry at 0.62 Fibonacci level
        stop_loss = max_price - 0.78 * price_diff    # Stop-loss at 0.78 Fibonacci level
```

```
        take_profit = max_price                    # Take-profit at 0
Fibonacci level

        # Verify if the current close price is within the threshold zone
and confirm entry conditions
        if abs(df.close[l] - entry_level) < zone_threshold and df.high[l-
gap:l].min() > entry_level:
            return (2, stop_loss, take_profit, index_min, index_max)
        else:
            return (0, 0, 0, 0, 0)

    # Check for long signal
    elif (df.EMASignal[l] == 1 and index_min > index_max and
price_diff > price_diff_threshold):
        entry_level = min_price + 0.62 * price_diff  # Position entry
at 0.62 Fibonacci level
        stop_loss = min_price + 0.78 * price_diff    # Stop-loss at
0.78 Fibonacci level
        take_profit = min_price                    # Take-profit at 0
Fibonacci level

        # Verify if the current close price is within the threshold zone
and confirm entry conditions
        if abs(df.close[l] - entry_level) < zone_threshold and df.low[l-
gap:l].max() < entry_level:
            return (1, stop_loss, take_profit, index_min, index_max)
        else:
            return (0, 0, 0, 0, 0)

    # Return no signal if conditions are not met
    return (0, 0, 0, 0, 0)
```

Now we can use another function to apply trading signals by leveraging the previously defined generate_signal function. This new function will add five additional columns to the DataFrame: signal, SL (stop loss), TP (take profit), MinSwing (minimum

swing index), and MaxSwing (maximum swing index). These columns will store the results returned by the generate_signal function, effectively integrating the generated trading signals and associated levels into the DataFrame.

```
def apply_trading_signals(df, gap_candles=5, backcandles=40,
                          zone_threshold=0.001,
price_diff_threshold=0.01):
    """
    Applies trading signals to the DataFrame using the
generate_signal function.

    Parameters:
    df (pandas.DataFrame): The DataFrame containing the market
data.
    gap_candles (int):  The number of candles to use as a gap
                        for pivot point detection. Default is 5.
    backcandles (int): The number of candles to look back
                       for identifying pivot points. Default is 40.
    zone_threshold (float): The maximum allowed difference
between the
                            current closing price and the entry level.
Default is 0.001.
    price_diff_threshold (float): The minimum price difference
between identified
                                  pivot points to validate a signal. Default is
0.01.

    Returns:
    pandas.DataFrame: The updated DataFrame with new columns
for trading signals,
                      stop-loss, take-profit, minimum swing, and
maximum swing.
    """
    signal = [0] * len(df)
    TP = [0] * len(df)
    SL = [0] * len(df)
```

```
    MinSwing = [0] * len(df)
    MaxSwing = [0] * len(df)

    for row in range(backcandles, len(df)):
        gen_sig = generate_signal(df, row,
backcandles=backcandles, gap=gap_candles,
                                  zone_threshold=zone_threshold,
                                  price_diff_threshold=price_diff_threshold)
        signal[row] = gen_sig[0]
        SL[row] = gen_sig[1]
        TP[row] = gen_sig[2]
        MinSwing[row] = gen_sig[3]
        MaxSwing[row] = gen_sig[4]

    df['signal'] = signal
    df['SL'] = SL
    df['TP'] = TP
    df['MinSwing'] = MinSwing
    df['MaxSwing'] = MaxSwing

    return df
```

And now we can just call the function and save the returned and updated dataframe:

```
df_with_signals = apply_trading_signals(df, gap_candles=10,
                                        backcandles=50,
                                        zone_threshold=0.002,
                                        price_diff_threshold=0.02)
```

We could plot the chart with signals obtained using the Fibonacci retracement approach, a typical signal is shown in the figure below:



4 Conclusion

In conclusion, we have seen how to implement support, resistance, and Fibonacci levels in our coding in Python. In trading these are considered powerful tools, offering valuable insights into potential market turning points and guiding entry and exit decisions. However, it's important to recognize that these tools don't need to be used all the time or all at once. Depending on the strategy and market conditions, a trader can selectively apply these techniques to enhance their approach. The key is to understand how each tool fits into your broader trading strategy and to use them in a way that complements your overall goals. Whether you're focusing on support and resistance for trend identification or using Fibonacci levels for precision in entries and exits, the flexibility to adapt and choose the right tool for the right situation is what ultimately strengthens your trading strategy.

In algorithmic trading, there's no need to guess. If you're uncertain about whether to include an additional indicator in your trading system and strategy, you can simply run an

experiment programmatically. By backtesting the strategy on historical data, you can compare different configurations and scenarios to determine the best possible setup for real-time trading. This ability to test and refine strategies is one of the most exciting aspects of algorithmic trading. So, whenever you're unsure about an indicator and wonder if it's worth including in your strategy, take it to the backtesting lab, the results will speak for themselves.

Chapter 7: Statistical Evaluation of Technical Indicators

Evaluating technical indicators is a crucial step in developing and refining a trading strategy. There are various methods to assess an indicator's effectiveness, ranging from simple statistical approaches to more complex techniques that require extensive coding and trading expertise. While fully backtesting an indicator within a complete strategy provides a comprehensive evaluation, such methods can be intricate and time-consuming. These advanced approaches will be discussed in later chapters of this book. For now, we will focus on simpler forms of evaluation using basic statistical methods such as correlations, counting percentages, and calculating price averages.

The advantage of using simpler statistical methods for evaluating technical indicators lies in their ability to isolate the performance of the indicator from the broader context of a full trading strategy. By doing so, we can better understand how the indicator behaves independently, without the noise introduced by other components of a trading system. This isolation is particularly important during the testing phase, as it helps to filter out irrelevant data and allows us to determine whether the

indicator has the potential to provide a genuine edge in the market.

In this chapter, we will explore various statistical approaches that can be applied to evaluate technical indicators. The goal is not to implement a full trading system but to focus on the indicator itself. This type of analysis is valuable because it allows us to test a small component of a strategy, specifically the trading indicator, before committing to a more complex evaluation.

Technical indicators typically play a significant role in influencing the entry points of a trade. In some cases, they are also used in exit strategies, trade management, or to measure market volatility. For instance, an indicator might be employed to predict the category of future price movement, whether the price is likely to rise, fall, or remain stable, another indicator might probe volatility and provide a range of movement regardless of the price direction, which is important to set exit levels. In this chapter, however, our focus will primarily be on the role of indicators in determining when to enter the market and in which direction. This will form the basis of our evaluation in the following sections.

Methods presented in this chapter can be considered as the easiest and most accessible ways to start testing trading systems, or at least a small part of them.

1 Labelling Future Price

To effectively test and evaluate technical indicators, we need a reference or ground truth data to either correlate the indicator's signals with this data or apply a statistical comparison. There are several ways to label the future price of an asset. For instance, we could count the number of red and green candles over the next 10 candles to estimate the momentum of the future price. Alternatively, we could calculate the total movement

of red candles versus green candles to get a sense of the price direction. Another approach is to compute the average closing price of future candles, providing a numerical value that can be compared to the current candle's closing price.

By comparing this average with the current closing price, we can categorize the future price movement into three categories: if the average is higher than the current price, it suggests an uptrend; if lower, a downtrend; and if the average remains within a certain range around the current price, it indicates a ranging market. It's crucial to understand that we're not using this labeling for trading; rather, we're using historical data to look into the future from the perspective of a specific candle. This allows us to evaluate how often a signal, like a bullish engulfing pattern for example, correctly predicts future price movement by comparing the pattern with what actually happened afterward.

In the following subsections, we'll explore two methods for labeling future candles and price movements. The first method involves calculating the average of the next several candles, such as their closing or middle prices, and comparing this average to the current candle's closing price. We will then categorize the future movement into three categories: uptrend, downtrend, and ranging market.

But before we start just a quick reminder, we can easily get the data of any acid using Yahoo finance as seen earlier in this book, here's the code I used to get and clean the data for the following examples:

```
import yfinance as yf
import pandas as pd
df = yf.download("BTC-USD", period='400d', interval='1d')
df=df[df["High"]!=df['Low']]
df.reset_index(inplace=True)
```

1.1 Average Future Price

To compute the average of future candles or the future price

values we can consider different variations. Either we compute the average of the closing prices or the average of the middle prices. The following example shows a function that takes a data frame, the index of the current candle, and the number of future candles to consider for the average value. It returns the mean of closing prices of the next N candles.

```
def average_next_n_candles(df, i, N=10):
    """
    Calculate the average closing price of the next N candles.

    Parameters:
    df (DataFrame): The data frame containing the candle data.
    i (int): The index of the current candle.
    N (int): The number of candles to consider after the current
one.

    Returns:
    float: The average closing price of the next N candles, or None
if there are
           fewer than N candles remaining.
    """
    # Check if there are N candles after the current one
    if i + N >= len(df):
        return None

    # Compute the average closing price of the next N candles
    return df['Close'].iloc[i+1:i+N+1].mean()
```

We can modify the function to compute the mean of the mid prices of candles, and this is shown in the following example:

```
def average_next_n_mid_prices(df, i, N=10):
    """
    Calculate the average mid price (average of High and Low) of
the next N candles.

    Parameters:
    df (DataFrame): The data frame containing the candle data.
```

```
    i (int): The index of the current candle.
    N (int): The number of candles to consider after the current
one.

    Returns:
    float: The average mid price of the next N candles, or None if
there are
           fewer than N candles remaining.
    """
    # Check if there are N candles after the current one
    if i + N >= len(df):
        return None

    # Compute the mid price for each of the next N candles
    mid_prices = (df['High'].iloc[i+1:i+N+1] +
df['Low'].iloc[i+1:i+N+1]) / 2

    # Return the average of these mid prices
    return mid_prices.mean()
```

Now we can apply any of these functions to create a new column in the data frame where we save the results of the averaged prices.

```
df['future_average'] = df.apply(lambda row:
average_next_n_candles(df, row.name, 10), axis=1)
```

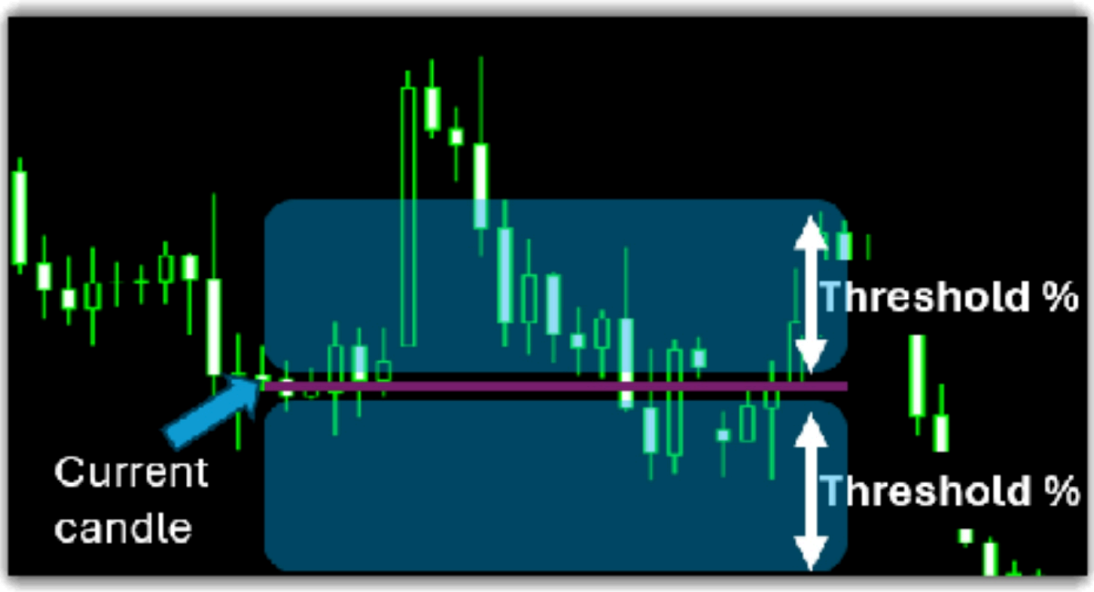
And the dataframe now looks like follows:

	Date	Open	High	Low	Close	Adj Close	Volume	future_average
0	2023-07-15	30331.783203	30407.781250	30263.462891	30295.806641	30295.806641	8011667756	29812.691602
1	2023-07-16	30297.472656	30437.560547	30089.669922	30249.132812	30249.132812	8516564470	29723.275586
2	2023-07-17	30249.626953	30336.400391	29685.783203	30145.888672	30145.888672	13240156074	29629.755664
3	2023-07-18	30147.070312	30233.656250	29556.427734	29856.562500	29856.562500	13138897269	29576.024023
4	2023-07-19	29862.046875	30184.181641	29794.269531	29913.923828	29913.923828	12128602812	29520.323437

1.2 Category Of Future Price

Now we can add some lines of code and modify the function

so that it will categorize the future of the price. The function returns 2 if the future average price is more than a threshold percentage above the current close, and it returns one if the future average price is further than a threshold percentage below the current close. The function also returns 0 if the future average price is within the threshold range. The threshold idea is illustrated in the following figure:



The function written in python is the following:

```
def category_next_n_candles(df, i, N=10, threshold_pct=1.0):
    """
    Calculate the average closing price of the next N candles and compare it
    to the current candle's close price based on a percentage threshold.

    Parameters:
    df (DataFrame): The data frame containing the candle data.
    i (int): The index of the current candle.
    N (int): The number of candles to consider after the current one.
```

```
    threshold_pct (float): The percentage threshold to compare the future
                           average price.

    Returns:
    int:
        - 2 if the future average price is above the current close price by
          more than the threshold.
        - 1 if the future average price is below the current close price by
          more than the threshold.
        - 0 if the future average price is within the threshold range.
    """
    # Check if there are N candles after the current one
    if i + N >= len(df):
        return None

    # Compute the average closing price of the next N candles
    future_avg = df['Close'].iloc[i+1:i+N+1].mean()

    # Calculate the percentage difference between the current close and the future average price
    current_close = df['Close'].iloc[i]
    pct_diff = ((future_avg - current_close) / current_close) * 100

    # Return the appropriate value based on the percentage difference
    if pct_diff > threshold_pct:
        return 2
    elif pct_diff < -threshold_pct:
        return 1
    else:
        return 0
```

Applying the function to our dataframe:

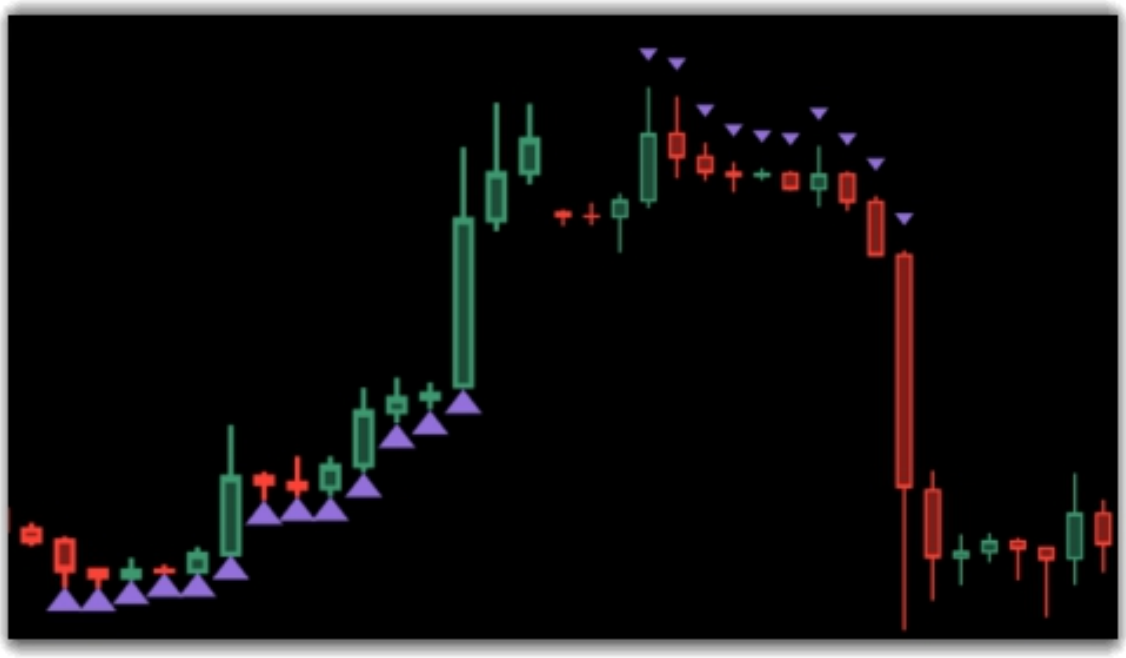
```
df['future_cat'] = df.apply(lambda row:
category_next_n_candles(df, row.name, 10, threshold_pct=2),
axis=1)
```

Now our dataframe has an additional column:

	Date	Open	High	Low	Close	Adj Close	Volume	future_average	future_cat
8	2023-07-23	29790.111328	30330.640625	29741.527344	30084.539062	30084.539062	9220145050	29297.924414	1.0
24	2023-08-08	29180.019531	30176.796875	29113.814453	29765.492188	29765.492188	17570561357	28708.235742	1.0
25	2023-08-09	29766.695312	30093.435547	29376.800781	29561.494141	29561.494141	18379521213	28361.706836	1.0
26	2023-08-10	29563.972656	29688.564453	29354.447266	29429.591797	29429.591797	11865344789	28037.706055	1.0
27	2023-08-11	29424.902344	29517.773438	29253.517578	29397.714844	29397.714844	10195168197	27710.348633	1.0

Here I only printed rows where the future_cat value is different than 0. And as we can see it's hard to assess how the function is working and whether the threshold of 2% that we have used is the optimal value for our data. We can however visualize the signals or the future categories on a chart for further inspection.

The visualization can be done using the add_pointpos_column() and the plot_candlestick_with_signals() functions that were introduced in the chapter on advanced indicators and automated patterns detection. The results are shown for samples from the data frame in the figure below:



We can see the purple triangles below the candles that are followed by an uptrend or an upward price movement. And the purple points are above the candle 4 candles that are followed by a downward movement of the price. There's no specific way to choose the threshold it's somehow a process of trial and error just make sure that you're not getting signals on all the candles because that wouldn't be possible and not to get too selective in a way that you'll get very few signals on the whole dataframe. In this example I found the value of 8% has a good threshold because it cleans up some of the noisy signals and it just keeps the signals where significant movements occurred, the following figure shows the signals on a wider range of the dataframe.



2 Testing The Rejection Candle

In this section, we will build on the concepts introduced earlier in this chapter to statistically test the validity of rejection candles. Our approach involves detecting all rejection candles using the functions we covered in the chapter on advanced technical indicators and automated candle pattern detection. Once these rejection patterns are identified, we'll count how often they are followed by the expected price movement over the next 10 days, since we're using the daily time frame in this analysis. Specifically, if a bullish rejection candle is detected, we'll check whether the future average price or price category aligns with the direction indicated by the rejection candle. The same process will be applied to bearish rejection candles. We'll track how frequently these patterns correctly forecast the subsequent price direction. At this point in the book, you have all the necessary tools to program and perform this testing. We've already discussed how to label and categorize future price movements, as well as how to detect rejection candles and other candle patterns that we will explore further in the next section.

The following function, test_signal_accuracy, takes three parameters: the DataFrame, the name of the signal column, and the name of the price category column within the DataFrame. This function compares the values in the signal column with those in the price category column, counting how often each signal category is confirmed by the future price category. In other words, it calculates the accuracy of the signal based on the actual future price movements. The function returns two metrics: the number of times the signal was correct and the number of times the predicted signal differed from the actual future price trend.

```
def test_signal_accuracy(df, signal_col, price_cat_col):
    """
    Calculate the accuracy of the signal column against
    the price category column.

    Parameters:
    df (pd.DataFrame): The dataframe containing the data
    signal_col (str): The column name for signal
    price_cat_col (str): The column name for price category

    Returns:
    tuple: A tuple containing the percentage of
           equal signals and different signals
    """
    equal_count = 0
    different_count = 0
    total_count = 0

    for i in range(len(df)):
        if df[signal_col].iloc[i] != 0:
            total_count += 1
            if df[signal_col].iloc[i] == df[price_cat_col].iloc[i]:
                equal_count += 1
            else:
                different_count += 1
```

```
    equal_percentage = (equal_count / total_count) * 100
    different_percentage = (different_count / total_count) * 100

    return equal_percentage, different_percentage
```

Upon the initial inspection using this function, I found that only 5% of the signals were accurate, while 95% were incorrect, meaning the signal did not align with the future price direction. As you can see, this is a poor outcome, indicating that the rejection candle alone is not a reliable trading signal. However, we can refine our approach by adjusting some parameters, such as reducing the number of days we look ahead to categorize the mean of prices. Instead of 10 days, we could decrease this to five days. The rationale behind this adjustment is that a rejection candle is typically followed by an immediate price reaction, so we don't need to wait 10 days to observe the movement. If the price moves after such a long period, it is likely unrelated to the initial rejection candle. This example highlights the importance of applying thoughtful consideration to the numbers we use in our trading algorithms.

For instance, I used the is_rejection function with a body percentage of 0.5% and a wick ratio of 3. For the future price category, I used a 5-day period with a threshold of 2%, assuming that within five days, a price movement exceeding 2% is a favorable outcome. Testing with these parameters resulted in a signal accuracy of 33.3% and an inaccuracy of 66.6%. While this is not perfect, it’s important to remember that we are only analyzing the rejection candle here. The purpose of this brief study is not to prove that the rejection candle can predict future prices with 100% accuracy but to identify the best conditions and configurations under which the rejection candle can be useful on the daily time frame.

From this, we can conclude that a rejection candle is more closely related to price movements within the next five days rather than ten days. It also appears to be predictive of price movements up to 2%, but not for larger movements, such as

those above 10%, which would be excessive. We can further build on this pattern by combining it with other candle patterns. For example, what if a rejection candle is followed by a confirmation candle in the same direction, would this improve the prediction accuracy? Or perhaps using the rejection candle in conjunction with two moving averages to detect rejections that align with the current trend? As you can see, it is now quite straightforward to piece together these code elements to test different patterns and assess whether they enhance the accuracy of our predictions.

3 Testing The Engulfing Pattern

We can apply a similar approach to test the engulfing pattern. In this example, I used the is_engulfing function introduced earlier in the chapter on advanced technical indicators and automated candle pattern detection. For this trial, I set the engulfing difference percentage threshold to 0.01%. I then compared the signal generated by this function to price categories based on a 2% price change within the next 5 candles (or the next 5 days following the engulfing pattern). The results showed that 33.3% of the predictions were correct, while 66.6% were incorrect in forecasting the future price trend.

4 Inferential Statistics and Future Price Correlation

In this paragraph, we will explore a more advanced analysis using financial statistics. However, it's important to remember that more complex or difficult methods don't necessarily lead to better results. The simple percentages we discussed in the previous sections are often sufficient to determine whether the pattern or indicator being studied has potential and if it's worth

incorporating into a full trading strategy.

That said, if you're curious and enjoy delving into numbers, and you want to take your analysis to the next level, advanced statistical techniques can be applied, as we will demonstrate in this section.

For this example, we'll shift our focus from the categorical features of future price movements to the numerical data we calculated earlier using the average_next_n_candles function. This function returns the average price for a set number of future candles. By calculating the difference between this average value and the current closing price, we can determine whether the price is trending up or down. Additionally, this approach allows us to quantify the magnitude of the price movement, providing insight into how much the price has increased or decreased.

With this numerical data, we can perform statistical analysis to explore correlations between these price differences and signals like engulfing or rejection patterns, or any other indicator you might want to investigate. For each candle (or row in our DataFrame), we will have a column indicating the signal category and another column reflecting the price movement. This setup creates a relationship between a categorical variable and a numerical variable, which is a key consideration when applying inferential statistics.

```
df["future_diff"] = df["future_average"]-df["Close"]
```

4.1 Descriptive Analysis

Starting with descriptive analysis is a straightforward approach that can give an initial sense of the correlation between the two columns under study. The function calculate_descriptive_statistics takes the DataFrame, the signal column name, and the numerical column name as inputs, and it provides a statistical summary for each category within the signal column. In this context, we have three categories: 0

represents no specific candle pattern, 1 indicates a bearish signal such as a bearish rejection or bearish engulfing pattern, and 2 denotes a bullish signal, whether it's a bullish rejection or bullish engulfing pattern.

```
def calculate_descriptive_statistics(df, signal_col, num_col):
    """
    Calculate descriptive statistics for each category in the signal column.

    Parameters:
    df (DataFrame): The DataFrame containing the data.
    signal_col (str): The name of the categorical signal column.
    num_col (str): The name of the numerical column.

    Returns:
    DataFrame: Descriptive statistics for each category in the signal column.
    """
    return df.groupby(signal_col)[num_col].describe()

calculate_descriptive_statistics(df, "isRejection", "future_diff")
```

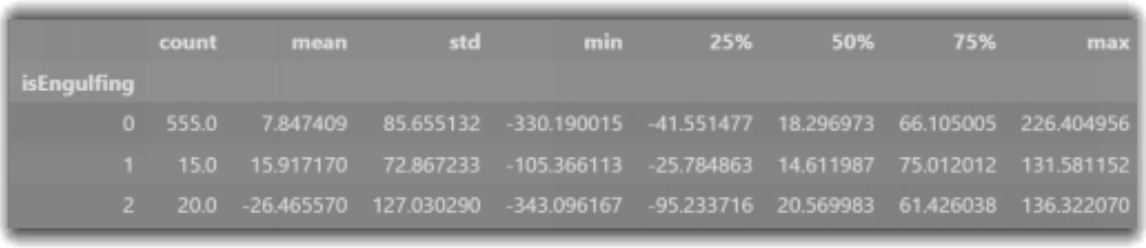
Notice that we've defined the function and called it for the isRejection signal. Here are the results:

isRejection	count	mean	std	min	25%	50%	75%	max
0	561.0	5.812462	87.076974	-343.096167	-43.327881	16.914258	64.761084	226.404956
1	10.0	23.257383	96.439506	-123.043896	-23.186279	37.447461	79.266199	177.720947
2	19.0	30.073303	83.703217	-148.887085	8.190405	40.992773	74.956616	173.660913

Notice that we have three categories, and the mean value when we have a bullish rejection is 30, which is higher than the mean of 23 for bearish rejections. Additionally, the median (represented by the 50% column) is around 41% for bullish signals and approximately 37% for bearish signals. Since this example is applied to the S&P 500, a market that generally

trends upward, most values tend to be positive for both signals. However, in this case, the bullish signals indicate a stronger upward momentum compared to the bearish signals.

I also ran the same analysis for the engulfing pattern, and here are the results:



isEngulfing	count	mean	std	min	25%	50%	75%	max
0	555.0	7.847409	85.655132	-330.190015	-41.551477	18.296973	66.105005	226.404956
1	15.0	15.917170	72.867233	-105.366113	-25.784863	14.611987	75.012012	131.581152
2	20.0	-26.465570	127.030290	-343.096167	-95.233716	20.569983	61.426038	136.322070

As we can observe, the mean is negative for the bullish signal and positive for the bearish signal. However, the medians tell a different story, with 20.5 for bullish signals and only 14 for bearish signals.

This highlights an important point in both statistics and algorithmic trading: the sample size. To draw statistically valid conclusions, especially in trading, it's crucial to have a large number of data points. In this example, we have 10 bearish rejection candles, 19 bullish rejection candles, 15 bearish engulfing candles, and 20 bullish engulfing candles. While this sample size might be somewhat acceptable for general statistical analysis, it is insufficient for trading purposes. The financial markets are highly noisy, with frequent fluctuations, and such a small sample size is unlikely to produce reliable signals amid all the market noise.

4.2 One Way ANOVA Testing

A one-way ANOVA is a statistical test used to compare the means of a numerical variable across different categories or groups. The purpose of the test is to determine whether the differences observed in group means are statistically significant by examining the variance within each group and the variance

between the group means. Essentially, ANOVA helps us identify whether the variability in the data can be attributed to the differences between group means or if it is simply due to random variation within the groups. This analysis can help us assess whether the signal categories we're studying have a meaningful impact on the numerical values of future prices.

The perform_anova_test function takes three parameters: the data frame, the signal column name, and the numerical column name. It returns two key metrics: the F-statistic and the p-value. A higher F-statistic indicates a greater likelihood that the group means are significantly different, suggesting that the categorical variable might influence the numerical values. Conversely, a high p-value suggests that any observed differences in means are likely due to random variation rather than the effect of the group categories. Typically, a p-value below 0.05 is considered statistically significant. In summary, the F-statistic measures the extent to which group means differ relative to the variability within the groups, while the p-value indicates whether this difference is statistically significant.

```
from scipy.stats import f_oneway

def perform_anova_test(df, signal_col, num_col):
    """
    Perform a one-way ANOVA test to check if the means of the numerical column differ between signal categories.

    Parameters:
    df (DataFrame): The DataFrame containing the data.
    signal_col (str): The name of the categorical signal column.
    num_col (str): The name of the numerical column.

    Returns:
    dict: A dictionary with the F-statistic and p-value.
    """
    categories = df[signal_col].unique()
```

```
    category_data = [df[df[signal_col] == cat][num_col].dropna()
for cat in categories]
    anova_result = f_oneway(*category_data)
    return {
        "F-statistic": anova_result.statistic,
        "p-value": anova_result.pvalue
    }
perform_anova_test(df, "isRejection", "future_diff")
```

I applied the function to the rejection pattern and obtained the following results: 'F-statistic': 0.8919769036414312, 'p-value': 0.41039948564163575.

The F-statistic can take on any non-negative value, with no upper limit. It reflects the ratio of variance between group means relative to the variance within the groups:

- **Low F-statistic (close to 1):** This indicates that the variance between the group means is similar to the variance within the groups, suggesting that the group means are not significantly different from each other.
- **High F-statistic:** A high F-statistic suggests that the variance between group means is much larger than the variance within the groups, indicating a significant difference between the group means.

In this case, the F-statistic of 0.89 is relatively low, even below 1, which suggests that the variance between the groups is not significantly different from the variance within the groups.

The p-value of 0.41 represents the probability of observing an F-statistic as extreme as the one calculated, assuming there is no difference between the groups. Since the p-value is greater than the commonly used threshold of 0.05 for statistical significance, we fail to reject the null hypothesis. This means there is no statistically significant difference in the mean of the future_diff values across the different categories of the rejection

signal column.

4.3 Chi-Squared Independence Testing

The Chi-square test is a statistical method used to evaluate whether there is a significant association between two categorical variables. In this analysis, we apply the Chi-square test to determine if the signal categories (e.g., bullish, bearish, or neutral) in the isRejection column are significantly related to the discretized numerical column representing the future price movement.

The perform_chi2_test function is used for this purpose. It takes three parameters: the data frame, the signal column name (categorical), and the numerical column name. The function first discretizes the numerical column into three bins, creating categories for different ranges of future price movement. A contingency table is then generated to show the distribution of these categories across the different signal groups. The Chi-square test is applied to this table, and the function returns several important metrics:

- **Chi2-statistic**: This measures the extent to which the observed distribution of the data deviates from the expected distribution under the assumption of independence.
- **p-value**: The p-value indicates whether the observed deviation is statistically significant. A p-value below 0.05 typically suggests a significant association between the variables.
- **Degrees of Freedom**: This value represents the number of independent comparisons being made within the contingency table. The formula to calculate the degrees of freedom for a Chi-square test is: Degrees of Freedom=

(Number of Rows−1)×(Number of Columns−1)

- The isRejection signal column has 3 categories (e.g., 0, 1, 2). The discretized numerical column num_category has 3 categories (bins). So we will apply the formula to obtain the degrees of freedom: Degrees of Freedom= (3−1)×(3−1)=2×2=4
- **Expected Frequencies**: These are the expected counts in each category if there were no association between the signal and the future price movement.

Important: If you're not comfortable with inferential statistics, it might be helpful to explore some introductory resources, such as books or online courses, that cover the basics of independence tests. However, it's worth noting that a deep understanding of these tests isn't essential for algorithmic trading. In many cases, simply calculating the percentage of correct versus incorrect predictions can be a sufficient way to evaluate the accuracy of an indicator, especially when the number of signals is large (e.g., over 1000 signals, with around 500 per category). Inferential statistics becomes more relevant when the number of signals is low and the results are ambiguous, making it difficult to draw clear conclusions.

```
from scipy.stats import chi2_contingency

def perform_chi2_test(df, signal_col, num_col):
    """
    Perform a Chi-square test to examine the association between the categorical signal and numerical values.

    Parameters:
    df (DataFrame): The DataFrame containing the data.
    signal_col (str): The name of the categorical signal column.
    num_col (str): The name of the numerical column.
```

```
    Returns:
    dict: A dictionary with the Chi2-statistic, p-value, degrees of
freedom, and expected frequencies.
    """
    df['num_category'] = pd.cut(df[num_col], bins=3, labels=[0, 1,
2])  # Discretize the numerical column into 3 bins
    contingency_table = pd.crosstab(df[signal_col],
df['num_category'])
    chi2_result = chi2_contingency(contingency_table)
    return {
        "Chi2-statistic": chi2_result[0],
        "p-value": chi2_result[1],
        "degrees_of_freedom": chi2_result[2],
        "expected_frequencies": chi2_result[3]
    }
perform_chi2_test(df, "isRejection", "future_diff")
```

When applying this function to the rejection pattern, the results were as follows:

- Chi2-statistic: 4.28
- p-value: 0.37
- Degrees of Freedom: 4

The Chi2-statistic of 4.28 suggests a moderate level of deviation between the observed and expected frequencies. However, the p-value of 0.37, which is well above the conventional threshold of 0.05, indicates that this deviation is not statistically significant. Therefore, we fail to reject the null hypothesis, meaning there is no strong evidence of an association between the signal categories and the future price movements in this context.

5 Leveraging Statistical Results

Although our initial signal testing showed some promise with

a 33% accuracy in correct predictions, this figure alone doesn't guarantee a profitable trading system. With a 2.5 to 1 reward-to-risk ratio, such an accuracy could indeed be the foundation for a profitable strategy. However, the results from inferential statistics did not confirm any significant correlation between the signals we tested and the actual price trends. It's important to note that we only tested basic signals like classic engulfing candles and rejection candles, so these underwhelming results were somewhat expected.

That said, even with less-than-ideal results, this method of measuring and comparing signals can still be valuable. It allows us to assess which indicators might be more effective. For instance, by comparing rejection candles to engulfing candles, we can determine which might be more reliable or better correlated with future trends. While both might seem insufficient, one could still perform marginally better, and by identifying such "champion" indicators, we can combine them to reinforce entry signals in a comprehensive trading system.

The goal isn't necessarily to find the perfect indicator with 90%+ accuracy, as the market's noisy nature makes this challenging. Instead, it's about understanding which indicators work better in certain contexts. Keep in mind, however, that our sample size was relatively small, which limits the reliability of the statistical tests we conducted. Expanding the historical data set could enhance the validity of these results.

Furthermore, if you're considering using machine learning models in the future, you'll need to select the best-performing indicators as input features. Indicators that don't correlate well with the target labels can introduce noise, making it harder for the model to learn effectively, and leading to poor results.

If you're interested in improving the results from this section, you might try combining support and resistance levels with candlestick patterns that occur near these key levels on the daily time frame. Many experienced traders report that rejections at

these levels tend to offer stronger signals for future price direction.

These are just a few ideas to consider before we close this chapter. The possibilities for building and analyzing technical indicators against future price movements are vast. In the next chapter, we'll shift our focus to developing a complete trading strategy and backtesting it on historical data using Python. Interestingly, sometimes indicators that didn't perform well in statistical tests can still yield positive results when paired with effective trade management strategies. While it's easier when an indicator shows strong forecasting ability, remember that a successful strategy functions as an integrated system, where every component, from indicators to trade management, works together.

Chapter 8: Building Trading Strategies

1 Introduction

Let me start by sharing a story from the early days of my trading journey, over a decade ago. A friend and I came up with what we thought was a foolproof strategy: we would trade in opposite directions. If he bought, I would sell, and if he sold, I would buy. Our thinking was that at least one of us would come out on top, after all, we couldn't both lose if we were on opposite sides of the market. But to our surprise, we both ended up losing our accounts. One of us might have lost more slowly, but in the end, we both wiped out.

This experience taught me a critical lesson about trading: success isn't just about finding the right entry point. It's equally

about managing the trade after it's opened. Trading isn't over once you enter the market; in fact, much of the work begins after that. How you react to market changes after entering a trade, how you manage the position, when you decide to exit, and how you adapt to new information, can make the difference between winning and losing. A well-timed entry is important, but leaving a trade to its fate without any further management is not a reliable strategy. It would be as if you are counting only on your entry indicators, and a full strategy system goes beyond just indicators.

Markets are dynamic and ever-changing. If your trade is open for a day or two, a lot can happen during that time. Ignoring these developments is like leaving valuable data on the table. This new information, which emerges after you've entered the trade, is technically in the future relative to your initial decision. Incorporating this data into your trading plan, whether manually or through automation, can significantly improve your chances of success. Of course, doing so adds complexity to your trading system, making backtesting and coding more challenging, but it's often worth the effort.

In this chapter, we'll discuss the importance of having a comprehensive trading strategy. We'll explore how to build a strategy step by step, understand the different types of strategies, and identify the key components that make a trading plan robust. A strategy that doesn't perform well in a backtest isn't necessarily a bad strategy, it might just mean that one component needs adjusting to fit better with the overall system. We'll explore these nuances and learn how to design and backtest a trading system using historical data.

While backtesting is an essential part of building a solid trading strategy, it's just the beginning. In the next chapter, we'll take things further by testing strategies live on the market. For now, let's focus on how to create a trading system that not only gets you into the market at the right time but also keeps you on

the right side of the trade as the market evolves.

2 Types of Trading Strategies

When it comes to trading in financial markets, one size definitely does not fit all. Different traders adopt different strategies based on their risk tolerance, market outlook, time availability, and overall goals. Understanding the variety of trading strategies will help you design a system that aligns with your individual style and objectives. In this section, we'll explore some of the most common types of trading strategies, including their key characteristics, advantages, and potential drawbacks.

2.1 Scalping

Scalping is often the go-to trading style for beginners, largely because it offers immediate results. New traders are typically drawn to scalping because they don’t have to wait long to see the outcome of their trades, positions are opened and closed within minutes. However, this fast-paced strategy comes with its own set of challenges that shouldn’t be overlooked.

Scalping is a high-frequency trading strategy where traders make numerous trades throughout the day, aiming to profit from small price movements. Positions are held for just a few seconds to a few minutes, with the goal of accumulating small gains that collectively result in a significant profit by the end of the trading session.

Scalping strategies usually operate on very short time frames, typically ranging from the 1-minute to the 5-minute chart, allowing traders to close trades quickly. This approach can lead to a high number of trades, sometimes hundreds per day, necessitating a focus on transaction costs and trade management. While the risk per trade is generally low, scalping requires rapid decision-making and the use of very tight stop losses.

Due to the rapid nature of scalping, traders typically rely on technical indicators rather than fundamental analysis. There simply isn't time to analyze news or financial reports, and even if you do, there's no guarantee that the market will move in the expected direction at the right time. Instead, technical indicators provide immediate feedback, often within seconds, making scalping an ideal candidate for algorithmic and automated trading, programs can execute trades entering and exiting the market much faster than traders. Commonly used indicators in scalping include moving averages, Bollinger Bands, and momentum oscillators, tools that are easily accessible through the Pandas technical analysis package we covered in an earlier chapter.

The main advantages of scalping include the potential for quick profits and reduced exposure to market changes. Since trades are held for only a short period, the impact of sudden market events or overnight changes is minimized. If the market moves against you, the worst-case scenario is usually a small loss on a trade or two before the system can be paused to avoid further losses.

However, there's a significant drawback to scalping that shouldn't be underestimated: the high transaction costs. Frequent trading incurs costs such as spreads, commissions, and slippage, which can quickly add up. Depending on your broker and account type, these costs might even surpass the profits from your trades. It's possible to have a profitable trading system on paper but end up with a loss in equity at the end of the day due to fees. The limited profit from each trade in scalping can be easily eroded by these costs. To mitigate this, it's essential to perform thorough historical backtesting, including commission calculations, to determine if a scalping strategy can remain profitable over time.

2.2 Day Trading

Day trading involves buying and selling financial instruments within the same trading day, ensuring that all positions are closed before the market closes. Day traders focus on capitalizing on intraday price movements and typically utilize a mix of technical analysis and news-based trading strategies. The time frames used in day trading are generally slower than those in scalping, ranging from 15-minute charts to hourly charts. These time frames allow traders to enter and exit positions within the same day, ensuring trades are closed before the market session ends.

The number of trades in day trading is moderate, often averaging around five trades per day, though this can vary based on the strategy and market conditions. The risk level is considered medium, with the primary challenge being the need to close all positions by the end of the day. This time constraint can put pressure on traders to exit trades prematurely, potentially affecting overall profitability.

Day trading relies heavily on technical indicators, but also considers the news calendar to avoid trading during major news events that could lead to unpredictable market volatility. Commonly used tools for algorithmic day trading include price action indicators, volume analysis, moving averages, Bollinger Bands, and custom technical indicators.

One of the main advantages of day trading is the elimination of overnight risk, as all positions are closed before the end of the trading session. This reduces exposure to market fluctuations that can occur after hours or over the weekend. For algorithmic trading, day trading is often more cost-effective than scalping due to the lower number of trades and the larger profit margins per trade, especially when using longer time frames like hourly charts.

In manual trading, one drawback of day trading is the

amount of time traders need to spend monitoring the markets. Traders must stay glued to their screens, waiting for opportunities and managing trades to ensure they are closed by the end of the day. This can be exhausting, particularly if the profits don't justify the time spent. This is where algorithmic trading shines, by automating the monitoring process, the algorithm watches the market for you, only alerting you when a trading opportunity arises. This makes day trading an ideal strategy for algorithmic trading, as it allows traders to avoid the long hours of screen time while still taking advantage of intraday market movements.

2.3 Swing Trading

Swing trading is a medium-term strategy that is slightly slower than day trading, with positions held for several days to a couple of weeks. The time frames used are typically above the hourly, often involving hourly and four-hour charts. For a broader perspective, traders might also glance at the daily time frame to assess the overall trend and identify key levels. The main objective in swing trading is to capture gains from price swings within a trend, using technical analysis to pinpoint entry and exit points based on market momentum and trends.

This strategy can be considered short to medium-term, meaning the number of trades is relatively low compared to scalping and day trading. This lower frequency of trades offers an advantage in terms of reduced trading costs and fees. However, the risk associated with swing trading may be slightly higher due to exposure to overnight and weekend market risks. If significant news is released during these periods when the market is closed, traders might find themselves unable to react until the next session opens. This can lead to volatile market movements, and in some cases, high slippage, especially if the market opens under heavy pressure with everyone reacting simultaneously.

Swing traders typically rely on indicators such as trend lines, moving averages, RSI, support and resistance levels, and Fibonacci retracement levels. These tools help in identifying key price levels and trends that can guide trading decisions. We've already covered key levels, Fibonacci levels, and classic technical indicators in earlier chapters, providing a solid foundation for developing swing trading strategies.

Algorithmic trading can be particularly advantageous for swing trading due to the slower pace of this strategy. Algorithms can efficiently monitor the market, waiting for specific patterns to form, and then notify the trader when opportunities arise. This allows the trader to make informed decisions about entering the market or staying out, with the aid of real-time data and other metrics. Profits from swing trading are often significant, as trades are held long enough to capture large price movements, benefiting from the extended time the market has to move in the desired direction.

The main drawback, as mentioned earlier, is the exposure to risks outside of trading hours. However, this is an inherent part of trading, and no strategy is without risk. Patience is also required, as traders must wait for the right conditions to align on the market. Fortunately, algorithmic systems can help mitigate some of these challenges by automating the monitoring process and reducing the emotional strain on traders as they wait for the ideal setup.

2.4 Position Trading

Position trading is a long-term strategy where traders hold positions for weeks, months, or even years to capitalize on sustained market trends. This approach is primarily based on fundamental analysis, with technical analysis often employed to refine the timing of entry and exit points. Given its long-term nature, position trading typically utilizes daily, weekly, or even monthly time frames. The number of trades executed is low, as

positions are maintained over extended periods. The risk associated with position trading is generally medium to high, as traders are exposed to market fluctuations and events over a prolonged duration.

This strategy leans heavily on fundamental analysis, focusing on economic indicators, interest rates, financial reports, and other macroeconomic data. Technical indicators are used primarily to fine-tune the timing of entry, such as waiting for a small retracement before entering a position based on the underlying fundamentals.

One of the key advantages of position trading is that it doesn't require constant market monitoring. Once a position is established, it can be left to run for months, with only occasional check-ins, perhaps once a week, to assess market conditions. This makes position trading an attractive option for those who are busy and cannot engage in more active trading styles. It offers a less hands-on approach, allowing traders to maintain a side trading activity with minimal engagement.

However, position trading demands patience and long-term commitment. From an algorithmic trading perspective, this style doesn't heavily rely on algorithms, as the decision-making process is largely driven by fundamental analysis. While it is possible to develop algorithms that analyze economic reports using natural language processing and artificial intelligence to automate fundamental analysis and market entry decisions, this type of trading requires less ongoing intervention from the trader. Decisions can be made over a few days and positions held for several months.

If you prefer a more active approach, conducting frequent fundamental analysis and increasing the number of trades to maximize potential returns, automated systems could be utilized to identify opportunities more regularly. However, this type of approach involves different tools, coding styles, and mathematical models, which are beyond the scope of this book.

The contrast between position trading and more active, algorithm-driven strategies highlights the diversity of trading approaches available to suit different preferences and lifestyles.

3 Algorithmic Trading

Each trading strategy comes with its own set of rules, risks, and rewards, making it essential to choose a strategy that aligns with your trading goals, risk tolerance, and the time you can dedicate to trading. Whether you prefer the fast pace of scalping or the patience required for position trading, your choice should also reflect your comfort level and experience. For instance, if you're not comfortable with reading and analyzing financial reports, position trading might be challenging. On the other hand, if you have a solid understanding of technical indicators, day trading might be a more natural starting point before transitioning to the faster-paced scalping.

Algorithmic trading can support any strategy, whether it involves high-frequency trading or longer-term systematic trading, depending on the design of the algorithm. There's no fixed time frame for algorithmic trading, it can be applied to any of the trading styles mentioned, with time frames ranging from minutes to daily or even weekly. The number of trades is also flexible, allowing for anything from low-paced strategies to extremely high-frequency trading.

In addition to the inherent risks of each trading strategy, algorithmic trading introduces specific risks related to the technology itself. Bugs can slip into the code, errors can occur in API usage, and issues can arise from the hardware or servers involved in the trading process. While these challenges can be addressed and corrected, it's crucial to monitor systems regularly, even when trades are going well.

One of the significant advantages of algorithmic trading is its ability to incorporate various types of indicators, technical,

fundamental, and statistical, into a single strategy. The analysis is performed quickly, providing clear results from complex mathematical computations to support your trading decisions. However, perhaps the most crucial advantage is that algorithmic trading removes emotional decision-making from the process. Emotional reactions, whether after a significant loss or a big win, can lead to poor trading decisions. By relying on algorithm-generated signals, you can avoid these emotional pitfalls.

Algorithmic trading also allows for backtesting strategies on historical data, which helps optimize performance and sets realistic expectations for a strategy's profitability over time. Rather than relying on luck, you can test a strategy over several years to understand its potential outcomes.

However, algorithmic trading comes with its own set of challenges. It requires significant upfront development time and technical expertise, particularly in coding and data science. Mastering a coding language like Python is essential for implementing these strategies effectively. Additionally, there's the risk of overfitting, where a strategy performs well in backtests but fails in real-world trading. Avoiding overfitting requires careful planning and testing to ensure the strategy is robust and adaptable to future market conditions.

Structuring the Core Components of an Algorithmic Trading Strategy

Designing an algorithmic trading strategy requires careful consideration and coordination of several crucial components. These components can be simplified into four primary categories that will guide us in constructing a reliable and effective

strategy:

- Trend Identification and Position Filtering
- Strategic Entry Points
- Intelligent Exit Strategies
- Managing Risk and Position Sizing

Each of these components plays a vital role in the overall success of your trading strategy. Understanding how they interconnect and influence one another is key to building a system that not only generates profitable trades but also manages risk effectively.

4.1 Trend Identification and Position Filtering

The first step in developing a robust trading strategy is to identify the prevailing market trend. This involves analyzing price movements to determine whether the market is trending upwards, downwards, or remaining relatively stable. By accurately identifying the trend, you can align your trading decisions with the market's direction, increasing the likelihood of success.

Position filtering comes into play once the trend is identified. This process involves sifting through potential trade opportunities and selecting only those that align with the identified trend. The goal here is to avoid trading against the market's momentum. By filtering out trades that go against the prevailing trend, you reduce the likelihood of entering positions that are likely to result in losses.

4.2 Strategic Entry Points

Once the trend is identified, the next step is determining the optimal entry points for your trades. The effectiveness of your strategy heavily depends on how well you time your entries.

Entering the market at the right time can significantly enhance your chances of profiting from the trade.

Strategic entry points are identified using various indicators and analysis tools. For instance, moving averages, Bollinger Bands, and RSI (Relative Strength Index) can provide valuable insights into when the market is poised to move in a direction that aligns with your strategy. The key is to enter trades when the conditions are most favorable, even if this means waiting for the perfect moment.

4.3 Intelligent Exit Strategies

A well-defined exit strategy is just as important as identifying the right entry point. An intelligent exit strategy ensures that you lock in profits when the market moves in your favor and minimize losses when it doesn't.

Exit strategies can be based on various factors, including price targets, trailing stops, or market conditions that signal a reversal. The exit strategy you choose should align with your overall trading goals and risk tolerance. For example, some traders prefer to set a fixed profit target, while others might use a trailing stop to allow profits to run while protecting against a sudden reversal.

4.4 Managing Risk and Position Sizing

Risk management is the cornerstone of any successful trading strategy. Without a proper risk management plan, even the best trading system can lead to significant losses. Managing risk involves determining how much capital you are willing to risk on each trade and adjusting your position size accordingly.

Position sizing should be based on the level of risk you are comfortable with and the volatility of the market. For example, in a highly volatile market, you might reduce your position size to

limit potential losses. Conversely, in a stable market, you might increase your position size to maximize profits.

An essential aspect of risk management is understanding that not all trades are equal. Some trades present higher risks, and it's important to recognize these situations and adjust your position size accordingly. By managing risk effectively, you can protect your capital and ensure that you remain in the market for the long term.

5 Trend And Entry Detection Examples

The trend can be measured in various ways, and there's plenty of room for creativity in this area. It's crucial to clarify that when we talk about measuring trends, we're assessing the past and present trends rather than predicting future trends. Forecasting future trends with certainty is impossible; instead, we base our trading decisions on the assumption that the current trend will likely continue for the next few hours or days. This assumption underscores the importance of integrating strong risk management practices into our strategy.

Here are a few methods you might experiment with to measure trends:

- **Moving Averages:** One common approach involves using two moving averages, a fast moving average and a slow moving average. By examining the order of these averages over a window of several candles (e.g., five or more), you can gauge the trend direction. If the fast moving average is above the slow moving average consistently over this window, you're likely in an uptrend. Conversely, if the fast moving average is below the slow moving average, it suggests a downtrend.

Remember, it's important to consider this over a slice of time rather than just a single candle.

- **Price Relative to Moving Average:** Another method is to check whether the price candles themselves are positioned above or below the moving average curve. This is done by analyzing a window of candles and determining if their open and closing prices are consistently above or below the moving average. If the candle bodies are mostly above the moving average, you're in an uptrend; if they're below, you're in a downtrend.
- **Volume Weighted Average Price (VWAP):** Instead of a simple moving average, you might use the VWAP, which not only smooths the price but also incorporates trading volume. You can then assess whether candles are trading above or below the VWAP curve to determine the trend direction. As with other methods, this condition should be verified over a window of candles, not just a single point in time.
- **Slope of Moving Averages:** A more refined method involves calculating the slopes of multiple moving averages over a recent window of time (e.g., the last 4, 5, or 10 candles). By computing two or three moving averages and checking their slopes, you can determine the trend direction. If the slopes are all positive and aligned in the same direction, it indicates an uptrend. If the slopes are all negative, it suggests a downtrend.

Next, we'll be coding an example of trend detection using candles relative positions. We'll skip the first point about checking the alignment of two moving averages, and measuring the slopes of moving average curves, as these were covered earlier in the chapter on advanced technical indicators.

The function below demonstrates how to check whether candles are positioned above or below a moving average curve. The function takes three inputs: a DataFrame, the number of backcandles (the slice of candles over which we apply the condition), and the name of the moving average column.

The function returns a category for each row (i.e., for each candle). If the candles in the given window are all positioned above the moving average, it returns a category of 2 (indicating an uptrend). If they are all below the moving average, it returns a category of 1 (indicating a downtrend). If the candles are mixed, some above and some below the curve, the function returns 0, indicating no clear trend.

This function can also be used to test if candles are trading above or below the VWAP curve by simply providing the VWAP column name instead of the moving average column. While this example focuses on the closing price of the candles for simplicity, the same logic can be applied to test both the opening and closing prices relative to the curve.

In the following example, we apply the function to a DataFrame using a 5-candle window and the SMA_20 moving average column.

```
def check_candles(df, backcandles, ma_column):
    """
    Categorize the trend based on the position of candle closes relative to a moving average.

    Parameters:
    df (DataFrame): The DataFrame containing the candle data.
    backcandles (int): The number of backcandles to consider for the trend calculation.
    ma_column (str): The name of the column representing the moving average.

    Returns:
    list: A list of trend categories where:
```

```
        - 2 indicates an uptrend (all close prices are above the moving average),
        - 1 indicates a downtrend (all close prices are below the moving average),
        - 0 indicates no clear trend.
    """
    categories = [0] * backcandles  # Initialize the first few elements as 0 (no trend)

    for i in range(backcandles, len(df)):
        close_prices = df['Close'][i-backcandles:i]
        ma_values = df[ma_column][i-backcandles:i]

        if all(close_prices > ma_values):
            categories.append(2)  # Uptrend
        elif all(close_prices < ma_values):
            categories.append(1)  # Downtrend
        else:
            categories.append(0)  # No trend

    return categories

# Apply the function to the DataFrame
df['Category'] = check_candles(df, 5, 'SMA_20')
```

5.1 Position Entry Signal Examples

Entry positions can be triggered using a combination of technical indicators, such as volatility measures, candle patterns, and key levels like support, resistance, or Fibonacci levels. For instance, the "golden zone" in Fibonacci retracement, briefly discussed in a previous chapter on advanced technical indicators, can be particularly useful.

By combining these indicators, we can generate more reliable entry signals. For example, consider a bullish rejection pattern forming near a support level, accompanied by a confirmed uptrend indicated by two moving averages where the fast moving average is above the slow moving average over the

last five candles. In this case, an entry position in the long direction would be triggered at the closing price of the rejection candle. The stop loss could be placed just below the low of the rejection candle, with a take profit target set at approximately twice the stop-loss distance.

Another approach could involve assessing the trend using the two moving averages and then utilizing Bollinger Bands to detect when the price deviates from the Bollinger center. For instance, in an uptrend, you might look for the price to dip below the lower Bollinger Band and then wait for confirmation that the price is reversing back up, aligning with the ongoing trend and converging towards the Bollinger Band's average (middle) line.

These scenarios can be efficiently coded in Python. Let's explore how to code Bollinger Bands and trigger an entry signal when a candle closes below the lower band. To keep the code simple, we'll focus solely on the Bollinger Band trigger, without additional confirmation from candle patterns.

We'll assume that the data frame is already loaded and that Bollinger Bands and Simple Moving Averages (SMA) have been computed. For the following examples, I've used a Bollinger Band length of 10 with a standard deviation of 1.5, and an SMA length of 20. This setup is applied to the daily time frame over 300 days of Bitcoin to U.S. dollar exchange prices. Additionally, we'll assume that the trend detection function has been applied, determining whether the closing prices of the back candles are trading above or below the SMA curve. This process has resulted in a new column called Trend_Category, which was created using the following code line:

```
# Apply the function to the DataFrame
df['Trend_Category'] = check_candles(df, 5, 'SMA')
```

With both the Bollinger Bands and the Trend_Category columns in place, we can now apply entry conditions to generate trading signals using the following function:

```
def apply_entry_conditions(df, trend_col, upper_band_col, lower_band_col):
    """
    Apply entry conditions to the DataFrame and assign entry categories based on trend and Bollinger Bands.

    Parameters:
    df (DataFrame): The DataFrame containing the data.
    trend_col (str): The name of the column indicating the trend category.
    upper_band_col (str): The name of the column indicating the upper Bollinger Band.
    lower_band_col (str): The name of the column indicating the lower Bollinger Band.

    Returns:
    DataFrame: The DataFrame with an additional 'entry' column indicating the entry signals.
    """
    df['entry'] = 0

    # Buy entry condition
    buy_entry_condition = (df[trend_col] == 2) & (df['Open'] < df[lower_band_col]) & (df['Close'] > df[lower_band_col])
    df.loc[buy_entry_condition, 'entry'] = 2

    # Sell entry condition
    sell_entry_condition = (df[trend_col] == 1) & (df['Open'] > df[upper_band_col]) & (df['Close'] < df[upper_band_col])
    df.loc[sell_entry_condition, 'entry'] = 1

    return df
```

The function accepts the data frame, the trend category column name, and the upper and lower Bollinger Band column names as input. You can apply it to generate a new, updated data frame using the following code line:

```
df= apply_entry_conditions(df, "Trend_Category", "Upper Band", "Lower Band")
```

We can visualize the results on a chart, displaying signal points as discussed in earlier chapters, along with the Bollinger Bands and the moving average curve.



On this chart, we can observe the moving average curve in white and how the candles crossed from below to above it, indicating a potential start of an uptrend that is confirmed later by the series of candles trading above the curve. We see one candle opening below the lower Bollinger Band, which is where the signal to enter a trade was generated. This represents an ideal scenario where the entry signal aligns well with market conditions. However, it's important to note that the algorithm also missed other potential opportunities, such as a previous instance where a candle didn't fully close or open below the lower Bollinger Band. Even though it might have been a good opportunity to enter a position, our algorithm didn't flag it because not all conditions were met. This illustrates that while the algorithm can be effective in certain situations, it may not be as sensitive to all market movements. An experienced trader

might outperform the algorithm in some cases, but relying on the algorithm provides consistency and reduces the emotional toll, even if it occasionally misses some opportunities.

As mentioned earlier, this same example can be expanded by adding an additional condition: the candle should open below the lower Bollinger Band and also be an engulfing candle, overtaking the previous red candle in an uptrend confirmed by the candles trading above the moving average curve.

6 How To Place a Stop Loss

The initial phase of trading involves detecting and entering the market, but the more challenging aspect is managing the trade after it's opened. This includes determining where to place the stop loss and take profit levels, as well as deciding whether to stick to these levels or adjust them dynamically according to market changes. This can complicate trade management significantly. In this section, we will explore various techniques for positioning stop losses.

The simplest method is to set a fixed stop loss distance, which is easy to implement but often less effective because it doesn't account for market conditions or volatility. More dynamic approaches include:

- **Candle-Based Stop Losses**: If you're using candle patterns like rejection or engulfing candles, you can place the stop loss above or below the high or low of the current candle, or nearby candles that mark a key swing level.
- **Key Level Stop Losses**: For strategies involving support and resistance or Fibonacci retracement, you can position the stop loss above or below these key levels, particularly the golden zone. This method adapts to significant market levels, potentially increasing the chances of a successful

trade.

- **Volatility-Based Stop Losses**: By using the Average True Range (ATR), you can set the stop loss as a multiple of the ATR value. This method adjusts the stop loss distance based on current market volatility, allowing for more flexibility and responsiveness to market conditions.
- **Risk-Based Stop Losses**: In a more advanced approach, the stop loss can be tied to the size of the trade, ensuring that the amount risked aligns with your overall risk management strategy. This will be discussed further in the next section on position sizing.
- **Trailing Stop**: A trailing stop moves with the market as long as the price is moving in your favor. If the price reverses, the trailing stop locks in profits by exiting the trade. This method is especially useful for capitalizing on long trends.

While stop losses are crucial, take profit levels are typically simpler to manage. In the most straightforward cases, you can set the take profit based on the stop loss, using a predetermined reward-to-risk ratio. For instance, if your stop loss is 10 pips and you aim for a 2:1 reward-to-risk ratio, your take profit would be set at 20 pips. This approach compensates for the inherent uncertainty of entry indicators, ensuring that even if your system has only a 45-50% success rate, it can still be profitable.

Ultimately, the best way to determine which stop loss method works best is through thorough backtesting. This ensures that your chosen indicators and stop loss strategies are well-suited to your trading system, this will be detailed in the chapter, we will delve into backtesting strategies and provide detailed coding examples for implementing the main stop loss methods.

7 Risk And Trade Sizing

When it comes to building a successful trading strategy, one of the most crucial yet often overlooked components is trade and risk sizing. While many traders focus intensely on predicting market moves, the reality is that how much you risk on each trade can have a far greater impact on your overall success. This section dives into three essential methods for trade sizing, each with its own approach to managing risk in algorithmic trading.

Before we discuss the methods, it's important to understand why trade sizing is so vital. Trade sizing is directly linked to risk management and, by extension, to your overall trade management strategy. It's not just about entering the market; it's about ensuring that each trade aligns with your risk tolerance and overall trading plan. Poor trade sizing can lead to excessive risk, potentially wiping out your account, or conversely, overly conservative sizing might result in missed profit opportunities, or slow account growth.

7.1 Method 1: Constant Trade Size

The first method, often favored by beginner traders, is the Constant Trade Size approach. Here, the lot size remains fixed regardless of market conditions, stop loss (SL) distances, or the current equity in the account. While this method is straightforward and easy to implement, it fails to adapt to the dynamic nature of the market or changes in your account balance.

For instance, if you always trade with a fixed lot size of 0.1, you might be risking too much on one trade when your account balance is low, or too little when your balance has grown. This lack of flexibility makes the Constant Trade Size method less effective in the long term, particularly as your account balance changes or market volatility increases.

7.2 Method 2: Equity-Based Trade Size

The Equity-Based Trade Size method introduces a dynamic element by adjusting the trade size based on the current equity in your account. Instead of using a fixed lot size, this approach calculates the trade size as a percentage of your total account balance. For example, if you decide to risk 2% of your $1,000 account, the trade size would be $20.

This method is advantageous because it allows your trade size to grow with your account, leveraging the compounding effect for potentially faster gains. As your account balance increases, so does your trade size, allowing you to capitalize on winning streaks. Conversely, during losing streaks, the method naturally reduces your trade size, helping to preserve capital. This adaptive nature makes the Equity-Based Trade Size method more resilient and aligned with the principles of sound risk management.

7.3 Method 3: Risk-Reward Ratio-Based Trade Size

The most advanced approach is the Risk-Reward Ratio-Based Trade Size method, which takes into account the distance of the stop loss and take profit (TP) levels. This method ensures that your trade size is not only aligned with your equity but also with the risk-reward ratio of each trade.

Here's how it works: before entering a trade, you calculate the distance to your stop loss in pips. This distance, combined with the amount of equity you're willing to risk (e.g., 2%), determines your trade size. For instance, if you're risking 12.5 pips and your account balance is $1,000, you'll calculate the lot size that corresponds to risking $20 (2% of $1,000).

This method is particularly effective because it aligns the

trade size with the actual risk of the trade. If the stop loss is closer, the trade size increases, and if the stop loss is further away, the trade size decreases. This ensures that each trade is optimized according to the risk-reward ratio, enhancing your ability to manage risk effectively.

7.4 Balancing Risk and Reward

An important consideration in the Risk-Reward Ratio-Based method is ensuring that the TP/SL ratio meets a minimum threshold. For instance, if your take profit is twice the distance of your stop loss, your risk-reward ratio is 2:1, which is generally favorable. However, if the ratio is less than 1.5, you might want to reconsider the trade. In such cases, even if the entry signal is strong, the trade might not be worth taking due to the unfavorable risk-reward ratio.

This method introduces an additional layer of decision-making: not only do you need to ensure that the entry signals are met, but you also need to confirm that the trade management signals (like the TP/SL ratio) are favorable. If either condition isn't met, it's often better to skip the trade and wait for a better opportunity.

8 Conclusion

We are obviously interested in applying these methods to automated trading strategy using python. In the next chapter, we'll walk through Python code examples implementing what we have learned so far, we'll demonstrate how to implement these trade sizing methods during backtesting, allowing you to see how they perform under historical market conditions. This hands-on approach will help you understand the practical implications of each method and how to incorporate them into your algorithmic trading systems.

Finally, while predicting market direction is important, the

size of your trades and the risks you take can significantly influence your overall success. Considering these three trade sizing methods when you are building a trading system will allow you to build a more robust and resilient trading strategy, that is capable of handling the ups and downs of the market.

Chapter 9: Backtesting Trading Strategies

1 Introduction

This chapter is dedicated entirely to the art and science of backtesting trading strategies. We'll dive into hands-on methods, complete with code samples, that you can replicate in your own environment. These examples will give you the foundation to experiment with different strategies, understand how they work, and eventually build and optimize your own profitable trading system. We'll build a strategy, analyze the results, and explore how to optimize various parameters to enhance profitability.

Before we dive into the complexities of creating and testing trading algorithms, it's essential to go through a backtesting checklist. This checklist ensures that we have all the necessary components in place to effectively evaluate our strategies. The checklist elements were previously discussed in earlier chapters here we're just recapping quickly what we'll need to start the strategy modeling. From your indicators

- **Define Trend Indicators and Entry Signals:** As discussed in earlier chapters these can range from simple moving averages to more complex indicators like the Relative Strength Index (RSI) or Bollinger Bands. Alongside trend indicators, you'll need to define entry signals.

- **Establish Lot Sizing Rules:** While not always mandatory, setting up lot sizing rules can help you manage risk, particularly in challenging market conditions. Lot sizing determines how many units of an asset you will buy or sell in each trade. This can be a fixed number or adjusted dynamically based on factors like asset volatility or account size.
- **Define Trade Management Rules:** Which involves deciding when to close your trades, whether it's at a specific profit target, stop loss level, or based on a signal from your indicators. You also need to decide whether to close trades partially or entirely. Partial closures allow you to lock in some profits while keeping a position open for further gains.

Backtesting allows you to test your strategies on historical data to see how they would have performed in the past but also this helps in refining and improving your strategies and tuning the parameters of your indicators to maximize profits.

Python offers a variety of backtesting tools, each with its own strengths. While we won't cover every tool in detail, mastering one or two will give you the capability to backtest almost any strategy. These tools are constantly updated, so it's important to stay informed about new features and improvements.

Some popular backtesting libraries include: Zipline, PyAlgoTrade, Catalyst, Backtrader, Vectorbt, and Backtesting.py.

While all these tools serve the same basic purpose, backtesting a strategy, they differ in terms of functionalities. For example, Vectorbt excels in testing strategies across a high number of assets at high speed, thanks to its vectorized approach. On the other hand, Backtesting.py offers a user-

friendly interface with a gentle learning curve, making it ideal for those new to backtesting. It includes features like leverage, commissions, trading fees, trailing stops, and visualization, which we'll be exploring in this chapter.

For this chapter, we'll focus on using Backtesting.py, but feel free to explore other tools if they align better with your needs. Documentation and support for each tool are readily available online, including forums and examples of trading strategies.

2 Case Study 1: Daily Support And Resistance Strategy

In this case study, we'll explore a strategy that serves as a starting foundation for developing your own trading system. The code provided can be copied and adapted to suit your needs. You can draw inspiration from the examples to create custom indicators and entry/exit signals that align with your strategy.

Our focus will be on backtesting this strategy using historical data to assess its performance and determine its viability for live trading. The strategy itself is relatively straightforward, targeting the daily time frame for Bitcoin/U.S. dollar exchange prices. We'll follow the checklist introduced earlier to build our algorithmic trading system.

Unlike trend-reliant strategies, this approach doesn't require trend identification. Instead, it centers around pinpointing key support and resistance levels and identifying a rejection candle or pattern that occurs near these levels. So, this strategy doesn't depend on trending markets or aim to follow a specific long-term trend. Instead, it focuses on capturing price swings and reacting to bouncing candles that occur between and around key levels. For instance, in our example, we observe a strong rejection candle forming just above a key support level, with all preceding candles maintaining positions above this level. This observation

is critical and will be a key factor in our Python code. When the price nears a support level and a rejection candle appears, this will act as our entry signal.

On the chart, the purple point highlights the actual signal generated by the algorithm we're developing.



When it comes to the exit signal and lot sizing, there are numerous approaches to consider. Options include using fixed stop-loss (SL) and take-profit (TP) ratios, basing SL and TP on market volatility through the ATR indicator, employing a trailing stop, or splitting the trade into two parts, closing half at a partial TP level and letting the other half continue to capture additional profits if the trend remains favorable.

In our example, we worked with 1,000 days of daily candles for the BTC/USD exchange rate. We first applied the identify_rejection function to mark rejection candles in both bullish and bearish directions. We then used the isPivot, get_support_resistance_levels, and clean_support_resistance functions to detect and refine key levels. This process marked both the rejection candles and the support and resistance levels in our data.

The next step involves filtering and retaining only those rejection candles that occur near these key support and resistance levels. We also ensure that the surrounding candles

support the rejection pattern's significance. For a bullish rejection, the preceding candles should be above the support level, indicating that the price is approaching and being rejected by the support. Conversely, for a bearish rejection, the preceding candles should be below the resistance level, showing that the price is being rejected by the resistance.

These conditions are tested in a function named check_candle_signal. This function takes a DataFrame (df), the index of the current candle (l), a windowbackCandles parameter that specifies the number of previous candles to consider, a pivotwindow parameter for avoiding lookahead bias, and a distance_threshold_coef, which is used to determine how close the candle needs to be to a key level to be considered significant.

```
def check_candle_signal(df, candle_index,
                        backcandles_window, pivot_window,
                        distance_threshold_coef=0.003):
    """
    Determines if a candle is a valid rejection signal near support or
resistance levels.

    Parameters:
    df (pd.DataFrame): The DataFrame containing the trading data.
    candle_index (int): The index of the current candle being
analyzed.
    backcandles_window (int): The number of previous candles to
                              consider for detecting support and resistance
levels.
    pivot_window (int): The window size for detecting pivots,
                        used to avoid lookahead bias.
    distance_threshold_coef (float): The coefficient to determine the
proximity
                                     threshold for a candle to be considered
near a key level.

    Returns:
```

```
    int: 1 for a bearish rejection, 2 for a bullish rejection, 0 for no
valid signal.
    """
    if candle_index < backcandles_window + pivot_window:
        return 0

    # Get and clean support/resistance levels from previous candles
    support_resistance_levels = get_support_resistance_levels(df,
                                                start_row=candle_index
-backcandles_window,
                                                end_row=candle_index-
pivot_window)

    resistance_levels, support_levels =
clean_support_resistance(support_resistance_levels,
                                                  threshold_percent=1)

    # Flatten support and resistance levels
    key_levels = [x[1] for x in resistance_levels + support_levels]

    # Determine proximity threshold based on the current closing
price
    close_distance_threshold = df.Close[candle_index] *
distance_threshold_coef

    # Check if the current candle is near any resistance or support
levels
    close_to_resistance = closeResistance(candle_index,
                                  key_levels,
                                  close_distance_threshold,
                                  df)
    close_to_support = closeSupport(candle_index,
                                key_levels,
                                close_distance_threshold,
                                df)

    # Evaluate if the candle is a valid rejection signal
    if (df.rejection[candle_index] == 1 and
```

```
        close_to_resistance and
        is_below_resistance(candle_index, pivot_window,
close_to_resistance, df) ):
        return 1  # Bearish rejection
    elif (df.rejection[candle_index] == 2 and
          close_to_support and
          is_above_support(candle_index, pivot_window,
close_to_support, df) ):
        return 2  # Bullish rejection
    else:
        return 0  # No valid signal
```

The function leverages additional helper functions to determine whether a candle is near a resistance or support level, as well as to check if candles are positioned above or below key levels. These supporting functions are straightforward and are provided in the code snippet below. closeResistance and closeSupport functions take 4 input parameters the index of the candle being tested a list of key levels to be included in the test a limit parameter which is basically the distance threshold within which a candle is considered close enough to a key level and the dataframe where the data is loaded. Both functions return 0 if a candle is far from all key levels and they return the level price in the opposite case where the conditions are true and the candle is close to a key level that's provided in the levels list.

```
def closeResistance(l, levels, lim, df):
    if not levels:
        return 0
    nearest = min(levels, key=lambda x: abs(x - df['High'][l]))
    if (abs(df['High'][l] - nearest) <= lim or abs(max(df['Open'][l],
df['Close'][l]) - nearest) <= lim) \
            and min(df['Open'][l], df['Close'][l]) < nearest \
            and df['Low'][l] < nearest:
        return nearest
    return 0
```

```
def closeSupport(l, levels, lim, df):
    if not levels:
        return 0
    nearest = min(levels, key=lambda x: abs(x - df['Low'][l]))
    if (abs(df['Low'][l] - nearest) <= lim or abs(min(df['Open'][l],
df['Close'][l]) - nearest) <= lim) \
            and max(df['Open'][l], df['Close'][l]) > nearest \
            and df['High'][l] > nearest:
        return nearest
    return 0
```

is_below_resistance and is_above_support are also two functions that were used to test whether a candle and its previous neighbors within a backCandles window are all simultaneously below a resistance level or above a support level that are provided as an input parameter.

The functions is_below_resistance and is_above_support are used to check whether a candle and its preceding neighbors within a specified backCandles window are consistently positioned below a resistance level or above a support level, which are provided as input parameters.

```
def is_below_resistance(l, backCandles, level, df):
    return df.loc[l-backCandles:l-1, 'High'].max() < level

def is_above_support(l, backCandles, level, df):
    return df.loc[l-backCandles:l-1, 'Low'].min() > level
```

By applying these functions, we can generate trading signals based on the criteria we've outlined. These signals can be visualized on a chart, as shown in the sample below, where the purple triangles indicate trading signals aligned with the direction of the trend. While some signals may not be perfectly accurate, the final signal demonstrates an excellent entry point that could potentially offset three to four losing trades. This is particularly true if a trailing stop management approach is used, allowing the trade to capture the strong upward movement that follows the

entry signal.



3 The Backtesting.Py Package

When developing a new trading strategy using the backtesting.py package, it's essential to extend the Strategy class and override two key methods: init() and next(). These methods are the backbone of your strategy, controlling both the preparation of your trading signals and the logic of how trades are executed as the market data progresses.

The init() method is invoked before your strategy starts running. This is where you should set up and precompute all the indicators, signals, and variables that your strategy will rely on during the backtest. The goal here is to do as much pre-processing as possible in a vectorized manner. Vectorization means operating on whole arrays (or columns in the DataFrame) rather than looping through individual elements, which makes the computation faster and more efficient.

For example, if your strategy uses moving averages,

Bollinger Bands, or other technical indicators, you would calculate these within the init() method. This ensures that the indicators are ready for use when the strategy begins iterating through the market data. But in our example we will precompute any indicators before the backtesting part so all our indicators are already within the dataframe.

The next() method is called once for each data point in your dataset (each row in the DataFrame). This method simulates the incremental availability of each new candlestick bar, allowing your strategy to react to new market data as it becomes available.

Within next(), you define the core logic of your trading strategy. This includes making decisions on whether to enter or exit trades based on the indicators and signals you've set up in init(). For example, you might check if a short-term moving average crosses above a long-term moving average to signal a buy, or if certain candlestick patterns suggest a trend reversal. And if you already computed these trading signals in the dataframe you can simply access this information to trigger trades instead of repeating the computation during the backtest.

One important thing to note about backtesting.py is that it does not allow for trade decisions within a single candlestick. All decisions made in next() are executed on the next candlestick's open. If you wish to execute trades at the close of the current candlestick, you can set trade_on_close=True, which will execute the trade at the current candlestick's closing price.

If your strategy requires intraday trading or decisions based on price movements within a single candlestick (such as day trading), you need to use more granular data, such as hourly or minute-level data, rather than daily candlesticks. This allows your strategy to make more frequent decisions, aligning with the faster pace of intraday trading.

4 Understanding Backtesting Reports

A typical backtesting report provides a comprehensive overview of the performance of a trading strategy over a specified period. Let's breakdown the main elements provided by the Backtesting.py report:

- **Start & End**: These indicate the time of the backtest, showing when the backtesting period began and ended.
- **Duration**: This is the total length of time covered by the backtest, expressed in number of days.
- **Exposure Time [%]**: The percentage of the total time during which the strategy had open positions. A lower percentage indicates that the strategy spent more time out of the market which also indicates that we are using an indicator that's relatively selective and not very sensitive to different market patterns. In scalping strategies typically this number would be higher than slow paced strategies except when we are dealing with investment strategies where trades are opened for months.
- **Equity Final [$]**: The final value of the trading account at the end of the backtest, expressed in dollars.
- **Equity Peak [$]**: The highest value the trading account reached during the backtest.
- **Return [%]**: The total percentage return of the strategy over the backtesting period, and this is the parameter we usually tend to work on to improve because it's the reason we would do trading in the first place.

- **Buy & Hold Return [%]**: The return you would have earned by simply buying the asset at the start of the period and holding it until the end, expressed as a percentage. This serves as a benchmark to compare the strategy's performance against a simple buy-and-hold approach. Usually in increasing stocks this is a good comparison benchmark but in some other cases where the overall trend of the asset is a downtrend then the buy and hold return would be in the negative and it wouldn't make sense to compare any strategy to it because simply any strategy would beat a losing strategy.
- **Return (Ann.) [%]**: The annualized return of the strategy, which shows what the return would be if the strategy's performance continued over a full year.
- **Volatility (Ann.) [%]**: The annualized volatility, representing the strategy's risk by measuring the degree of variation in returns. It's usually a good rule of thumb to have a low volatility percentage because it means we have a lower risk when it comes to the equity rate of change.
- **Sharpe Ratio**: A measure of risk-adjusted return, indicating how much excess return the strategy generated for each unit of risk. A higher Sharpe Ratio suggests a better risk-adjusted return.
- **Sortino Ratio**: Similar to the Sharpe Ratio, but it only considers downside volatility, focusing on the risk of negative returns. A higher Sortino Ratio indicates better downside risk management.
- **Calmar Ratio**: The ratio of annualized return to maximum drawdown, showing how well the strategy performs relative to the largest equity decline. A higher Calmar Ratio indicates better performance relative to risk.

- **Max. Drawdown [%]**: The largest peak-to-trough decline in the equity curve during the backtest, expressed as a percentage. It indicates the maximum potential loss an investor could have experienced. This is also one of the most crucial parameters to look for in the back test report we need to limit the maximum drawdown to a very narrow percentage which also means limiting the risk. In general, if you increase the risk which includes also increasing the leverage on your account and the size of the trading positions you would see an increase in the maximum drawdown percentage, which will also be reflected on the Sharpe ratio that will decrease with increasing risk.
- **Avg. Drawdown [%]**: The average percentage decline during periods when the equity curve fell from a peak. This parameter is also related to the maximum drawdown, and we are also looking for strategies limiting the average drawdown to a very low percentage.
- **Max. Drawdown Duration**: The longest period during which the equity curve was in a drawdown, from peak to recovery. This is basically the toughest. That your strategy went through it was in a drawdown And you need to imagine the cytology you would be in if this duration is for example three or four months. In reality we're also looking to limit this duration to few days or couple of weeks.
- **Avg. Drawdown Duration**: The average length of time the equity curve spent in drawdown periods.
- **# Trades**: The total number of trades executed by the strategy during the backtest. This is also one of the important numbers to look add because if you have the perfect strategy running for a very low number of trades you can't confirm the robustness of

this strategy. To have a more confident results we need to have a high number of traits I would say at least in the hundreds, but this number depends on the statistics and the precision of the results, to be on the safe side we're looking for a minimum of a hundreds of trades.

- **Win Rate [%]**: The percentage of trades that were profitable. A higher win rate indicates a greater proportion of successful trades. Obviously in this case we're looking for a high win rate percentage but this also need to consider the reward to risk ratio that we are using if we're using a ratio of two for example a win rate above 34% is already in the profitable zone on the other hand if we're using I want to risk ratio that is equal to 1 then we need to increase our win rate above 50% and so on.
- **Best Trade [%]**: The percentage gain of the most profitable trade.
- **Worst Trade [%]**: The percentage loss of the least profitable trade. If the worst strike percentage is excessive, we need to investigate this particular trade and to analyze the situation to see if there is anything that we have missed in the trading strategy and if we can avoid this particular situation where we have one trade that's increasing our drawdown by a high value.
- **Avg. Trade [%]**: The average percentage gain or loss per trade, providing an overall sense of trade performance.
- **Max. Trade Duration**: The longest time a trade was open during the backtest. You might want to keep an eye on this one if you are trading with a broker that charges overnight fees and over weekend fees for open trades.

- **Avg. Trade Duration**: The average time that trades were held open.
- **Profit Factor**: The ratio of gross profits to gross losses. A profit factor greater than 1 indicates that the strategy is profitable overall.
- **Expectancy [%]**: The average percentage gain or loss per trade, considering both winning and losing trades. A positive expectancy indicates that the strategy is expected to be profitable over time.
- **SQN (System Quality Number)**: A measure of the strategy's overall performance, considering the number of trades, average trade size, and standard deviation of returns. A higher SQN indicates a more robust trading system.
- **_strategy**: The name of the strategy being tested.
- **_equity_curve**: The full equity curve data showing the account balance over time, though it is often truncated in the summary.
- **_trades**: A detailed record of all the trades made during the backtest, including entry and exit points, trade size, and profit/loss, though this is also typically summarized.

This report provides a comprehensive view of how the strategy performed during the backtesting period, highlighting key metrics related to profitability, risk, and trading efficiency. By analyzing these metrics, traders can assess the viability of the strategy and identify areas for potential improvement.

5 Backtest Quality Ratios: Sharpe, Sortino, and Calmar

In algorithmic trading, creating a strategy is just the beginning. The real test lies in evaluating its performance, ensuring it's not just profitable, but also resilient under various market conditions. To achieve this, traders rely on backtest quality ratios, which provide insights into the risk-adjusted returns of a strategy. Among the most crucial of these are the Sharpe Ratio, Sortino Ratio, and Calmar Ratio. Each of these ratios serves to measure the effectiveness of a trading strategy by balancing returns against risk, but they do so in slightly different ways, providing a comprehensive view when used together.

5.1 Sharpe Ratio

The Sharpe Ratio is one of the most widely used metrics for evaluating the performance of an investment or trading strategy. It measures how much excess return you receive for the extra volatility you endure for holding a riskier asset. The ratio is especially useful in comparing the risk-adjusted performance of different strategies.

The formula for the Sharpe Ratio is:

Sharpe Ratio= (portfolio returns - Risk-free rate) / Standard deviation of returns.

The portfolio returns is the average return of the strategy during the backtest.

The risk-free rate in backtesting.py is often assumed to be zero unless specified otherwise. If you want to adjust for a non-zero risk-free rate, you will need to subtract this rate from the returns before calculating the Sharpe Ratio. Typically, we could use the return on U.S. Treasury Bills as a risk-free benchmark,

otherwise we can just leave it at 0% which is also kind of risk-free.

The standard deviation is of the portfolio returns representing the volatility of the strategy.

The Sharpe Ratio essentially tells us how much additional return a trader can expect to receive for the additional volatility (risk) they are exposed to by the strategy. A higher Sharpe Ratio indicates a more favorable risk-adjusted return. However, it's important to note that the Sharpe Ratio assumes returns are normally distributed, an assumption that may not always hold true in real-world trading.

5.2 Sortino Ratio

The Sortino Ratio builds on the Sharpe Ratio by refining the way risk is measured. Instead of using total volatility (which includes both upside and downside movements), the Sortino Ratio focuses solely on downside risk, penalizing only those returns that fall below a certain threshold, typically the risk-free rate or zero.

The formula for the Sortino Ratio is:

Sortino Ratio = (portfolio returns - Risk-free rate) / Standard deviation of returns.

The numerator elements are the same as for the Sharpe ratio, the standard deviation of returns is only about downside movements, so it only considers negative deviations from the mean or another target return.

By considering only downside risk, the Sortino Ratio provides a more focused measure of risk-adjusted performance, especially useful in scenarios where we are primarily concerned with protecting against significant losses. It's particularly favored by traders who believe that upside volatility (gains) should not be penalized in the same way as downside volatility (losses).

5.3 Calmar Ratio

The Calmar Ratio is another critical metric, particularly in evaluating strategies over longer time periods. It measures the return of an investment compared to its maximum drawdown, offering insight into how well a strategy balances returns with risk over time.

The formula for the Calmar Ratio is:

Calmar Ratio = Annual returns / Maximum drawdown

Just to be clear the maximum drawdown is the largest percentage drop from a peak to a trough in the portfolio's value.

The Calmar Ratio is especially sensitive to drawdowns, making it a valuable tool for strategies that aim to minimize significant losses. A higher Calmar Ratio suggests that the strategy not only generates strong returns but also does so with relatively smaller drawdowns, which is crucial for long-term capital preservation and growth.

5.4 Understanding the Metrics Together

While each ratio serves its purpose, using them in conjunction provides a more nuanced understanding of a trading strategy's performance. The following discussion is a brief overview around the typical values of these ratios and their general interpretations:

Sharpe Ratio is useful for general comparisons but might be misleading in volatile markets where returns are not normally distributed.

- **Bad: < 0.5** A Sharpe Ratio below 0.5 is generally considered poor. It indicates that the strategy's returns are not sufficiently compensating for the risk taken.

- **Acceptable: 0.5 - 1.0** A Sharpe Ratio between 0.5 and 1.0 is considered acceptable. It suggests the strategy is moderately compensating for the risk, but improvements could be made.
- **Good: 1.0 - 2.0** A Sharpe Ratio between 1.0 and 2.0 is considered good. It indicates that the strategy is generating good risk-adjusted returns.
- **Excellent: > 2.0** A Sharpe Ratio above 2.0 is excellent. It suggests the strategy is providing exceptional risk-adjusted returns.

Sortino Ratio offers a better view when your primary concern is downside risk, providing a more realistic assessment of the strategy's performance.

- **Bad: < 1.0** A Sortino Ratio below 1.0 is typically seen as poor. It suggests that the strategy is not adequately compensating for downside risk.
- **Acceptable: 1.0 - 2.0** A Sortino Ratio between 1.0 and 2.0 is acceptable, indicating that the strategy is providing reasonable returns relative to the downside risk.
- **Good: 2.0 - 3.0** A Sortino Ratio between 2.0 and 3.0 is considered good. It indicates that the strategy is effectively managing downside risk while generating returns.
- **Excellent: > 3.0** A Sortino Ratio above 3.0 is excellent, indicating that the strategy is providing strong returns with minimal downside risk.

Calmar Ratio helps in understanding how returns stack up against significant losses, which is especially important for strategies aiming for long-term success.

- **Bad: < 0.5** A Calmar Ratio below 0.5 is typically

seen as poor, indicating that the strategy's returns are not sufficient to justify the level of drawdown risk.

- **Acceptable: 0.5 - 1.0** A Calmar Ratio between 0.5 and 1.0 is considered acceptable, suggesting that the strategy's returns are reasonably balanced against drawdown risk.
- **Good: 1.0 - 2.0** A Calmar Ratio between 1.0 and 2.0 is good, indicating that the strategy is generating strong returns relative to the risk of drawdown.
- **Excellent: > 2.0** A Calmar Ratio above 2.0 is excellent, suggesting that the strategy provides high returns with relatively low drawdown risk.

These ranges offer a general framework for interpreting quality ratios in the context of algorithmic trading strategies. They assist in evaluating whether a strategy is underperforming, acceptable, good, or exceptional in terms of risk-adjusted returns. However, these values are just a starting point. As a trader develops and refines a strategy, they will gain a deeper understanding of which values are most relevant and how to interpret them based on the specific characteristics of the strategy they are using.

6 Case Study: Backtesting With Fixed Stop Loss And Take Profit Values

Using the backtesting.py package, we can define a trade management system that integrates with the position entry signals we've already generated and saved as a new column in

our DataFrame. We can then backtest this strategy over 1,000 daily candles.

In this section, we will use fixed stop-loss and take-profit distances. The stop-loss distance will be calculated as a percentage of the last closing price, for example, 10%. For a long position, the stop-loss level would be set at the current closing price minus the calculated stop-loss distance. The take-profit distance will be set as twice the stop-loss distance, based on a take-profit-to-stop-loss ratio of 2. The code snippet below demonstrates how to run a simple backtest using these fixed stop-loss and take-profit values for both long and short positions.

To enable the backtesting.py package to calculate and measure key performance metrics like the Sharpe, Sortino, and Calmar ratios, we need to provide a datetime index. Since our DataFrame already contains a date column, we can easily set this column as the index with the following line of code:

```
df.set_index(["Date"], inplace=True, drop=True)
```

And the backtest can be carried out using the following code:

```
from backtesting import Strategy, Backtest

def SIGNAL():
    return df.signal

class MyCandlesStrat(Strategy):
    def init(self):
        super().init()
        self.signal1 = self.I(SIGNAL)
        self.ratio = 2
        self.risk_perc = 0.1

    def next(self):
        super().next()
        if self.signal1==2:
            sl1 = self.data.Close[-1] - self.data.Close[-1]*self.risk_perc
```

```
            tp1 = self.data.Close[-1] +
(self.data.Close[-1]*self.risk_perc)*self.ratio
            self.buy(sl=sl1, tp=tp1)
        elif self.signal1==1:
            sl1 = self.data.Close[-1] +
self.data.Close[-1]*self.risk_perc
            tp1 = self.data.Close[-1] -
(self.data.Close[-1]*self.risk_perc)*self.ratio
            self.sell(sl=sl1, tp=tp1)
bt = Backtest(df, MyCandlesStrat, cash=100000, margin=1/2,
commission=0.02)
stat = bt.run()
```

First, we begin by importing the Strategy and Backtest modules from the backtesting package. Then, we define a signal function that returns the signal column from our DataFrame. Next, we create a class for our strategy, inheriting from the base Strategy class. In the constructor, we initialize instance variables for the signal, the risk-reward ratio, and the risk percentage coefficient.

In the following method, we define the stop-loss levels for both bullish and bearish signals, triggered when the signal variable equals 2 or 1, respectively. We also calculate the take-profit level using the closing price of the last candle, the stop-loss distance, and the take-profit-to-stop-loss ratio.

We then use the buy and sell methods to execute trades, passing the calculated stop-loss and take-profit levels. Since we don't specify a trade size, the strategy assumes trading with the entire equity, effectively flipping the entire account from one currency to another. This approach is acceptable as long as leverage is not increased, as higher leverage would significantly raise the strategy's risk.

Finally, we call the Backtest function, providing the DataFrame, the strategy class, the initial cash amount for the account, the margin for leverage (set at 2:1 in this example),

and a commission rate of 2%. We then run the backtest, storing the results in a variable named stat. The results of this backtest, which can be printed from the stat variable, are displayed in the following table.

Start	2021-11-23 00:00:00
End	2024-08-18 00:00:00
Duration	999 days 00:00:00
Exposure Time [%]	21.1
Equity Final [$]	238849.038555
Equity Peak [$]	238849.038555
Return [%]	138.849039
Buy & Hold Return [%]	4.597459
Return (Ann.) [%]	37.408968
Volatility (Ann.) [%]	49.950064
Sharpe Ratio	0.748927
Sortino Ratio	1.653478
Calmar Ratio	1.043231
Max. Drawdown [%]	-35.858771
Avg. Drawdown [%]	-9.499003
Max. Drawdown Duration	228 days 00:00:00
Avg. Drawdown Duration	31 days 00:00:00
# Trades	9
Win Rate [%]	77.777778
Best Trade [%]	18.378618
Worst Trade [%]	-11.768164
Avg. Trade [%]	7.914585
Max. Trade Duration	51 days 00:00:00
Avg. Trade Duration	29 days 00:00:00
Profit Factor	4.30169

Let's review the key results from the backtest. The first three lines summarize the duration of the test in days, along with the starting and ending dates. Following this, we see the final equity, the equity peak, and the overall return percentage, which impressively reached 138%, compared to just 4.59% for a simple buy-and-hold strategy. The maximum drawdown percentage of -35% and the average drawdown of around

-9.49%. The win rate is also strong at approximately 77%, although the total number of trades is just 9, which raises some concerns.

While these figures suggest a promising strategy, it's important to recognize that 9 trades are not statistically significant enough to confidently evaluate the strategy's robustness. The low number of signals indicates that our indicator is highly selective and not very sensitive to market fluctuations. To improve the reliability of our results, we might consider extending the backtest period by incorporating more historical data or applying the same strategy across multiple assets. Testing on, say, 30 different assets could generate at least 300 trades, providing a more solid foundation for drawing meaningful conclusions.

7 Case Study: Backtesting With ATR Related Stop Loss Levels

In this section we will be using the average true range to compute the stop loss distance for each trade. This way we can compute the stop loss and take profit distances and levels with respect to the volatility of the market at the moment of opening the position. And to do this we need to compute the ATR first and to save it as a new column in the data frame. This can be easily done using pandas_ta package and the atr() function:

```
df['ATR'] = ta.atr(high=df.High, low=df.Low, close=df.Close, length=14)
```

Now we can run the back test using the following code:

```
from backtesting import Strategy, Backtest
import pandas_ta as ta
```

```
df['ATR'] = ta.atr(high=df.High, low=df.Low, close=df.Close,
length=14)

def SIGNAL():
    return df.signal

class MyCandlesStrat(Strategy):
    atr_f = 2.5
    ratio_f = 1.5
    def init(self):
        super().init()
        self.signal1 = self.I(SIGNAL)

    def next(self):
        super().next()
        if self.signal1==2:
            sl1 = self.data.Close[-1] - self.data.ATR[-1]*self.atr_f
            tp1 = self.data.Close[-1] +
self.data.ATR[-1]*self.ratio_f*self.atr_f
            self.buy(sl=sl1, tp=tp1)
        elif self.signal1==1:
            sl1 = self.data.Close[-1] + self.data.ATR[-1]*self.atr_f
            tp1 = self.data.Close[-1] -
self.data.ATR[-1]*self.ratio_f*self.atr_f
            self.sell(sl=sl1, tp=tp1)
bt = Backtest(df, MyCandlesStrat, cash=100000, margin=1/2,
commission=0.02)
stat = bt.run()
```

Notice that we've utilized the atr_f coefficient to multiply the ATR (Average True Range) value, which determines the stop loss distance. This means the stop loss level is set at a distance equal to the ATR value multiplied by atr_f, measured from the closing price of the candle when the trade is executed. Additionally, we apply another coefficient, ratio_f, which is the take profit to stop loss ratio.

Using these coefficients, we calculate the stop loss and take

profit levels for both bullish and bearish signals. The trade positions are then executed with the buy and sell functions for long and short positions, respectively.

We conducted the backtest with a starting cash amount of $100,000, a leverage margin of 1:2, and a commission rate of 2%. The results of this backtest are summarized in the table below:

Start	2021-11-23 00:00:00
End	2024-08-18 00:00:00
Duration	999 days 00:00:00
Exposure Time [%]	19.0
Equity Final [$]	229432.120141
Equity Peak [$]	229432.120141
Return [%]	129.43212
Buy & Hold Return [%]	4.597459
Return (Ann.) [%]	35.406275
Volatility (Ann.) [%]	47.310392
Sharpe Ratio	0.748383
Sortino Ratio	1.587165
Calmar Ratio	1.082884
Max. Drawdown [%]	-32.696291
Avg. Drawdown [%]	-7.898617
Max. Drawdown Duration	285 days 00:00:00
Avg. Drawdown Duration	29 days 00:00:00
# Trades	7
Win Rate [%]	85.714286
Best Trade [%]	15.033419
Worst Trade [%]	-9.943886
Avg. Trade [%]	7.122757
Max. Trade Duration	44 days 00:00:00
Avg. Trade Duration	27 days 00:00:00
Profit Factor	6.230216

The returns in this backtest are impressive, showing a total return of 129% with an annual return of 35.4%. However, there is a maximum drawdown of -32% and an average drawdown of

-7.8%. The strategy demonstrates a solid win rate of 85%, but it's important to note the low number of trades, which is just 7.

The reason for the reduced number of trades, despite using the same data and signals as in the previous section, is that some trades remain open for an extended period. When this happens, the next signal is missed because the strategy is designed to utilize the entire $100,000 account balance to either buy or sell. Given the price of Bitcoin in the dataset, this approach only allows for one trade at a time, leaving no available funds to open a second trade while the first is still active. In a different scenario let's suppose we have enough funds in our account to open multiple trades and we want to avoid this situation meaning we only want to allow one trade at a time we can modify the back testing code including the following snippet:

```
if self.signal1==2 and len(self.trades)==0:
            sl1 = self.data.Close[-1] - self.data.Close[-1]*self.risk_perc
            tp1 = self.data.Close[-1] +
(self.data.Close[-1]*self.risk_perc)*self.ratio
            self.buy(sl=sl1, tp=tp1)
        elif self.signal1==1 and len(self.trades)==0:
            sl1 = self.data.Close[-1] +
self.data.Close[-1]*self.risk_perc
            tp1 = self.data.Close[-1] -
(self.data.Close[-1]*self.risk_perc)*self.ratio
            self.sell(sl=sl1, tp=tp1)
```

Notice how we added a condition on top of the signal direction where the length of the list of trades needs to be 0 to allow for a new trade.

8 Case Study: Backtesting Using A Trailing Stop

A trailing stop functions as a dynamic stop loss that adjusts

with the price, effectively "chasing" the price within a specified distance set by the trader. If the price moves in a favorable direction, the trailing stop follows, maintaining its set distance. However, if the price reverses direction, the trailing stop remains fixed. If the price touches this level, the trade is closed, and depending on the closing position, the trade can be classified as either a win or a loss.

The key advantage of using a trailing stop is its ability to capitalize on potential wide-range price movements. Markets can sometimes experience rapid shifts, either climbing sharply or dropping significantly. A trailing stop allows traders to maximize profits during these significant price moves by locking in gains as the price moves in the trade's favor.

In the backtesting.py package, a trailing stop is not a built-in function, but it can be implemented using custom Python code within the backtesting framework. The following example demonstrates how to code a trailing stop as part of a backtesting strategy in Python:

```
class MyCandlesStrat(Strategy):
    def init(self):
        super().init()
        self.signal1 = self.I(SIGNAL)

    def next(self):
        super().next()
        sltr=self.data.Close[-1]*0.02

        for trade in self.trades:
            if trade.is_long:
                trade.sl = max(trade.sl or -np.inf, self.data.Close[-1] -
sltr)
            else:
                trade.sl = min(trade.sl or np.inf, self.data.Close[-1] +
sltr)

        if self.signal1==2 and len(self.trades)==0:
```

```
            sl1 = self.data.Close[-1] - sltr
            self.buy(sl=sl1)
        elif self.signal1==1 and len(self.trades)==0:
            sl1 = self.data.Close[-1] + sltr
            self.sell(sl=sl1)
```

In this code snippet, the trailing stop mechanism is implemented within the MyCandlesStrat class, which extends the Strategy class. The key part of this mechanism is how the stop loss (sl) is adjusted for each active trade during the strategy's execution. Let's break down the relevant section:

```
sltr=self.data.Close[-1]*0.02
```

Here, sltr is calculated as 2% of the last closing price (self.data.Close[-1]). This value represents the distance by which the stop loss should trail behind the current price. The 2% value is a placeholder and can be adjusted depending on the trader's preference.

```
for trade in self.trades:
            if trade.is_long:
                trade.sl = max(trade.sl or -np.inf, self.data.Close[-1] -
sltr)
            else:
                trade.sl = min(trade.sl or np.inf, self.data.Close[-1] +
sltr)
```

This loop iterates over all active trades (self.trades).

For Long Positions (trade.is_long):

The stop loss (trade.sl) is updated to the greater of its current value or the new trailing stop level (self.data.Close[-1] - sltr). This ensures that the stop loss only moves upward (trailing the price) but never downward.

For Short Positions:

The stop loss is updated to the lesser of its current value or the new trailing stop level (self.data.Close[-1] + sltr). This

ensures that the stop loss only moves downward (trailing the price) but never upward.

The max and min functions ensure that the stop loss is adjusted in the correct direction, depending on whether the trade is long or short, without violating the trailing principle.

The results obtained using the training stop with the same trading signal and using exactly the same conditions for back testing as the previous sections are shown in the following table:

Start	2021-11-23 00:00:00
End	2024-08-18 00:00:00
Duration	999 days 00:00:00
Exposure Time [%]	6.2
Equity Final [$]	107422.106914
Equity Peak [$]	140981.750078
Return [%]	7.422107
Buy & Hold Return [%]	4.597459
Return (Ann.) [%]	2.647692
Volatility (Ann.) [%]	16.568704
Sharpe Ratio	0.159801
Sortino Ratio	0.283481
Calmar Ratio	0.085782
Max. Drawdown [%]	-30.865224
Avg. Drawdown [%]	-12.273957
Max. Drawdown Duration	448 days 00:00:00
Avg. Drawdown Duration	229 days 00:00:00
# Trades	9
Win Rate [%]	33.333333
Best Trade [%]	22.499703
Worst Trade [%]	-4.086295
Avg. Trade [%]	0.785366
Max. Trade Duration	18 days 00:00:00
Avg. Trade Duration	6 days 00:00:00
Profit Factor	1.510092

We can see that the returns are now less than the previous trade management approaches we have a maximum drawdown of -30% and an average drawdown of -12% and our win rate

dropped to 33% also the quality ratios dropped in comparison to classic stop loss take profit trading style.

9 Case Study: Break Even Dynamic Management Approach

This approach represents a more sophisticated trade management technique. Here's how it works: we initiate a trade with predefined stop loss and take profit levels. Once the trade moves in our favor by at least half the take profit distance, we close half of the position to secure some profits. At the same time, we adjust the stop loss to the entry level, effectively eliminating the risk on the remaining half of the trade. If the price continues to move in our favor, the remaining half of the trade will close at the full take profit level, maximizing our gains. However, if the price reverses, the trade will close at breakeven, ensuring no loss on the second half of the position. This dynamic, break-even approach reduces risk while offering the potential for further gains. Although it may slightly reduce overall profits in some cases, it adds robustness to our trading system, making it a valuable option to consider.

The following backtesting class was modified to include this type of trade management:

```
class MyStrat(Strategy):
    def init(self):
        super().init()
        self.signal = self.I(SIGNAL)
        self.mysize = 0.5

    def next(self):
        super().next()
        TPSLRatio = 2
        perc = 0.1
```

```
        if len(self.trades) == 1:
            self.trades[-1].sl = self.trades[-1].entry_price

        if self.signal == 2 and len(self.trades) == 0:
            sl1 = self.data.Close[-1] - self.data.Close[-1] * perc
            sldiff = abs(sl1 - self.data.Close[-1])
            tp1 = self.data.Close[-1] + sldiff * TPSLRatio
            tp2 = self.data.Close[-1] + sldiff
            self.buy(sl=sl1, tp=tp1, size=self.mysize)
            self.buy(sl=sl1, tp=tp2, size=self.mysize)

        elif self.signal == 1 and len(self.trades) == 0:
            sl1 = self.data.Close[-1] + self.data.Close[-1] * perc
            sldiff = abs(sl1 - self.data.Close[-1])
            tp1 = self.data.Close[-1] - sldiff * TPSLRatio
            tp2 = self.data.Close[-1] - sldiff
            self.sell(sl=sl1, tp=tp1, size=self.mysize)
            self.sell(sl=sl1, tp=tp1, size=self.mysize)

bt = Backtest(df, MyStrat, cash=100000, margin=0.5,
commission=0.02)
stat = bt.run()
```

Here are some key points to consider in this trading class:

First, it's crucial to define the trade size, as we're splitting the trade into two parts. We can't use the entire account balance for the initial trade, as there wouldn't be any funds left for the second half. This is why we set the mysize factor to 0.5, meaning each trade will use 50% of the current equity.

In the next() function, we define a single stop loss value, calculated as a certain percentage of the entry position (i.e., the current closing price). We then establish two take profit levels: one for the full take profit, using a stop-loss-to-take-profit ratio (set to 2 in this example), and another at the midpoint between the entry and full take profit levels. This gives us two take profit levels, TP1 and TP2.

Next, we place two long trades using the buy() function twice, with the only difference between these two trades being their take profit levels. As a result, one position will close when it reaches the midpoint, while the other remains open, aiming for the full take profit target.

Although we're effectively opening two trades (each at half the size), it's conceptually similar to opening one trade and closing half of it at the midpoint. However, once the first trade (the closer take profit level) is closed, we need to adjust the stop loss of the remaining trade to the entry level. This ensures that the remaining trade is protected from any reversal, securing a breakeven outcome if the market moves against us.

This stop loss adjustment is triggered when only one trade remains open, indicating that the first trade has already reached its take profit target. The stop loss update is managed using the following snippet:

```
if len(self.trades) == 1:
        self.trades[-1].sl = self.trades[-1].entry_price
```

We can observe that when the number of open trades is reduced to 1, it triggers an adjustment where the stop loss of the remaining trade is moved to its entry price. This ensures that the position is protected from any potential losses if the market reverses. I ran a backtest using this approach, and the results are displayed in the following table:

Start	2021-11-23 00:00:00
End	2024-08-18 00:00:00
Duration	999 days 00:00:00
Exposure Time [%]	21.0
Equity Final [$]	220336.035898
Equity Peak [$]	220336.035898
Return [%]	120.336036
Buy & Hold Return [%]	4.597459
Return (Ann.) [%]	33.421623
Volatility (Ann.) [%]	30.297312
Sharpe Ratio	1.103122
Sortino Ratio	2.610942
Calmar Ratio	1.604285
Max. Drawdown [%]	-20.832719
Avg. Drawdown [%]	-5.809206
Max. Drawdown Duration	98 days 00:00:00
Avg. Drawdown Duration	18 days 00:00:00
# Trades	14
Win Rate [%]	78.571429
Best Trade [%]	18.378618
Worst Trade [%]	-11.768164
Avg. Trade [%]	8.285185
Max. Trade Duration	51 days 00:00:00
Avg. Trade Duration	26 days 00:00:00
Profit Factor	6.225719

In this backtest, our strategy delivered a total return of 120% with an annualized return of 33%. The quality ratios are particularly strong: a Sharpe Ratio above 1 indicates solid risk-adjusted returns, a Sortino Ratio above 2 reflects good management of downside risk, and a Calmar Ratio above 1 suggests a favorable balance between return and drawdown. The maximum drawdown has been reduced to -20%, with an average drawdown of -5%, and a win rate of 78%. However, the number of trades remains low at just 14, which is consistent with our previous backtests due to the same underlying signal being employed.

This shift to a dynamic trade management approach has significantly enhanced the overall health and robustness of our

strategy. It's important to recognize, though, that this method might not be universally optimal. Its effectiveness can vary depending on the specific strategy, signal, market conditions, and asset being traded. This example highlights how creative trade management can significantly improve a strategy's robustness, making it more adaptable across various assets and ultimately leading to greater profitability.

Another idea worth exploring, inspired by this section, is to split the trade into three equal parts with different take-profit levels. You might close the first two portions when their respective take-profit targets are reached, while allowing the third portion to continue with a trailing stop. This approach enables you to capitalize on accurate predictions while maximizing profits on a portion of your position during wide-ranging market moves. While this book doesn't have the space to cover every possible idea, once you begin experimenting and see positive results, you'll often find that further fine-tuning can improve your strategy even more.

10 Visualizing The Backtest Data

The backtesting package also provides seamless visualization of equity, charts, trades, and related information. In Python, you can easily generate these visualizations by running the following line after completing the backtest.

```
bt.plot()
```

The visualization results are displayed in the figure below.



The top section of the figure illustrates the equity curve over time, highlighting key points in the trading performance. A red dot marks the point of maximum drawdown, and a red line indicates the duration of this drawdown. The equity peak is identified with a light blue point, while the final equity is shown with a dark blue point. The legend provides numerical details corresponding to these visual markers.

The second chart from the top displays the profit and loss percentages for each past trade. Red triangles indicate losing trades, and green triangles represent winning trades, with the size of the triangle proportional to the magnitude of the profit or loss.

The third chart presents the price candles alongside an overlay of executed trades, showing where trades were opened and closed. Winning trades are marked in green, while losing trades are in red, allowing for easy visual inspection of the algorithm's performance.

The fourth chart shows trading volume, offering additional context for the trades. Finally, the bottom chart visualizes the

signal used in the backtest, with peaks in the signal represented graphically, corresponding to the values of 0, 1, or 2 used in this example.

11 Conclusion

In this chapter, we've explored how the same technical indicators and entry signals can yield vastly different outcomes in terms of trading returns, volatility, drawdown, risk, and reward—all simply by altering the way trades are managed and exited. While the entry point is typically defined by technical indicators, the exit strategy is an equally critical component that must be carefully developed.

The various case studies presented highlight the significant role of trade management and demonstrate the creative potential in applying different tools and approaches. Trade management strategies don't necessarily have to come from algorithmic trading books or programming courses; they can be inspired by a wide range of sources, including online trading forums, courses, or books by experienced traders. These resources often offer valuable insights and strategies that, even if not applied exactly as presented, can inspire you to develop your own approach. Remember, no one understands your strategy better than you do. While others may offer guidance, it is ultimately your responsibility to refine your strategy and make it profitable.

Crafting effective trade management approaches is undoubtedly challenging, as there is no one-size-fits-all solution. The best approach is to gather as much information as possible, critically analyze it, and then tailor a strategy that fits your specific needs. This may involve merging multiple approaches or selecting elements from various strategies that work best together.

Before concluding, it's crucial to emphasize the importance

of robustness in any trading system. Robustness means testing your strategy across a sufficient number of trades to ensure that your results aren't merely due to chance. Testing over a small number of trades might lead to misleading results, where luck plays a larger role than skill. To ensure robustness, you should increase the number of trades by testing across multiple assets and extending the duration of your data. Ultimately, it's better to trade with a robust, low-risk strategy that yields steady, modest profits than to pursue a highly profitable but volatile strategy with greater risk.

In summary, "trading safe strategies is better than trading high-profit strategies." Algorithmic tools are incredibly powerful, allowing you to test virtually any strategy you can imagine with just a laptop and some data. Python and algorithmic modeling provide the means to backtest, fine-tune, and evaluate your ideas, helping you decide whether to put them into real-time trading or keep them on record for further refinement.

Chapter 10: Robustness Analysis Using the Monte-Carlo Method

1 Introduction

The Monte Carlo approach, at its core, involves the use of random numbers to solve problems or simulate processes. Imagine flipping a coin or rolling a dice; these are simple examples of Monte Carlo experiments, where the outcome is determined by chance. The method derives its name from the Monte Carlo Principality, renowned for its casinos and games of chance, where random outcomes are the very essence of

entertainment and gambling.

Monte Carlo methods have gained widespread application in various fields, ranging from physics and engineering to finance and economics. The underlying reason for their utility lies in their ability to model and simulate complex systems influenced by randomness. In the natural world, randomness is not just common; it's intrinsic to the behavior of many phenomena. Whether it's the diffusion of particles, the weather patterns, or the movement of stock prices, randomness plays a crucial role in determining outcomes.

In the context of science, Monte Carlo methods are frequently used to simulate natural phenomena. This is because randomness is an inherent aspect of the environment we live in. When scientists seek to understand or predict the behavior of complex systems, they often rely on Monte Carlo simulations to account for the inherent uncertainty and variability present in these systems. For instance, in physics, Monte Carlo simulations are used to model the behavior of particles at the quantum level, where uncertainty is a fundamental characteristic.

Finance, a field deeply intertwined with uncertainty and risk, has found Monte Carlo methods particularly valuable. The noise that drives price volatility, market reactions, and the movement of financial assets is, to a large extent, a product of random actions and decisions made by a myriad of market participants, ranging from large institutional investors to individual retail traders. These market dynamics can be seen as a complex web of random interactions that collectively shape the financial landscape.

Given the stochastic nature of financial markets, Monte Carlo methods provide a powerful tool for traders, analysts, and investors to model and understand potential future scenarios. Rather than relying solely on deterministic models, which may assume that markets behave in a predictable manner, Monte Carlo simulations allow for the incorporation of randomness,

enabling a more realistic assessment of potential outcomes. This is particularly important when testing trading strategies, as it helps account for the unpredictable and often volatile nature of markets.

In this chapter, we will delve into the application of Monte Carlo methods in algorithmic trading. We will explore how these techniques can be used to test trading strategies under a wide range of random market conditions. By simulating thousands or even millions of different scenarios, Monte Carlo methods allow traders to assess how a strategy might perform under various market conditions, including those that are rare or extreme.

One of the key advantages of using Monte Carlo simulations in trading is the ability to stress-test strategies. Traditional backtesting methods often rely on historical data, which may not fully capture the range of possible market conditions that could occur in the future. Monte Carlo simulations, on the other hand, can generate a wide variety of hypothetical market conditions, allowing traders to evaluate the robustness of their strategies across different scenarios. This can be particularly valuable in identifying potential weaknesses in a strategy before deploying it in live trading.

Moreover, Monte Carlo methods can help traders better understand the risk associated with their strategies. By analyzing the distribution of outcomes generated by the simulations, traders can estimate the probability of different levels of profit and loss, as well as the likelihood of extreme outcomes, such as significant drawdowns. This information can be used to refine risk management techniques and make more informed decisions about position sizing and leverage.

As we proceed through this chapter, we will provide detailed insights into how Monte Carlo methods can be implemented in the testing of trading strategies. We will discuss the steps involved in setting up Monte Carlo simulations, the types of random inputs that can be used, and how to interpret the results

to make more informed trading decisions.

2 Applying Normally Distributed Blur

In the context of data simulation and algorithmic trading, "blur" refers to the intentional introduction of randomness or variability to data values, typically within a specific range. This technique is particularly useful when testing the robustness of trading strategies, as it simulates the unpredictable nature of real-world financial markets. By applying a blur to data, we can observe how small variations in price or other market factors might impact the performance of a trading strategy, ensuring that the strategy is not overly sensitive to minor fluctuations.

The concept of blur is closely tied to the normal distribution, a fundamental statistical distribution that is widely observed in natural and financial phenomena. A normal distribution is symmetrical, with most values clustering around a central mean (average) and tapering off as they move further from the mean. The spread of values around the mean is measured by the standard deviation, often denoted by the Greek letter sigma (σ).

In practice, the blur effect is applied by generating new data points using a normal distribution centered around the original value, with the standard deviation controlling the extent of the blur. For example, if we blur a price value with a standard deviation of σ, about 68% of the new values will fall within one standard deviation ($\pm 1\sigma$) of the original value. If we extend this to two standard deviations ($\pm 2\sigma$), approximately 95% of the new values will be within this range. Finally, for three standard deviations ($\pm 3\sigma$), around 99.7% of the values will be within this broader range.

When applying blur to a specific data column, we typically calculate the standard deviation as a percentage of the original

value. This percentage determines the extent of the variability we introduce. For instance, if we use a blur percentage of 1%, and the original price is $100, the standard deviation might be $1. Consequently, when we generate new values using the normal distribution, they will generally fall within a range of $97 to $103 (±3σ).

The utility of applying a blur lies in its ability to simulate the random, noise-driven movements in market prices. By doing so, we can test whether a trading strategy can withstand these variations and still perform reliably. For instance, if a strategy consistently succeeds despite the introduced blur, it may be more resilient to market volatility. Conversely, if a strategy fails when even slight randomness is introduced, it might be overly sensitive and require adjustments to improve its robustness.

In this section, we introduce a method for adding randomness to a specific column in a DataFrame using a normal distribution blur. The function, named apply_blur, takes three input parameters: the DataFrame, the column name on which to apply the blur, and the percentage of blur to apply to each value.

The purpose of this blur is to simulate potential variations or noise that might be present in real-world data, helping to create a more robust testing environment for trading strategies. The blur percentage is crucial in determining the extent of randomness applied to each value.

Here's how the function works:

1. **Input Parameters**:
    - df: The DataFrame containing the data.
    - column_name: The name of the column in the DataFrame to which the blur will be applied.
    - blur_percent: The percentage used to calculate the blur. This percentage is applied to each value in the specified column to determine the range (max and

min values) within which the new value will be randomly selected.

2. **Process**:

   - For each value in the specified column, the function calculates the range of acceptable values. This range is determined by calculating 3 times the standard deviation of a normal distribution centered around the current value.
   - The maximum and minimum values of this range are computed based on the blur percentage applied to the current value.
   - A new value is then generated using a normal distribution, with the standard deviation calculated to ensure that the value lies within the defined range (max and min).

3. **Output**:

   - The function returns a new DataFrame with the specified column values blurred according to the normal distribution, introducing randomness within the controlled range.

This approach is particularly useful when you want to test how small variations in data might affect the outcome of a trading strategy, ensuring that the strategy is not overly sensitive to minor fluctuations in the market data.

```
import numpy as np

def apply_blur(df, column_name, blur_percent):
    """
    Apply a normal distribution blur to a specified column in a DataFrame.
```

```
    Parameters:
    df (DataFrame): The DataFrame containing the data.
    column_name (str): The name of the column to apply the blur on.
    blur_percent (float): The percentage used to calculate the blur. This determines the range for generating new random values.

    Returns:
    DataFrame: A new DataFrame with the specified column blurred.
    """
    # Create a copy of the DataFrame to avoid modifying the original
    df_blurred = df.copy()

    for i in range(len(df_blurred)):
        original_value = df_blurred.at[i, column_name]
        std_dev = blur_percent * original_value / 3.0

        # Calculate the min and max bounds for the blur
        min_value = original_value - 3 * std_dev
        max_value = original_value + 3 * std_dev

        # Generate a new value using the normal distribution
        new_value = np.random.normal(loc=original_value, scale=std_dev)

        # Assign the new value to the DataFrame
        df_blurred.at[i, column_name] = new_value

    return df_blurred
```

To apply the blur to our strategy, which we've introduced in previous chapters where we analyze rejection candles near key levels on the daily timeframe, we need to implement a process of blurring certain price data, such as the closing price, by a specific percentage, in this example, we'll use 1%. After applying the blur, the modified data is saved in a new DataFrame. We

then regenerate all the trading signals, including rejection detection, pivot detection, support and resistance levels, and update the trading signals accordingly. This is done before conducting a backtest on the newly blurred DataFrame.

For the backtest, we will use the same conditions as before: a 2% commission fee, a starting account balance of $100,000, and a leverage ratio of 2. The backtest results will be stored in a list called stats_list, which will later be used to plot the results of the various simulations.

The following code snippet automates all the necessary steps to execute the backtest and generate the statistics list. It repeats the entire process 1000 times, providing 1000 different results. This sample size will allow us to effectively visualize the equity curves, quality ratios, drawdowns, and other key metrics to assess the robustness of our trading strategy.

```
stats_list = []
for i in range(1000):
    window=3
    df_rand=df.copy()
    df_rand.reset_index(inplace=True)
    df_rand = apply_blur(df, "Close", 0.01)
    df_rand['rejection'] = df_rand.apply(lambda row:
is_rejection(df_rand, row.name, body_perc = 0.1, wick_ratio=1.5),
axis=1)
    df_rand['isPivot'] = df_rand.apply(lambda x:
isPivot(x.name,window), axis=1)
    support_resistance_levels =
get_support_resistance_levels(df_rand, start_row=window,
end_row=300)
    cleaned_resistance, cleaned_support =
clean_support_resistance(support_resistance_levels,
threshold_percent=10)
    df_rand['signal'] = df_rand.apply(lambda x:
check_candle_signal(df_rand, x.name, 60, 8), axis=1)
```

```
    df_rand.set_index(["Date"], inplace=True, drop=True)
    bt = Backtest(df_rand, MyCandlesStrat, cash=100000,
margin=1/2, commission=0.02)
    stats_list.append(bt.run())
```

3 Visualizing Monte-Carlo Results

3.1 Equity Curves

To visualize the equity curves and analyze the edges along with the average equity behavior, we can use the following code. This script accesses all the equity curves stored in stats_list and plots them on a graph with different colors, making it easy to observe the variations and trends across different backtests.

```
import matplotlib.pyplot as plt

# Number of backtests
n_backtests = len(stats_list)

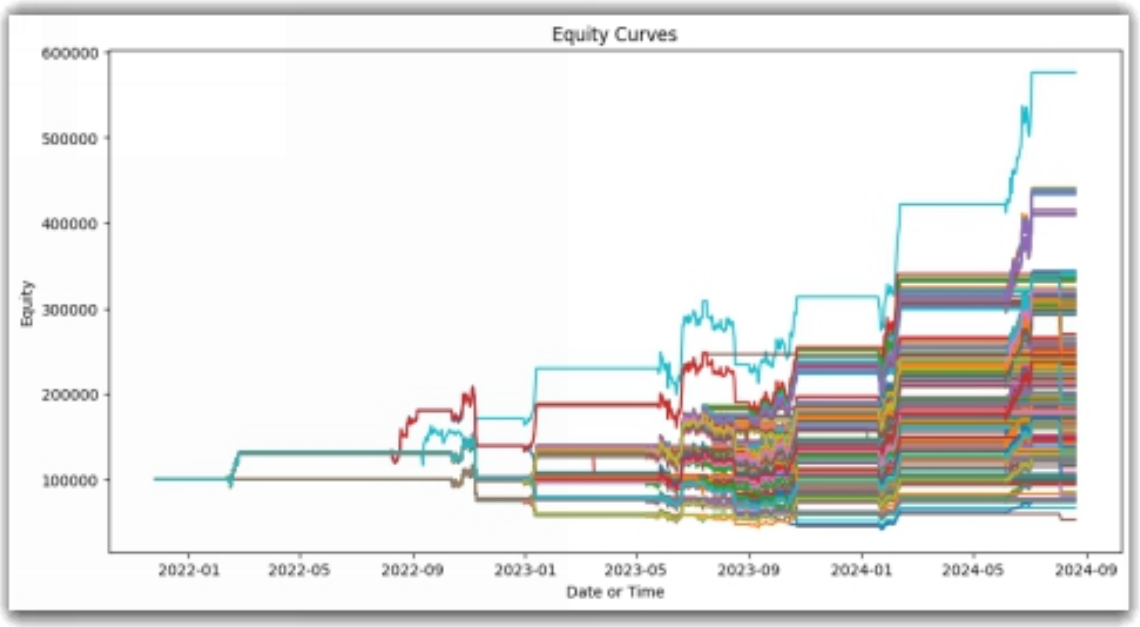
# Initialize a plot
plt.figure(figsize=(12, 6))

# Plot each equity curve
for i in range(n_backtests):
    equity_curve = stats_list[i]["_equity_curve"] # Get the equity
curve DataFrame
    plt.plot(equity_curve.index, equity_curve['Equity'],
label=f'Backtest {i+1}')

# Adding labels and title
plt.xlabel('Date or Time')
plt.ylabel('Equity')
plt.title('Equity Curves')

# Show the plot
plt.show()
```

The equity results are displayed in the figure below. Starting with an initial account balance of 100,000 U.S. dollars, we can observe that the majority of the equity curves end in a positive trajectory. This is encouraging, especially considering that the strategy is subjected to a blurring operation and incurs a 2% commission fee on each trade, making the conditions quite challenging. Despite these harsh conditions, the strategy still shows potential for positive returns, though a small percentage of equity curves do end in the negative. We will analyze these further in the next subsection.



3.2 Visualizing Distribution of Final Outcomes

At this stage, we're also interested in visualizing the final outcomes related to various quality factors and evaluation metrics provided in the backtest reports. To achieve this, we can use a function designed to analyze and plot the distribution of these metrics. The function takes two input parameters: stats_list, which contains the backtest data, and element_name,

which specifies the key of the element to plot and analyze. This element could be final equity, annual return percentage, average drawdown, maximum drawdown, or any other metric of interest from the backtest report.

The function generates a histogram showing the distribution of values across the different backtests, and overlays a fitted normal distribution curve. While some of these metrics may not strictly follow a normal distribution, the bell curve serves as a useful reference or benchmark. It illustrates the range of possible values, helping to define the expected width of this metric when market conditions vary randomly according to our initial blurring conditions.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm

def plot_distribution_with_normal_fit(stats_list, element_name):
    """
    Plots the distribution of a specified element from the backtest results
    and fits a normal distribution curve on top.

    Parameters:
    - stats_list: List of dictionaries containing backtest results.
    - element_name: The key of the element to plot (e.g., 'Return (Ann.) [%]').

    Returns:
    - A plot showing the distribution of the specified element with a normal distribution fit.
    """

    # Extract the specified element from each backtest
    element_values = [stats[element_name] for stats in stats_list]
```

```
    # Calculate mean and standard deviation for the normal
distribution fit
    mean = np.mean(element_values)
    std_dev = np.std(element_values)

    # Plot the histogram of the element values
    plt.figure(figsize=(10, 6))
    sns.histplot(element_values, bins=10, kde=False, color='blue',
stat='density', label=f'{element_name} Distribution')

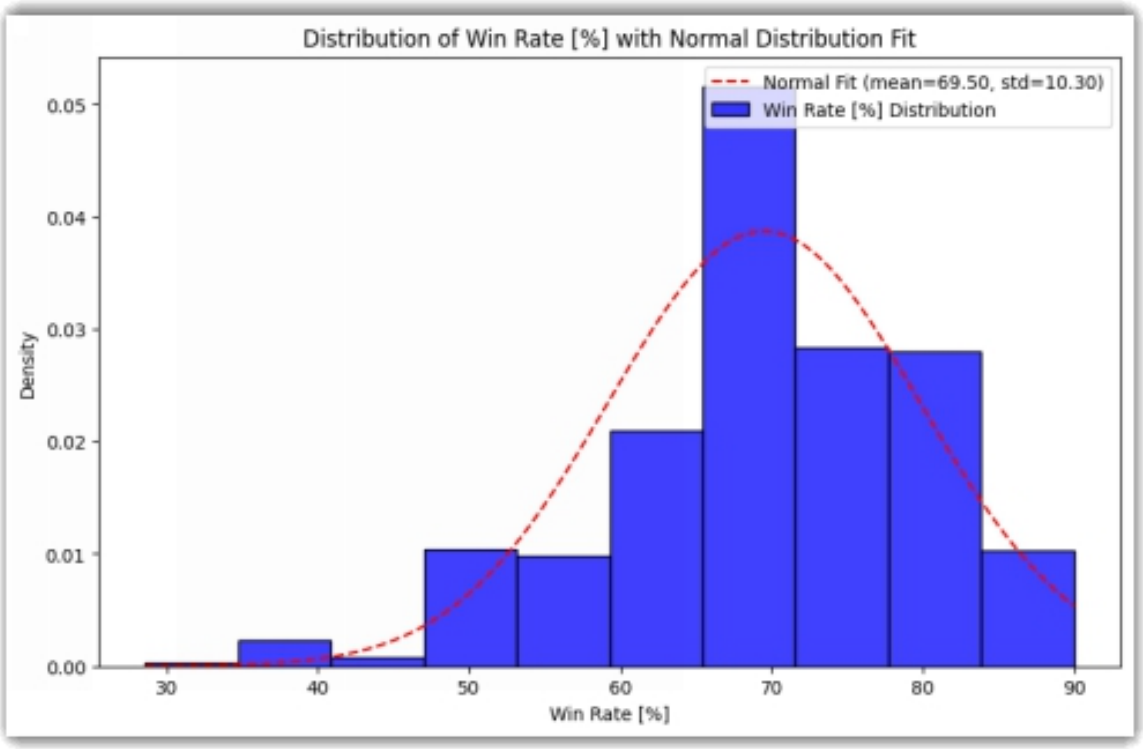
    # Plot the normal distribution curve
    x = np.linspace(min(element_values), max(element_values),
100)
    p = norm.pdf(x, mean, std_dev)
    plt.plot(x, p, 'r--', label=f'Normal Fit (mean={mean:.2f}, std=
{std_dev:.2f})')

    # Adding labels and title
    plt.xlabel(element_name)
    plt.ylabel('Density')
    plt.title(f'Distribution of {element_name} with Normal
Distribution Fit')
    plt.legend(loc='upper right')

    # Show the plot
    plt.show()
```

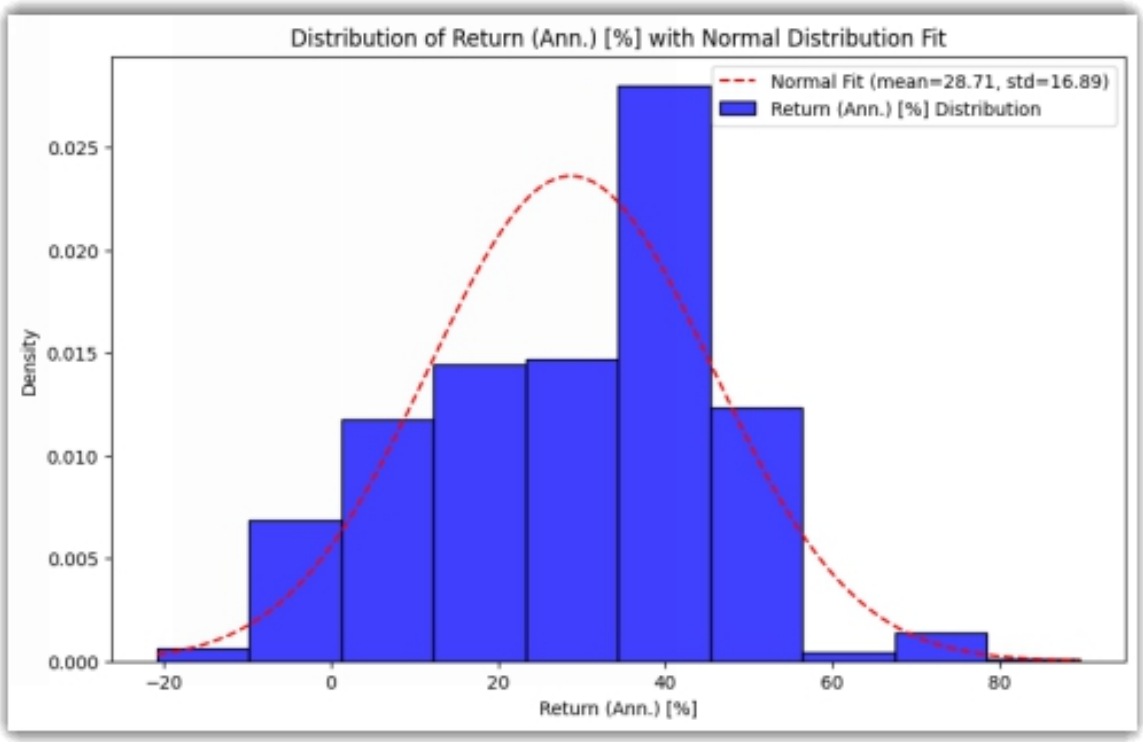
We can utilize this function by passing in the statistics list that contains all the recorded backtesting results and, for instance, the win rate percentage.

```
plot_distribution_with_normal_fit(stats_list, 'Win Rate [%]')
```

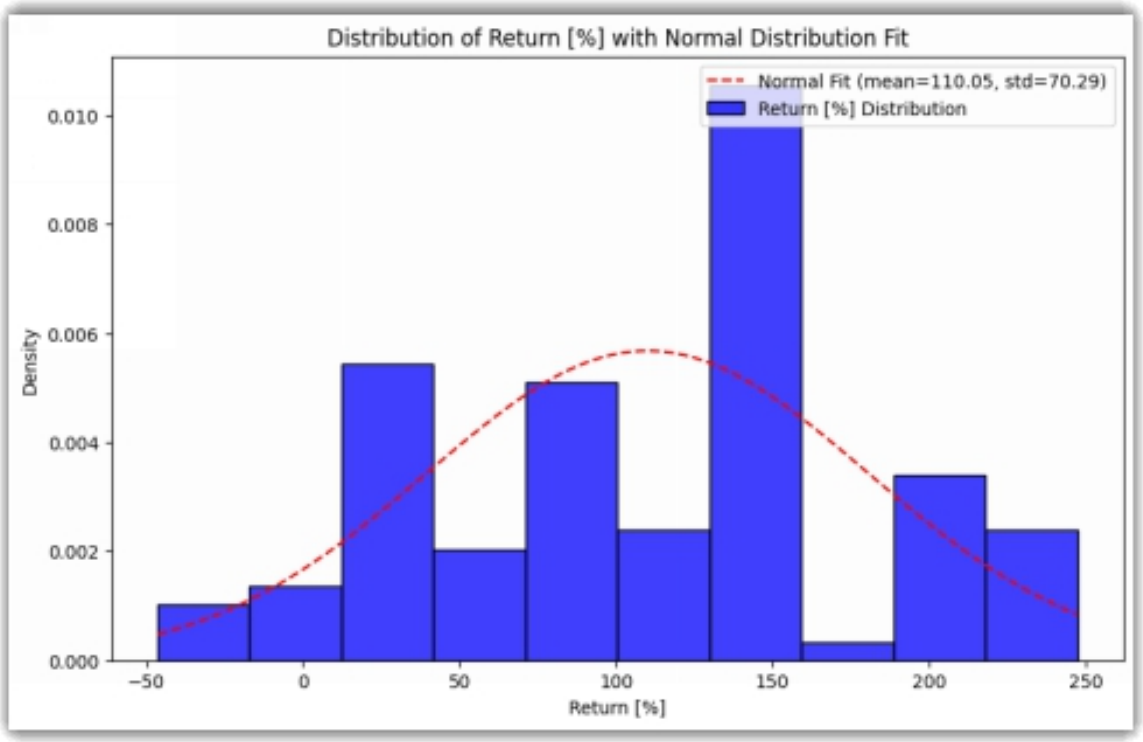


Let's also apply it to the annual return percentage and the total return percentage.

```
plot_distribution_with_normal_fit(stats_list, 'Return (Ann.) [%]')
```




```
plot_distribution_with_normal_fit(stats_list, 'Return [%]')
```

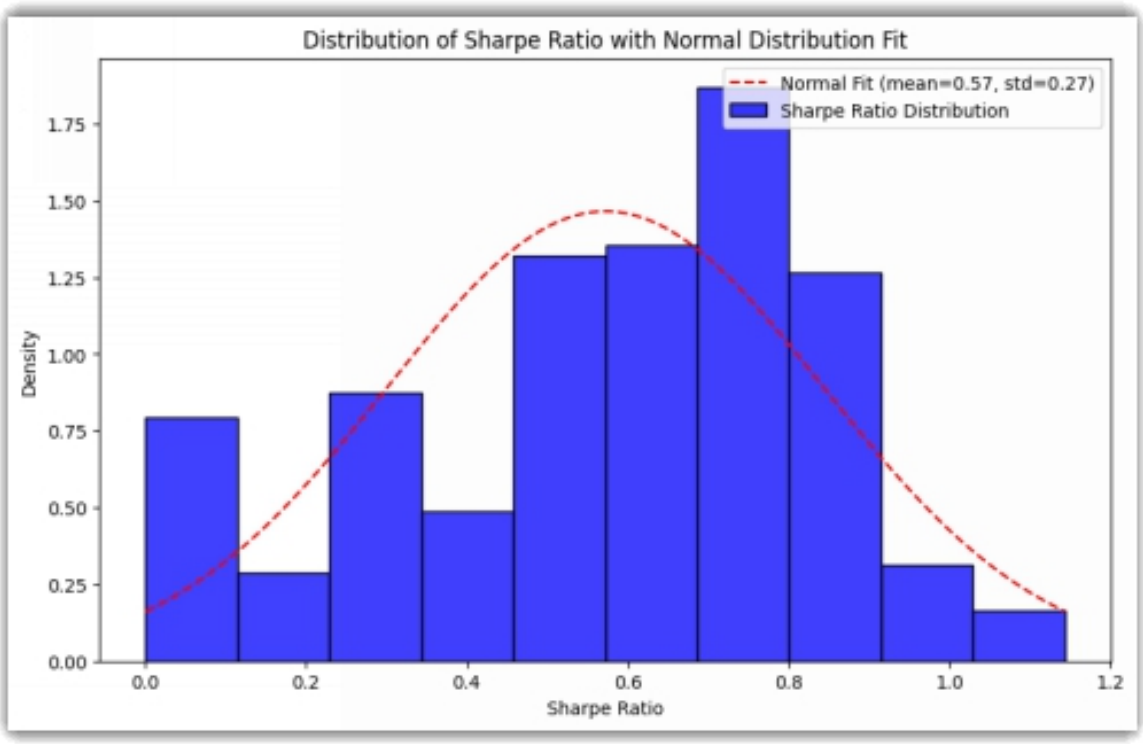


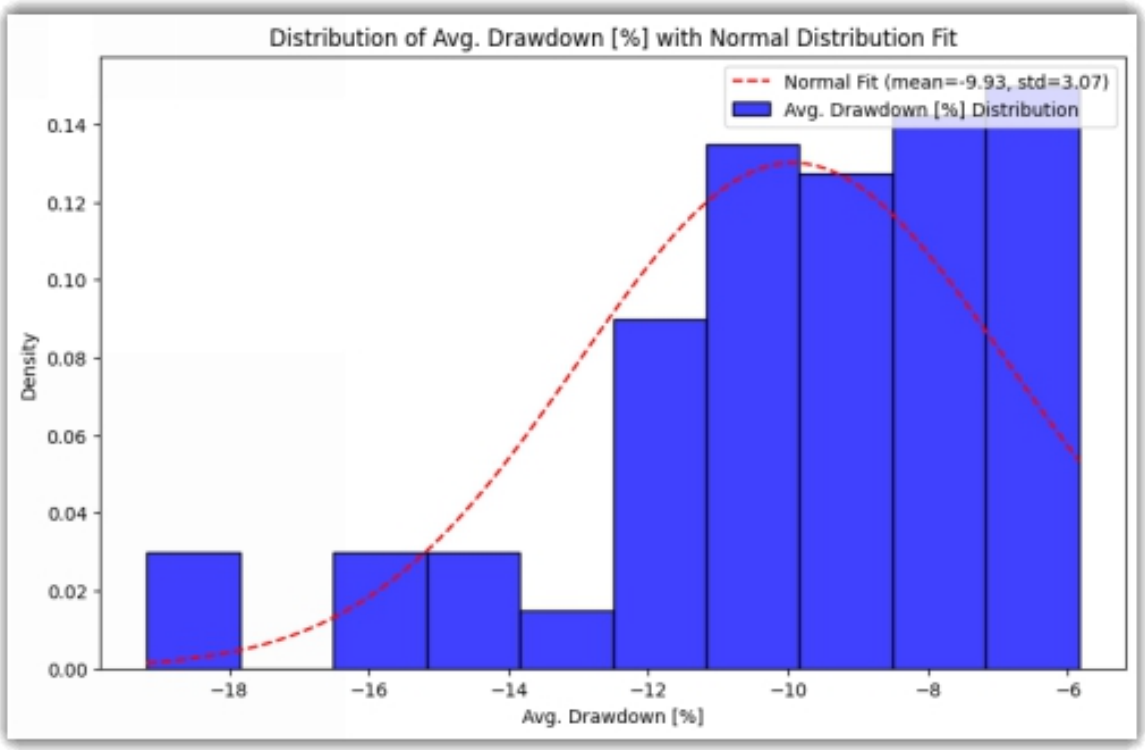
The win rate results indicate that most of the values are above 60%, with an average around 70%. Extreme cases show a minimum win rate of 30% and a maximum of 90%. Meanwhile, the annual return percentage averages around 30%, with a minimum of -20% and a mode (the most frequent value) at 40%. In some exceptional cases, the strategy can generate annual returns of up to 70% to 80%. The total return percentage over the entire dataset averages around 100%, with the mode at approximately 150%, indicating this is the most probable total return value. Extreme cases range from a minimum of -50% to a maximum of +250%.

These results suggest that the strategy is skewed towards positive returns and a favorable win rate. To further our analysis, we can also examine the distribution of the Sharpe ratio and the average drawdown, two metrics that offer insights into the risk associated with this trading strategy.

```
plot_distribution_with_normal_fit(stats_list, 'Sharpe Ratio')
```

```
plot_distribution_with_normal_fit(stats_list, 'Avg. Drawdown [%]')
```

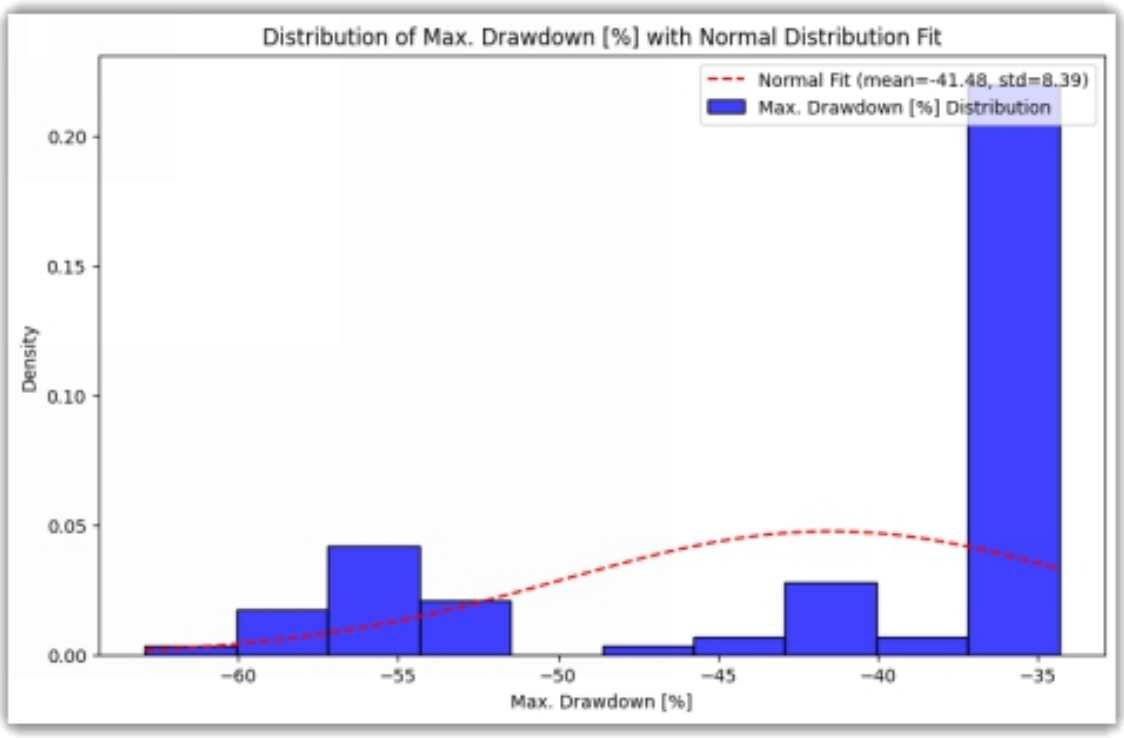




The Sharpe ratio distribution reveals an average value of 0.6, with a high likelihood of achieving a ratio above 0.5. The most common value is around 0.7, which, while not exceptional, is a reasonable outcome for a straightforward strategy like this one. It’s a decent starting point, although there's room for improvement.

The average drawdown displays an average of -10%, with the minimum reaching -18%. However, the majority of the values are above -13%, indicating that only in rare cases does the average drawdown fall below this threshold.

For a more thorough analysis, it's also important to examine the distribution of the maximum drawdown percentage.



The results of our 1000 Monte Carlo simulations reveal two distinct clusters in the maximum drawdown percentage: one below -50% and another above -50%. This pattern is significant because it suggests that in reality our strategy is likely to encounter two different drawdown behaviors under varying market conditions. The fact that we observe two clusters, rather than a smooth distribution, implies that some inherent factors are driving this divergence.

Understanding why these clusters exist requires further investigation. One plausible explanation is the effect of our blurring technique on the closing prices of candles. When we alter the closing prices, certain signals might disappear because the criteria for patterns like rejection candles may no longer be met. This change can invalidate previously valid trading signals or even generate new ones, affecting the overall trading performance. Additionally, the adjustment of closing prices could shift the distance between the candle and the nearest key level, leading to a similar disruption in signal generation. These

changes might be responsible for the formation of the two distinct clusters in the maximum drawdown data.

Despite this, the most common maximum drawdown observed in our simulations is around -35%, which stands out as the highest probability value. However, in extreme cases, maximum drawdowns can reach as deep as -55% or -60%, highlighting the potential risks associated with this strategy.

To mitigate these risks and improve the strategy's robustness, several adjustments can be considered:

- **Reduce Leverage or Margin Usage**: By minimizing or eliminating leverage, the risk of substantial drawdowns can be reduced, though this will likely result in lower overall returns.
- **Adjust Lot Size**: Instead of trading the entire $100,000 equity at once, consider reducing the lot size to lower the risk. This is especially important when using leverage, as it can magnify both gains and losses.
- **Refine Trade Management**: The current trade management approach is simplistic, using a fixed take profit to stop loss ratio of 2 and a 10% risk percentage. While this was sufficient for initial testing, more sophisticated trade management techniques could yield better risk-adjusted returns. For instance, dynamic stop loss adjustments or trailing stops might provide more flexibility in responding to market movements.

The key takeaway here is that even with a basic strategy, using classic candle patterns, key levels, and simple trade management, we can conduct a comprehensive and sophisticated evaluation using Monte Carlo simulations. This process helps us assess the strategy's robustness under varying conditions and provides valuable insights into potential

improvements.

4 Conclusion

In this chapter, we've explored the application of Monte Carlo methods to test the robustness of a trading strategy under varying conditions. While the initial analysis provided valuable insights, it's essential to recognize that robustness testing is not a one-size-fits-all approach. To gain a comprehensive understanding of a strategy's performance, it's advisable to extend this analysis across multiple assets. Ideally, conducting Monte Carlo simulations on at least 10 different assets, with 1000 backtests each, will yield a more holistic view of the strategy's behavior across diverse market environments. This broader approach will help to identify patterns, potential weaknesses, and areas for improvement that may not be apparent when testing on a single asset.

Moreover, the randomization technique used in this chapter, specifically blurring the closing price, is just one of many possible avenues to explore. Randomization can and should be applied to other aspects of the strategy. For instance, randomizing entry positions can simulate the effect of slippage, where the actual execution price differs from the intended price due to rapid market movements. By introducing variability in trade sizes, you can assess how position sizing impacts risk and return under different scenarios. Randomizing take profit values allows you to see how varying exit points affect overall performance, especially in volatile markets where prices can swing dramatically in short periods.

These variations in randomization techniques are crucial for simulating real-world trading conditions more accurately. Markets are inherently unpredictable, with numerous factors influencing price movements. By incorporating randomness into your backtesting, you're not just testing the strategy under ideal

conditions but also preparing it for the chaotic and often unforgiving reality of live trading. This approach provides deeper insights into the strategy's resilience, helping you to identify potential failure points and areas that require refinement.

One creative idea to further enhance your robustness testing is to introduce scenario-based randomization. Instead of randomizing variables uniformly, you could design specific market scenarios, such as sudden market crashes, prolonged bull runs, or periods of low volatility, and then apply randomization within these contexts. This allows you to see how your strategy performs under specific market conditions, which is crucial for understanding its strengths and weaknesses in different phases of the market cycle.

Another avenue for further research could be to incorporate machine learning techniques to optimize the parameters being randomized. For example, you could use reinforcement learning to identify the most critical parameters that influence the strategy's performance, allowing you to focus your randomization efforts more effectively. This would not only make your robustness tests more efficient but also provide insights into which aspects of your strategy are most sensitive to market changes.

Finally, it's essential to remember that robustness testing is an ongoing process. Markets evolve, and what works today may not work tomorrow. Continuously testing and refining your strategy ensures that it remains adaptable and resilient in the face of changing market conditions. By staying committed to this process, you're not just building a strategy that works; you're building a strategy that can withstand the test of time. Monte Carlo methods offer a powerful toolset for stress-testing trading strategies. By embracing the complexity and unpredictability of markets, and by continually seeking new ways to challenge your strategy, you set yourself on a path toward developing truly robust and reliable trading systems. This chapter is just the

beginning, there's a vast landscape of possibilities waiting to be explored.

Chapter 11: Automated Trading Systems

1 Introduction

In the introduction of this book, we discussed the distinction between algorithmic trading and fully automated trading. Technically, the difference is straightforward: algorithmic trading involves manual trading enhanced by advanced algorithms, whereas automated trading refers to a fully autonomous system that manages every aspect of trading, from opening and closing positions to handling market dynamics.

Algorithmic trading often outperforms fully automated systems because the human element plays a crucial role in decision-making during trading sessions. Humans are more adaptable and can respond more quickly to market fluctuations and price movements. On the other hand, automated trading systems have the advantage of eliminating emotional responses and irrational decisions, but they lack the flexibility to adjust to changing market conditions. This is why, even with fully automated systems, regular monitoring and supervision are necessary to ensure that the trading environment remains favorable.

Algorithmic trading, which combines human decision-making with automation, utilizes several automated tools such as direct market data feeds, market analysis based on predefined conditions, and identification of potential trading opportunities. However, the trader still has to make critical decisions, whether to open a position, where to set take profit and stop loss levels, and how to manage the trade once it's active. While this means that roughly 50% of the work still falls on the trader, it’s not a disadvantage. The algorithmic system saves the trader from the

time-consuming task of constantly monitoring the market for patterns, reacting more swiftly to market analysis tasks like drawing support and resistance levels or checking candlestick patterns. This reduces the tedium of manual monitoring, especially when market conditions do not always lead to a trade.

I emphasize these points because many traders, particularly beginners, often confuse algorithmic trading with fully automated trading, expecting a system that runs independently and generates profits with minimal intervention. The reality is that, while algorithmic trading can enhance results and increase profitability, the trader's presence, expertise, and decision-making are still essential components of the process. Fully automated systems, on the other hand, operate blindly based on preset conditions and indicators. Unless you're willing to invest significant time and effort in developing a dynamic automated system that performs comprehensive market analysis including monitoring economic calendars and managing risks, such systems are prone to failure when market conditions shift. I've seen systems perform exceptionally well for several months, generating returns of 6-7% per month, only to fail abruptly when market conditions change, because the system isn't dynamic enough to adapt. In these cases, even a small drawdown can wipe out the account and any profits previously earned.

In this short chapter, we'll explore an example of using algorithmic trading with a set of market conditions, where an email notification alerts you to potential trading opportunities. We will also discuss how to fully automate a trading strategy using a broker's Python API.

2 Automating trading alerts

As previously discussed, one effective way to leverage algorithmic trading is by allowing the algorithm to handle the waiting time, detect patterns, and send trading alerts or

opportunity notifications. These notifications can be sent via message, phone, or email. The simplest and most straightforward method, which requires no additional subscriptions or work, is using email notifications. With Python, you can easily set up an email notification system. For example, you can create a Gmail account specifically for this purpose and configure your Python algorithmic bot to send emails from this account. If you have email notifications enabled on your phone, you'll instantly receive these alerts, provided you have an internet connection. I'm sharing these technical details to demonstrate how easily and cost-effectively this can be done.

Assuming Yahoo Finance provides sufficiently reliable data for trading, we'll use it in the following example to retrieve data for Apple stock. We'll perform a quick analysis to check for any engulfing patterns on the last two candles, which correspond to the last two days since we're querying daily data. If an engulfing pattern is detected, the algorithm will send an email notification.

This example illustrates how you might use such a system as part of your trading strategy or experimentation. You could test an algorithm this way for a few months to see how it supports your trading decisions. Of course, you can extend this approach to detect more than just engulfing patterns. You can apply as many indicators as you like to build a more robust and reliable signaling system.

The key advantage of this system is that it saves you time. When using a daily timeframe, patterns may occur infrequently, perhaps only twice a month. Rather than checking for these patterns every day, you can simply launch the algorithm and let it do the work for you. More importantly, the algorithm can perform this repetitive task across multiple assets. For instance, it can analyze 50 assets daily in just a few seconds, a task that would be tedious for a human trader. You can also configure the algorithm to send emails only when there is a potential trading opportunity. In the example code I'll provide below, the system

sends notifications regardless of the trading configuration, meaning, even if no valid signal is detected, it will send an empty email just to test the tool and confirm that it's working. Receiving these daily emails serves as a confirmation that the system is still operational. If you stop receiving emails, it's time to check the server or the computer where the model is deployed to ensure everything is functioning properly.

Now, let's move on to the coding part. First, we're going to define a function to get the data using Yahoo finance module. Just remember very important at this point you can use any data source you prefer provided that this data source is updating its candles every day because if you're working on the daily timeframe, we need the candles to be coming at the correct time. On this note many brokers nowadays provide Python API to get the data, I can mention few examples like Oanda, Interactive Brokers and Binance among others.

```
import yfinance as yf

def get_data(symbol="AAPL"):
    # Define the ticker symbol
    ticker_symbol = symbol

    # Create a ticker object
    ticker = yf.Ticker(ticker_symbol)

    # Download historical data
    return ticker.history(period="2d", interval='1d')

historical_data = get_data()
```

In this initial part of the code, we define the get_data function to retrieve data on the daily time frame, focusing only on the last two days, essentially capturing today's and yesterday's candles. To keep things simple, we then implement a basic function to check for an engulfing pattern. For demonstration purposes, this function is a simplified version, though you could apply the more comprehensive engulfing

pattern detection function we've developed in previous chapters for a more accurate and reliable analysis.

```
def test_engulfing(df):
    last_open = df.iloc[-1, :].Open
    last_close = df.iloc[-1, :].Close
    previous_open = df.iloc[-2, :].Open
    previous_close = df.iloc[-2, :].Close

    if (previous_open < previous_close
        and last_open > previous_close
        and last_close < previous_open):
        return 1  # Bearish Engulfing Pattern

    elif (previous_open > previous_close
          and last_open < previous_close
          and last_close > previous_open):
        return 2  # Bullish Engulfing Pattern
    else:
        return 0  # No Engulfing Pattern

test_engulfing(historical_data)
```

Notice how this function takes a DataFrame as input, which in this case consists of only two rows since we've limited our query to the last two candles. We then use these two candles to check for the presence of a bullish or bearish engulfing pattern.

To enable email notifications, you'll need access to an email account. Gmail offers this service for free and allows you to create what's called an "application password." This is a unique password generated specifically for an application to access your email account. It's not the password for your entire Gmail account but rather a dedicated password for specific use cases. You can easily create one from the Gmail security page. Now, let's go through the details of the following code:

```
from apscheduler.schedulers.blocking import BlockingScheduler
from email.message import EmailMessage
```

```
import ssl
import smtplib
from creds import gmail_user, gmail_password

em = EmailMessage()

gmail_user = gmail_user
gmail_password = gmail_password
subject = 'info signal'
```

- **apscheduler.schedulers.blocking.BlockingScheduler:** This is part of the APScheduler library, which is used for scheduling jobs in Python. The BlockingScheduler is a type of scheduler that runs jobs in a blocking manner, meaning it will prevent the script from exiting until the scheduler has been shut down. We will use this later to schedule our calls for pattern detection.
- **email.message.EmailMessage:** This module from Python's email package is used to create email messages. The EmailMessage class provides a convenient way to set up the content of an email, including headers and body.
- **ssl:** This module is used for managing SSL (Secure Sockets Layer) connections, which is important for sending secure emails via SMTP (Simple Mail Transfer Protocol).
- **smtplib:** This is a Python library used to send emails using the Simple Mail Transfer Protocol (SMTP). It handles the actual transmission of the email.
- **creds:** This is a file named creds.py that I created and it contains the email credentials, specifically the gmail_user and gmail_password variables. These are the Gmail username and the application password that will be used to authenticate with

Gmail's SMTP server. If you use the Gmail password here it will not work Gmail expects the application password when you connect from external applications such as our python example.

Next, we create an email message using an instance of the EmailMessage class. We assign the Gmail credentials to the gmail_user and gmail_password variables, and set the subject of the email.

Then, we define a function named detection_job() which will handle the entire process. This function will create the email message, retrieve the latest historical market data, and apply the test_engulfing() function to identify any bearish or bullish patterns. Based on the detection results, it will formulate and send the email to the specified address. In this example, the email is sent to the same address that we are using for the Gmail account.

```
def detection_job():
    # Initialize the message body with a generic introduction.
    msg = "Trading Signal Message \n"

    # Retrieve the most recent historical data (e.g., the last two
candles).
    historical_data = get_data()

    # Check the historical data for an engulfing pattern.
    # If a bearish engulfing pattern is detected, update the
message accordingly.
    if test_engulfing(historical_data) == 1:
        msg = str("The signal is 1: Bearish")

    # If a bullish engulfing pattern is detected, update the message
accordingly.
    elif test_engulfing(historical_data) == 2:
        msg = str("The signal is 2: Bullish")
```

```
    # Set up the email details, specifying the sender, recipient, and
subject.
    em['From'] = gmail_user
    em['To'] = gmail_user
    em['Subject'] = subject

    # Set the content of the email with the message that was
created based on the signal.
    em.set_content(msg)

    # Create a secure SSL context to encrypt the email
communication.
    context = ssl.create_default_context()

    # Connect to Gmail's SMTP server using SSL on port 465.
    server = smtplib.SMTP_SSL('smtp.gmail.com', 465,
context=context)

    # Identify ourselves to the SMTP server.
    server.ehlo()

    # Log in to the Gmail account using the provided credentials.
    server.login(gmail_user, gmail_password)

    # Send the email to the same Gmail account (from
`gmail_user` to `gmail_user`).
    server.sendmail(gmail_user, gmail_user, em.as_string())

    # Close the connection to the SMTP server.
    server.close()
```

To execute this, we can simply call the function with a single line of code. This will also serve as a test to ensure the function is working correctly, if everything is set up properly, you should receive an email indicating whether a trading signal has been detected or not.

```
detection_job()
```

Depending on the time frame and strategy, we need to

schedule the function to run at specific times. In this example, we're using a daily time frame, so it's sufficient to call the function once per day, ideally at the end of the trading day after the daily candle has closed. This allows us to detect any emerging engulfing patterns in time to make an informed decision about entering the market.

To automate this process, we can use a scheduler in Python. In the following code, the function is scheduled to run once daily, from Monday to Friday, at midnight. The misfire_grace_time is set to 15 minutes, meaning that if, for any reason (like connection issues or hardware failure), the function is not executed at the scheduled time, it will only be allowed to run within a 15-minute window. If this window is missed, the function won't be executed until the next scheduled time.

```
scheduler = BlockingScheduler(job_defaults={'misfire_grace_time':
15*60})
scheduler.add_job(detection_job, 'cron', day_of_week='mon-fri',
hour=0, minute=0, timezone=utc)
scheduler.start()
```

The code begins by initializing a blocking scheduler instance from the APScheduler library. A blocking scheduler runs jobs in the foreground, meaning it occupies the main thread and doesn't return until the job is complete. This type of scheduler is ideal when the scheduled tasks are the primary focus of your script, as it ensures nothing else runs in parallel. In this case, the primary task is market probing, so a blocking scheduler is appropriate.

The misfire_grace_time is set to 15 minutes, allowing a job that missed its scheduled time (due to issues like connection or hardware problems) to still execute within a 15-minute window. After this setup, the job is scheduled using the add_job function, where we specify the function detection_job to be executed. The job is scheduled to run using a Cron-like syntax, meaning it will execute at midnight UTC from Monday to Friday.

Finally, the scheduler.start() function starts the scheduler, which begins executing the scheduled jobs according to the defined schedule.

3 Automating Alerts for Multiple Assets

We can automate the same logic on multiple assets by adding the symbols of these assets into a list. We can modify the previously introduced code in the following way:

```
symbols =  ['AAPL', 'NVDA', 'PYPL']

def some_job():
    msg="Trading Signal Message \n"
    for symb in symbols:
        historical_data = get_data(symb)
        if test_engulfing(historical_data)==1:
            msg = msg + str(symb+": the signal is 1 bearish") + "\n"

        elif test_engulfing(historical_data)==2:
            msg = msg + str(symb+": the signal is 2 bullish") + "\n"

    em['From'] = gmail_user
    em['To'] = gmail_user
    em['Subject'] = subject
    em.set_content(msg)

    context = ssl.create_default_context()

    server = smtplib.SMTP_SSL('smtp.gmail.com', 465,
context=context)
    server.ehlo()
    server.login(gmail_user, gmail_password)
    server.sendmail(gmail_user, gmail_user, em.as_string())
    server.close()
```

Here, we've added a list of symbols we're interested in tracking. Inside the some_job() function, we loop through each symbol, retrieve the data, and test for the engulfing pattern. As we process each asset, we build the email message by appending any new information to the existing message variable. This ensures that information about one asset doesn't overwrite the signals for another. Essentially, this setup allows us to run the same function across multiple assets seamlessly. There's no practical limit to the number of assets you can include, whether it's 10, 50, or even hundreds of assets, they can all be queried and tested in under a minute.

4 Fully Automated Trading Approach

To build fully automated systems we can follow somehow are similar approach to the previous section with some modifications that would allow us to connect to a broker and pass on trades on the market straight from the Python code. For this section I'm going to use a Oanda's API, just keep in mind this is not the only broker or the only API available on the market there are many other brokers providing Python interfaces for trading and algorithmic trading it might be worth going through their documentation 1st to see what are your options and which one fits better within your needs.

For this example we're going to use a strategy that we've already explained in the previous chapter using support and resistance along with the rejection candles happening close to these key levels, it's the same strategy that we have back tested and analyzed earlier in this book.

Now let's breakdown the code into different parts.

```
from apscheduler.schedulers.blocking import BlockingScheduler
from oandapyV20 import API
```

```
import oandapyV20.endpoints.orders as orders
from oandapyV20.contrib.requests import MarketOrderRequest
from oanda_candles import Pair, Gran, CandleClient
from oandapyV20.contrib.requests import TakeProfitDetails,
StopLossDetails
```

- **API from oandapyV20**: This imports the API class from the OANDA Python library, which is used to interact with the OANDA trading platform for placing trades, retrieving market data, and managing accounts.
- **Orders and MarketOrderRequest from oandapyV20.endpoints.orders**: These imports allow the script to create and send market orders (buy/sell orders) through the OANDA API.
- **Pair, Gran, and CandleClient from oanda_candles**: oanda_candles is a client to make getting Oanda candle data easy. These imports are custom classes or functions used to define currency pairs (Pair), time granularities (Gran), and to retrieve candlestick data (CandleClient) from the OANDA platform.
- **TakeProfitDetails and StopLossDetails from oandapyV20.contrib.requests**: These imports allow the script to define the parameters for setting take-profit and stop-loss levels when placing orders.

```
from config import access_token, accountID
access_token='Put Access Token Here'
accountID = 'Put Account ID Here'
def get_candles(n):
    client = CandleClient(access_token, real=False)
    collector = client.get_collector(Pair.BTC_USD, Gran.D1)
    candles = collector.grab(n)
```

```
    return candles
```

This code uses an access token and the account ID that are usually generated from Oanda's platform. These are necessary to give your Python application access to the account which will allow you to query data getting the candles and also to pass positions and stop loss and take profit values.

Then we defined a get_candles() function that takes one parameter which is the number of candles to be queried from the platform it will get the last number of candles provided as an input parameter. The function uses candle client function with the access token and the parameter real equal folds to mention that we're using this function on a paper account and not a real account. The collector uses get_collector function to get the Bitcoin U.S. dollar data using a granularity on the daily time frame.

Now let's define a new function fetch_candle_data that uses the get_candles function to query the candles apply type transformation and put the candles in a proper format in a pandas data frame:

```
def fetch_candle_data(num_candles=300):
    """
    Fetches the specified number of candles and returns a
DataFrame.
    """
    candles = get_candles(num_candles)
    dfstream = pd.DataFrame(columns=['Open', 'Close', 'High',
'Low'])

    for i, candle in enumerate(candles):
        dfstream.loc[i, 'Open'] = float(candle.bid.o)
        dfstream.loc[i, 'Close'] = float(candle.bid.c)
        dfstream.loc[i, 'High'] = float(candle.bid.h)
        dfstream.loc[i, 'Low'] = float(candle.bid.l)
```

```
    return dfstream.astype(float)
```

We also need a function to calculate the stop loss and take profit levels based on the most recent candles. This function will return the appropriate stop loss and take profit levels for both buy and sell signals. It takes as input a data frame containing the candle data and a take profit to stop loss ratio, which is set to 2 by default.

Next, we can define a function called execute_trade to handle trade execution. This function takes several parameters: the API client, the account ID, the trading signal (whether it's bullish or bearish), and the calculated stop loss and take profit levels for both long and short positions.

- If the signal indicates a sell (represented by 1), the function will execute a sell position.
- If the signal indicates a buy (represented by 2), it will execute a long position.
- If the signal is anything else, the function will return without executing any orders.

When a valid signal is received, the function creates an order using the OrderCreate function, passing the account ID and the market order data. After executing the trade, it prints the details of the order to the screen, allowing you to review the trade execution.

```
def execute_trade(client, accountID, signal, SLBuy, SLSell, TPBuy,
TPSell):
    """
    Executes a trade based on the provided signal.
    """
    if signal == 1:  # Sell signal
        mo = MarketOrderRequest(
            instrument="BTC_USD",
            units=-1,
```

```
            takeProfitOnFill=TakeProfitDetails(price=TPSell).data,
            stopLossOnFill=StopLossDetails(price=SLSell).data
        )
    elif signal == 2:  # Buy signal
        mo = MarketOrderRequest(
            instrument="BTC_USD",
            units=1,
            takeProfitOnFill=TakeProfitDetails(price=TPBuy).data,
            stopLossOnFill=StopLossDetails(price=SLBuy).data
        )
    else:
        return

    r = orders.OrderCreate(accountID, data=mo.data)
    rv = client.request(r)
    print(rv)
```

Finally, we will create a function named trading_job(), this will serve as the main trading function. It will fetch the candle data, check for trading signals, and execute trades by utilizing the functions we previously defined.

```
def trading_job():
    """
    Main trading function that fetches candle data, checks for a signal, and executes trades.
    """
    dfstream = fetch_candle_data()
    signal = check_candle_signal(
        l=len(dfstream) - 1,
        n1=6,
        n2=6,
        levelbackCandles=200,
        windowbackCandles=7,
        df=dfstream
    )
```

```
    SLBuy, SLSell, TPBuy, TPSell = calculate_sl_tp(dfstream)

    # Initialize API client
    client = API(access_token)

    # Execute trade based on the signal
    execute_trade(client, accountID, signal, SLBuy, SLSell, TPBuy,
TPSell)
```

Once everything is configured, we can automate the execution of orders using the scheduler, similar to how we scheduled algorithmic trading signals in the previous section. In this case, the trading_job function is set to run daily at 11:55 PM, just before midnight, using the Americas/Chicago time zone. This timing is just an example, you can adjust these parameters to suit your specific time zone and the trading hours that work best for your system.

```
scheduler = BlockingScheduler()
scheduler.add_job(trading_job, 'cron', hour='23', minute='55',
start_date='2023-10-20 23:55:00', timezone='America/Chicago')
scheduler.start()
```

5 Conclusion

And this brings us to the conclusion of this chapter. As we've explored, this section is highly technical, focusing on how to code your strategy so it can run on real-time data from a chosen data source. If you already have a strategy that you wish to test on a paper account, the best starting point is to thoroughly review the documentation provided by your broker. Brokers typically offer detailed information and numerous examples, often including templates that may closely align with the strategy you want to implement. These resources can be invaluable as you begin coding, modifying, and optimizing your strategy over time.

Whether you decide to pursue a fully algorithmic or fully automated approach is ultimately your decision as a trader. However, it's crucial to ensure that your strategy has been thoroughly backtested and proven robust under various market conditions and across multiple assets. Additionally, it's wise to test your strategy, whether it's a hybrid algorithmic model or fully automated, on a live paper account. This allows you to observe how it performs in real market conditions, helping to identify and resolve any edge cases or bugs that might only surface during live trading.

Chapter 12: Final Thoughts

1 Personal Experience, Lessons Learned, and Future Directions

From learning to trading, the journey to achieving success in algorithmic trading is not a quick one. It often takes years of dedication, experimentation, and learning to build a strategy that consistently delivers results. The reality is that unless you already possess a winning strategy with proven robustness across various markets, this path will require patience and persistence. However, the learning process itself is incredibly rewarding. I've found immense satisfaction in mastering AI, data science, coding in Python, C, and MetaTrader, and applying these skills to trading platforms.

Even though I had experience with these tools in my professional life, seeing them in action in the trading world was a different and exciting experience. It's important to remember, as we mentioned in the introduction, that even the legendary Medallion Fund by Renaissance Technologies took over a decade to start achieving substantial profits. Their returns, which average around 37% per year after taxes and fees, are a testament to the long and challenging road of algorithmic trading, even for the best in the business. So, it's crucial to approach this field with realistic expectations. Algorithmic trading is not a get-rich-quick scheme; it's a methodical process that requires careful study, testing, and refinement.

What makes this journey worthwhile is the sense of accomplishment and the intellectual curiosity that drives us to try different systems, discover where they succeed and fail, and continuously improve. Along the way, you'll also build a network of like-minded individuals, and as your skills develop, you may

even attract the attention of brokers, hedge funds, and financial firms looking for freelancers to help with tasks such as modeling, simulations, and strategy development. The expertise you gain in numerical trading can also be applied to other financial modeling tasks, helping you understand the difference between noise and signal, a critical skill in various fields.

However, it's essential to recognize that if you're only in this for the money, the odds of success are slim. The real key to thriving in algorithmic trading is to enjoy the process. When you’re passionate about learning, experimenting, and solving complex problems, the chances of success increase significantly.

2 The Intersection of Coding, Trading, and Learning

Algorithmic trading is one of the most challenging fields you can enter. While a price chart may seem simple, with time on one axis and price on the other, the reality is far more complex. The market is influenced by countless factors, from weather conditions affecting commodity prices to geopolitical events causing fluctuations in currency and stock markets. All these variables create a highly nonlinear problem where market conditions can shift rapidly and unpredictably.

Understanding and accepting the complexity of the market is crucial to avoid frustration. Developing a successful trading strategy is a long-term endeavor that may take years to perfect. You're not just using mathematical skills but also trading expertise, psychological control, mental focus, coding abilities, and the willingness to keep learning. Continuous education is vital because the landscape of trading strategies and methodologies is always evolving. By staying curious and open to new ideas, you'll be better equipped to develop a strategy that works.

3 The Role of AI in the Future of Algorithmic Trading

AI is becoming increasingly prevalent in the trading world, thanks in part to advancements like ChatGPT and other language models. However, incorporating AI into trading strategies is not as straightforward as it may seem. AI brings its own set of challenges, including a steep learning curve, data preparation requirements, and the complexity of choosing the right models

for specific tasks. Simply feeding candle data into an AI model often doesn't yield profitable results.

In my experience, mastering traditional trading methods is essential before moving on to AI. AI should be used to enhance an already profitable strategy, perhaps improving profitability by a few percentage points. This improvement alone can be significant.

If you're considering diving into AI for more complex models, be prepared for another year or two of learning. Although AI is becoming more accessible thanks to Python libraries and packages, you still need to understand data preparation and model selection to achieve your goals.

That said, there are smarter ways to integrate AI into trading without trying to predict the market directly. For example, AI can scrape news data from financial websites, providing a quick summary or sentiment analysis of market conditions. This approach, while complex, is feasible and can significantly enhance your trading decisions.

4 How to Stay Updated in the Fast-Paced World of Algorithmic Trading

Staying up-to-date in the fast-paced world of algorithmic trading is crucial, especially with the rapid advancements in AI. Joining social media groups where discussions are active, reading the latest books, and participating in online forums can help you stay informed. Choose groups where the focus aligns with your learning goals, whether it's technical analysis or fundamental insights.

5 Summary of Key Points

Throughout this book, we've covered the essential aspects of algorithmic trading, from the foundational concepts to advanced techniques like Monte Carlo simulations and AI integration. The key takeaway is that algorithmic trading is a complex, multifaceted discipline that requires continuous learning and adaptation. Whether you're optimizing a trading strategy, incorporating AI, or simply staying informed about market trends, the journey is ongoing.

6 Final Thoughts

Algorithmic trading isn't just about making profits; it's about embracing the intellectual challenge, finding satisfaction in solving complex problems, and taking joy in continuous learning. As you advance in this field, it's essential to remember that success is not just about financial gains, it's about the knowledge and skills you develop along the way, which ultimately pave the path to financial success. I know this might sound idealistic, but it's an honest truth. I began coding strategies over a decade ago, driven by the sheer excitement of tackling a tough challenge and fueled by a bet with a friend who once said, "If it were possible, anyone could do it." The reality is that it *is* possible, but not everyone has the consistency, discipline, and focus to pursue it over the years. Stay curious, keep experimenting, and most importantly, enjoy the journey. The path to success in algorithmic trading is long and demanding, but for those who are passionate about learning and innovation, it's a journey that is deeply rewarding.

7 Additional Features

If you enjoy video tutorials, backtesting exercises, and

strategies like those we've explored in this book, along with new indicators and a broader perspective, I invite you to visit my YouTube channel, @CodeTradingCafe. I started this channel during the COVID lockdown, initially as a way to continue teaching algorithmic and coding skills to my students while the university was closed. It allowed them to watch the videos on their own time, whenever it was convenient. What began as a temporary solution quickly grew into something much bigger, a vibrant community of learners from all over the world, united by their passion for algorithmic trading and coding challenges. So, if you're excited about trading, love to code, and enjoy a good brain teaser, grab a coffee and join us at Code Trading Café! https://www.youtube.com/@CodeTradingCafe

The code snippets used in this book are also available in a Jupyter notebook at the following link:

https://github.com/ZiadFrancis/AlgorithmicTradingHandsOnCodes/blob/main/Codes_snippets.ipynb

Glad you made it this far and I hope you enjoyed this small journey!

ABOUT THE AUTHOR

My name is Ziad Francis, and I hold a PhD in Radiation Physics and Numerical Modelling. Over the past twenty years, I have specialized in Monte Carlo simulations and stochastic processes, working across diverse industries to implement practical artificial intelligence (AI) solutions. Throughout my career, I've had the opportunity to lead research projects in fields such as medicine, radiology, radiation therapy, and financial modelling, where I've focused on translating complex AI techniques into accessible tools.

Alongside my consulting activities, I am also a university

professor, teaching Data Science, Artificial Intelligence, and numerical modelling. My research focuses primarily on numerical and computational methods, and I have published more than 70 scientific papers in internationally recognized, peer-reviewed journals.

Additionally, I have a keen interest in financial technology, particularly algorithmic trading and automation. I believe strongly that algorithmic methods, when thoughtfully structured, can effectively emulate the decision-making processes of experienced traders, offering practical insights into market dynamics.

I hope you enjoy reading this book and find the learning process as rewarding and enjoyable as I have.